

Counters

CHAPTER OUTLINE

- 9–1 Finite State Machines
- 9–2 Asynchronous Counters
- 9–3 Synchronous Counters
- 9–4 Up/Down Synchronous Counters
- 9–5 Design of Synchronous Counters
- 9–6 Cascaded Counters
- 9–7 Counter Decoding
- 9–8 Counter Applications
- 9–9 Logic Symbols with Dependency Notation
- 9–10 Troubleshooting
Applied Logic

CHAPTER OBJECTIVES

- Discuss the types of state machines
- Describe the difference between an asynchronous and a synchronous counter
- Analyze counter timing diagrams
- Analyze counter circuits
- Explain how propagation delays affect the operation of a counter
- Determine the modulus of a counter
- Modify the modulus of a counter
- Recognize the difference between a 4-bit binary counter and a decade counter
- Use an up/down counter to generate forward and reverse binary sequences
- Determine the sequence of a counter
- Use IC counters in various applications
- Design a counter that will have any specified sequence of states
- Use cascaded counters to achieve a higher modulus
- Use logic gates to decode any given state of a counter
- Eliminate glitches in counter decoding

- Explain how a digital clock operates
- Interpret counter logic symbols that use dependency notation
- Troubleshoot counters for various types of faults

KEY TERMS

Key terms are in order of appearance in the chapter.

- | | |
|-----------------|------------------|
| ■ State machine | ■ Synchronous |
| ■ Asynchronous | ■ Terminal count |
| ■ Recycle | ■ State diagram |
| ■ Modulus | ■ Cascade |
| ■ Decade | |

VISIT THE WEBSITE

Study aids for this chapter are available at <http://www.pearsonglobaleditions.com/floyd>

INTRODUCTION

As you learned in Chapter 7, flip-flops can be connected together to perform counting operations. Such a group of flip-flops is a counter, which is a type of finite state machine. The number of flip-flops used and the way in which they are connected determine the number of states (called the modulus) and also the specific sequence of states that the counter goes through during each complete cycle.

Counters are classified into two broad categories according to the way they are clocked: asynchronous and synchronous. In asynchronous counters, commonly called *ripple counters*, the first flip-flop is clocked by the external clock pulse and then each successive flip-flop is clocked by the output of the preceding flip-flop. In synchronous counters, the clock input is connected to all of the flip-flops so that they are clocked simultaneously. Within each of these two categories, counters are classified primarily by the type of sequence, the number of states, or the number of flip-flops in the counter. VHDL codes for various types of counters are presented.

9-1 Finite State Machines

A **state machine** is a sequential circuit having a limited (finite) number of states occurring in a prescribed order. A counter is an example of a state machine; the number of states is called the *modulus*. Two basic types of state machines are the Moore and the Mealy. The **Moore state machine** is one where the outputs depend only on the internal present state. The **Mealy state machine** is one where the outputs depend on both the internal present state and on the inputs. Both types have a timing input (clock) that is not considered a controlling input. A design approach to counters is presented in this section.

After completing this section, you should be able to

- ♦ Describe a Moore state machine
- ♦ Describe a Mealy state machine
- ♦ Discuss examples of Moore and Mealy state machines

General Models of Finite State Machines

A Moore state machine consists of combinational logic that determines the sequence and memory (flip-flops), as shown in Figure 9-1(a). A Mealy state machine is shown in part (b).

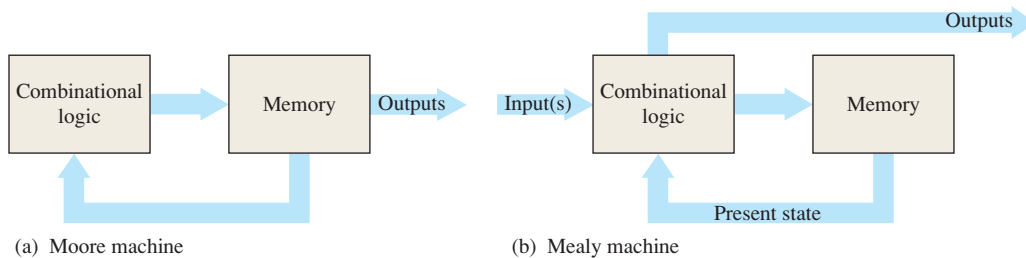


FIGURE 9-1 Two types of sequential logic.

In the Moore machine, the combinational logic is a gate array with outputs that determine the next state of the flip-flops in the memory. There may or may not be inputs to the combinational logic. There may also be output combinational logic, such as a decoder. If there is an input(s), it does not affect the outputs because they always correspond to and are dependent only on the present state of the memory. For the Mealy machine, the present state affects the outputs, just as in the Moore machine; but in addition, the inputs also affect the outputs. The outputs come directly from the combinational logic and not the memory.

Example of a Moore Machine

Figure 9-2(a) shows a Moore machine (modulus-26 binary counter with states 0 through 25) that is used to control the number of tablets (25) that go into each bottle in an assembly line. When the binary number in the memory (flip-flops) reaches binary twenty-five (11001), the counter recycles to 0 and the tablet flow and clock are cut off until the next bottle is in place. The combinational logic for the state transitions sets the modulus of the counter so that it sequences from binary state 0 to binary state 25, where 0 is the reset or rest state and the output combinational logic decodes binary state 25. There is no input in this case, other than the clock, so the next state is determined only by the present state, which makes this a Moore machine. One tablet is bottled for each clock pulse. Once a bottle is in place, the first tablet is inserted at binary state 1, the second at binary state 2, and the twenty-fifth tablet when the binary state is 25. Count 25 is decoded and used to stop the flow of tablets and the clock. The counter stays in the 0 state until the next bottle is in position (indicated by a 1). Then the clock resumes, the count goes to 1, and the cycle repeats, as illustrated by the state diagram in Figure 9-2(b).

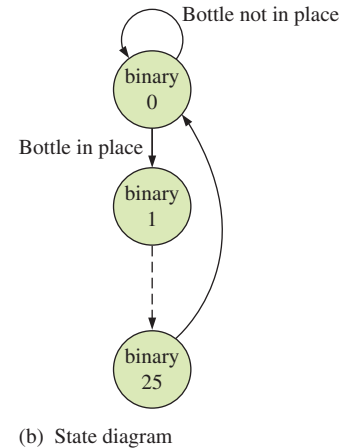
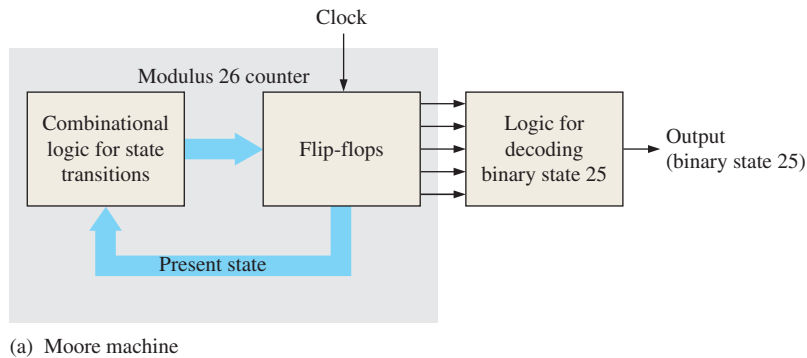


FIGURE 9-2 A fixed-modulus binary counter as an example of a Moore state machine. The dashed line in the state diagram means the states between binary 1 and 25 are not shown for simplicity.

Example of a Mealy Machine

Let's assume that the tablet-bottling system uses three different sizes of bottles: a 25-tablet bottle, a 50-tablet bottle, and a 100-tablet bottle. This operation requires a state machine with three different terminal counts: 25, 50, and 100. One approach is illustrated in Figure 9-3(a). The combinational logic sets the modulus of the counter depending on the modulus-select inputs. The output of the counter depends on both the present state and the modulus-select inputs, making this a Mealy machine. The state diagram is shown in part (b).

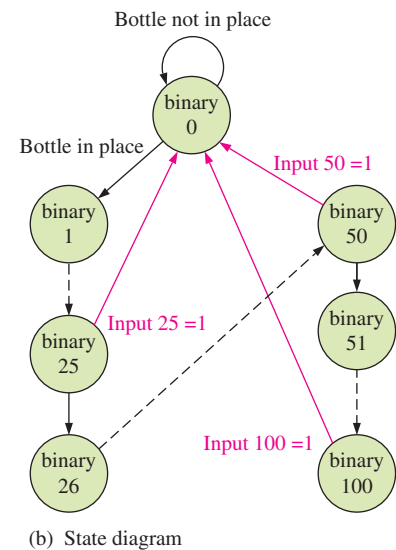
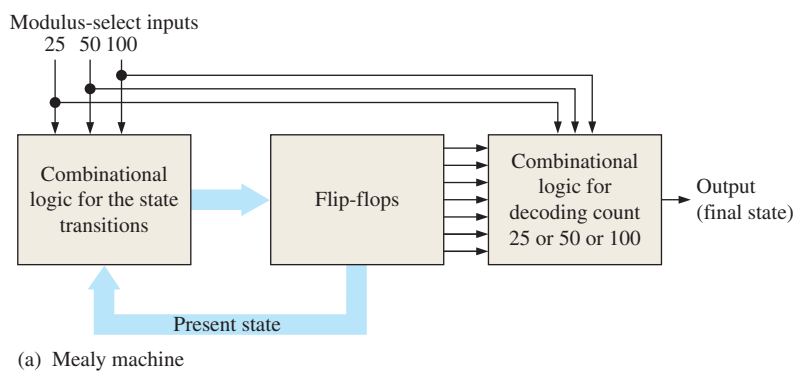


FIGURE 9-3 A variable-modulus binary counter as an example of a Mealy state machine. The red arrows in the state diagram represent the recycle paths that depend on the input number. The black dashed lines mean the interim states are not shown for simplicity.

SECTION 9-1 CHECKUP

Answers are at the end of the chapter.

1. What characterizes a finite state machine?
2. Name the types of finite state machines.
3. Explain the difference between the two types of state machines.

9-2 Asynchronous Counters

The term **asynchronous** refers to events that do not have a fixed time relationship with each other and, generally, do not occur at the same time. An **asynchronous counter** is one in which the flip-flops (FF) within the counter do not change states at exactly the same time because they do not have a common clock pulse.

After completing this section, you should be able to

- ♦ Describe the operation of a 2-bit asynchronous binary counter
- ♦ Describe the operation of a 3-bit asynchronous binary counter
- ♦ Define *ripple* in relation to counters
- ♦ Describe the operation of an asynchronous decade counter
- ♦ Develop counter timing diagrams
- ♦ Discuss the implementation of a 4-bit asynchronous binary counter

A 2-Bit Asynchronous Binary Counter

The clock input of an asynchronous counter is always connected only to the LSB flip-flop.

Figure 9-4 shows a 2-bit counter connected for asynchronous operation. Notice that the clock (CLK) is applied to the clock input (C) of *only* the first flip-flop, FF0, which is always the least significant bit (LSB). The second flip-flop, FF1, is triggered by the $\bar{Q}_0$ output of FF0. FF0 changes state at the positive-going edge of each clock pulse, but FF1 changes only when triggered by a positive-going transition of the $\bar{Q}_0$ output of FF0. Because of the inherent propagation delay time through a flip-flop, a transition of the input clock pulse (CLK) and a transition of the $\bar{Q}_0$ output of FF0 can never occur at exactly the same time. Therefore, the two flip-flops are never simultaneously triggered, so the counter operation is asynchronous.

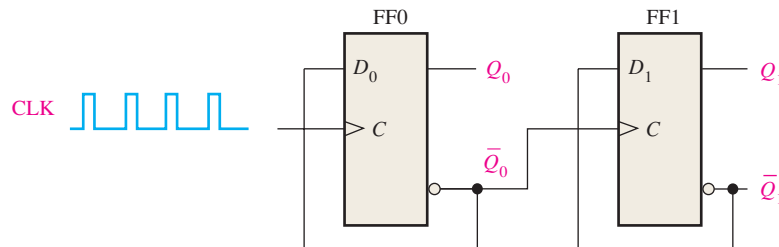


FIGURE 9-4 A 2-bit asynchronous binary counter. Open file F09-04 to verify operation. A Multisim tutorial is available on the website.

The Timing Diagram

Let's examine the basic operation of the asynchronous counter of Figure 9-4 by applying four clock pulses to FF0 and observing the Q output of each flip-flop. Figure 9-5 illustrates the changes in the state of the flip-flop outputs in response to the clock pulses. Both flip-flops are connected for toggle operation ($D = \bar{Q}$) and are assumed to be initially RESET (Q LOW).

The positive-going edge of CLK1 (clock pulse 1) causes the Q_0 output of FF0 to go HIGH, as shown in Figure 9-5. At the same time the $\bar{Q}_0$ output goes LOW, but it has no effect on FF1 because a positive-going transition must occur to trigger the flip-flop. After the leading edge of CLK1, $Q_0 = 1$ and $Q_1 = 0$. The positive-going edge of CLK2 causes Q_0 to go LOW. Output $\bar{Q}_0$ goes HIGH and triggers FF1, causing Q_1 to go HIGH. After the leading edge of CLK2, $Q_0 = 0$ and $Q_1 = 1$. The positive-going edge of CLK3 causes Q_0 to go HIGH again. Output $\bar{Q}_0$ goes LOW and has no effect on FF1. Thus, after the leading edge of CLK3, $Q_0 = 1$ and $Q_1 = 1$. The positive-going edge of CLK4 causes Q_0 to go LOW, while $\bar{Q}_0$ goes HIGH and triggers FF1, causing Q_1 to go LOW. After the leading

Asynchronous counters are also known as ripple counters.

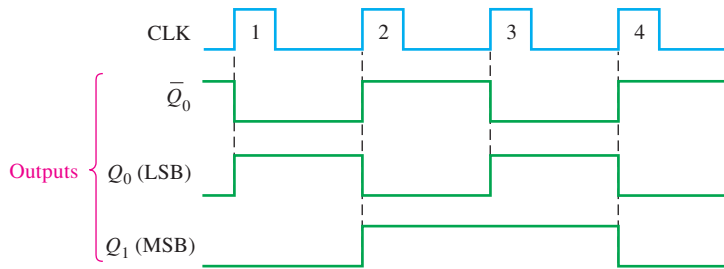


FIGURE 9-5 Timing diagram for the counter of Figure 9-4. As in previous chapters, output waveforms are shown in green.

edge of CLK4, $Q_0 = 0$ and $Q_1 = 0$. The counter has now recycled to its original state (both flip-flops are RESET).

In the timing diagram, the waveforms of the Q_0 and Q_1 outputs are shown relative to the clock pulses as illustrated in Figure 9-5. For simplicity, the transitions of Q_0 , Q_1 , and the clock pulses are shown as simultaneous even though this is an asynchronous counter. There is, of course, some small delay between the CLK and the Q_0 transition and between the $\bar{Q}_0$ transition and the Q_1 transition.

Note in Figure 9-5 that the 2-bit counter exhibits four different states, as you would expect with two flip-flops ($2^2 = 4$). Also, notice that if Q_0 represents the least significant bit (LSB) and Q_1 represents the most significant bit (MSB), the sequence of counter states represents a sequence of binary numbers as listed in Table 9-1.

In digital logic, Q_0 is always the LSB unless otherwise specified.

TABLE 9-1

Binary state sequence for the counter in Figure 9-4.

Clock Pulse	Q_1	Q_0
Initially	0	0
1	0	1
2	1	0
3	1	1
4 (recycles)	0	0

Since it goes through a binary sequence, the counter in Figure 9-4 is a binary counter. It actually counts the number of clock pulses up to three, and on the fourth pulse it recycles to its original state ($Q_0 = 0$, $Q_1 = 0$). The term **recycle** is commonly applied to counter operation; it refers to the transition of the counter from its final state back to its original state.

A 3-Bit Asynchronous Binary Counter

The state sequence for a 3-bit binary counter is listed in Table 9-2, and a 3-bit asynchronous binary counter is shown in Figure 9-6(a). The basic operation is the same as that of the 2-bit

TABLE 9-2

State sequence for a 3-bit binary counter.

Clock Pulse	Q_2	Q_1	Q_0
Initially	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8 (recycles)	0	0	0

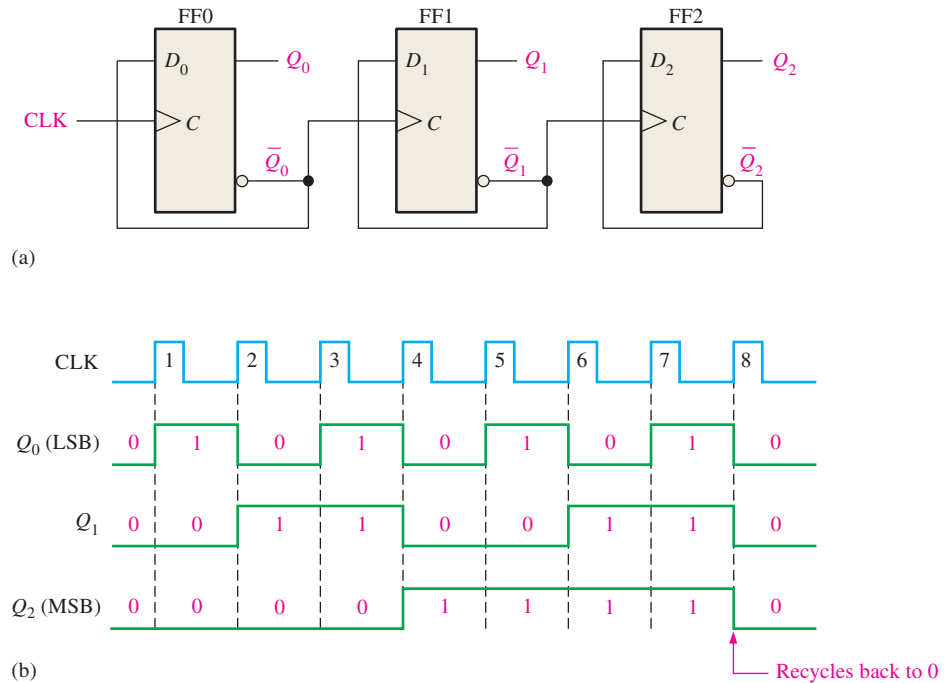


FIGURE 9-6 Three-bit asynchronous binary counter and its timing diagram for one cycle. Open file F09-06 to verify operation.

counter except that the 3-bit counter has eight states, due to its three flip-flops. A timing diagram is shown in Figure 9-6(b) for eight clock pulses. Notice that the counter progresses through a binary count of zero through seven and then recycles to the zero state. This counter can be easily expanded for higher count, by connecting additional toggle flip-flops.

Propagation Delay

Asynchronous counters are commonly referred to as **ripple counters** for the following reason: The effect of the input clock pulse is first “felt” by FF0. This effect cannot get to FF1 immediately because of the propagation delay through FF0. Then there is the propagation delay through FF1 before FF2 can be triggered. Thus, the effect of an input clock pulse “ripples” through the counter, taking some time, due to propagation delays, to reach the last flip-flop.

To illustrate, notice that all three flip-flops in the counter of Figure 9-6 change state on the leading edge of CLK4. This ripple clocking effect is shown in Figure 9-7 for the first four clock pulses, with the propagation delays indicated. The LOW-to-HIGH transition of

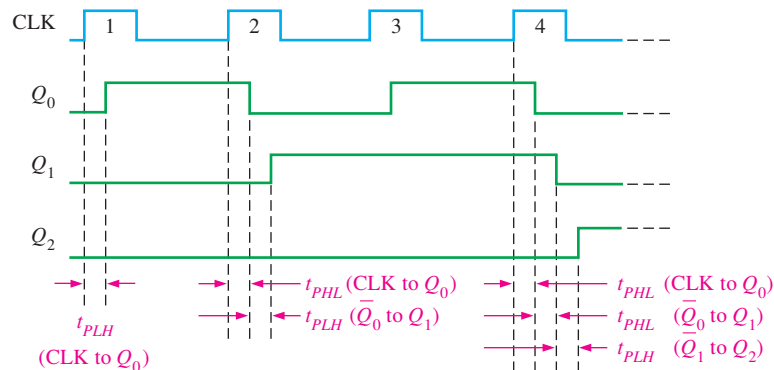


FIGURE 9-7 Propagation delays in a 3-bit asynchronous (ripple-clocked) binary counter.

Q_0 occurs one delay time (t_{PLH}) after the positive-going transition of the clock pulse. The LOW-to-HIGH transition of Q_1 occurs one delay time (t_{PLH}) after the positive-going transition of $\bar{Q}_0$. The LOW-to-HIGH transition of Q_2 occurs one delay time (t_{PLH}) after the positive-going transition of $\bar{Q}_1$. As you can see, FF2 is not triggered until two delay times after the positive-going edge of the clock pulse, CLK4. Thus, it takes three propagation delay times for the effect of the clock pulse, CLK4, to ripple through the counter and change Q_2 from LOW to HIGH.

This cumulative delay of an asynchronous counter is a major disadvantage in many applications because it limits the rate at which the counter can be clocked and creates decoding problems. The maximum cumulative delay in a counter must be less than the period of the clock waveform.

EXAMPLE 9-1

A 4-bit asynchronous binary counter is shown in Figure 9-8(a). Each D flip-flop is negative edge-triggered and has a propagation delay for 10 nanoseconds (ns). Develop a timing diagram showing the Q output of each flip-flop, and determine the total propagation delay time from the triggering edge of a clock pulse until a corresponding change can occur in the state of Q_3 . Also determine the maximum clock frequency at which the counter can be operated.

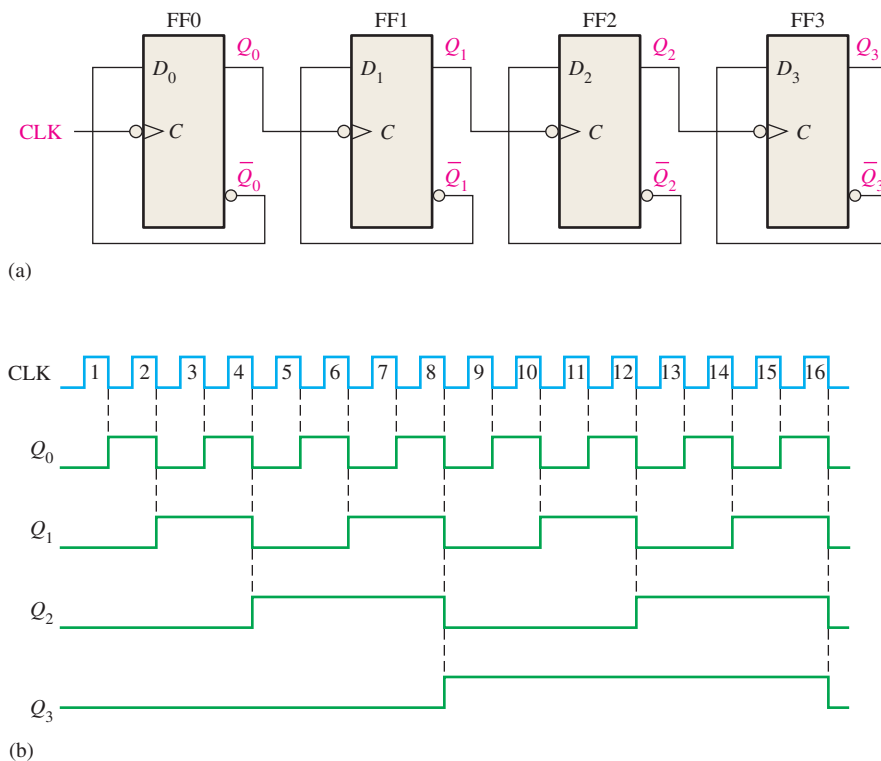


FIGURE 9-8 Four-bit asynchronous binary counter and its timing diagram. Open file F09-08 and verify the operation.



Solution

The timing diagram with delays omitted is as shown in Figure 9-8(b). For the total delay time, the effect of CLK8 or CLK16 must propagate through four flip-flops before Q_3 changes, so

$$t_{p(\text{tot})} = 4 \times 10 \text{ ns} = \mathbf{40 \text{ ns}}$$

The maximum clock frequency is

$$f_{\max} = \frac{1}{t_{p(\text{tot})}} = \frac{1}{40 \text{ ns}} = \mathbf{25 \text{ MHz}}$$

The counter should be operated below this frequency to avoid problems due to the propagation delay.

Related Problem*

Show the timing diagram if all of the flip-flops in Figure 9–8(a) are positive edge-triggered.

*Answers are at the end of the chapter.

A counter can have 2^n states, where n is the number of flip-flops.

Asynchronous Decade Counters

The **modulus** of a counter is the number of unique states through which the counter will sequence. The maximum possible number of states (maximum modulus) of a counter is 2^n , where n is the number of flip-flops in the counter. Counters can be designed to have a number of states in their sequence that is less than the maximum of 2^n . This type of sequence is called a *truncated sequence*.

One common modulus for counters with truncated sequences is ten (called MOD10). Counters with ten states in their sequence are called **decade** counters. A **decade counter** with a count sequence of zero (0000) through nine (1001) is a BCD decade counter because its ten-state sequence produces the BCD code. This type of counter is useful in display applications in which BCD is required for conversion to a decimal readout.

To obtain a truncated sequence, it is necessary to force the counter to recycle before going through all of its possible states. For example, the BCD decade counter must recycle back to the 0000 state after the 1001 state. A decade counter requires four flip-flops (three flip-flops are insufficient because $2^3 = 8$).

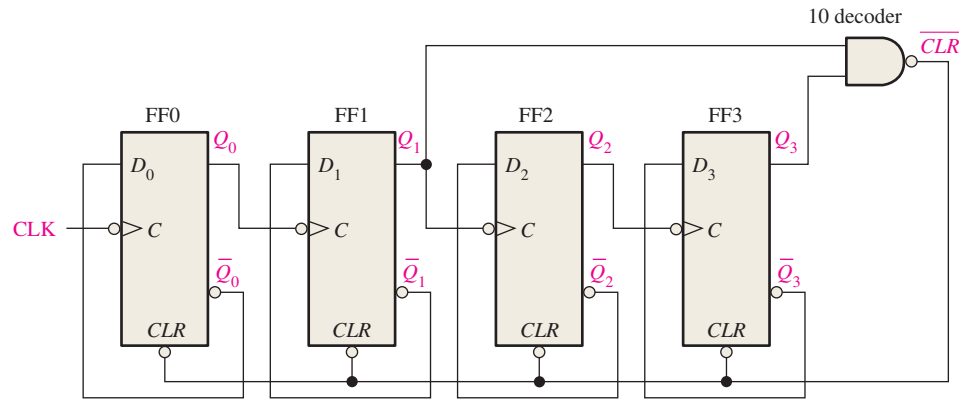
Let's use a 4-bit asynchronous counter such as the one in Example 9–1 and modify its sequence to illustrate the principle of truncated counters. One way to make the counter recycle after the count of nine (1001) is to decode count ten (1010) with a NAND gate and connect the output of the NAND gate to the clear ($\overline{CLR}$) inputs of the flip-flops, as shown in Figure 9–9(a).

Partial Decoding

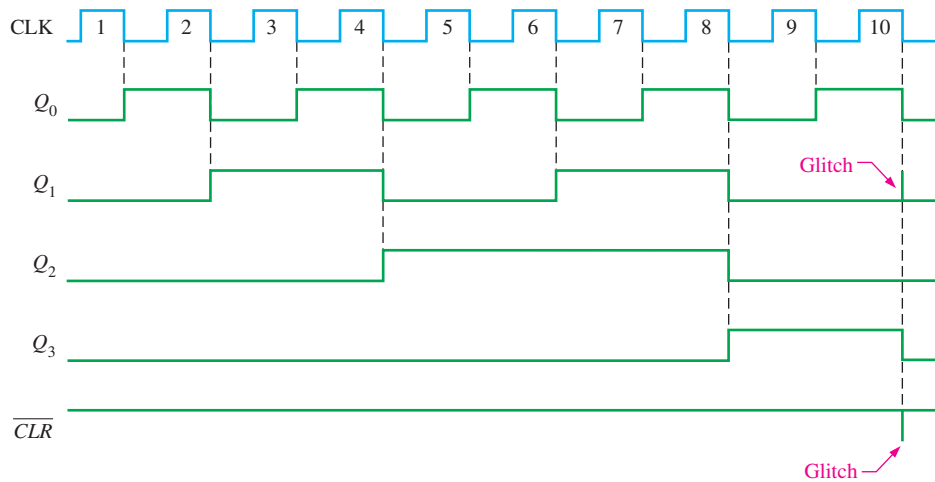
Notice in Figure 9–9(a) that only Q_1 and Q_3 are connected to the NAND gate inputs. This arrangement is an example of *partial decoding*, in which the two unique states ($Q_1 = 1$ and $Q_3 = 1$) are sufficient to decode the count of ten because none of the other states (zero through nine) have both Q_1 and Q_3 HIGH at the same time. When the counter goes into count ten (1010), the decoding gate output goes LOW and asynchronously resets all the flip-flops.

The resulting timing diagram is shown in Figure 9–9(b). Notice that there is a glitch on the Q_1 waveform. The reason for this glitch is that Q_1 must first go HIGH before the count of ten can be decoded. Not until several nanoseconds after the counter goes to the count of ten does the output of the decoding gate go LOW (both inputs are HIGH). Thus, the counter is in the 1010 state for a short time before it is reset to 0000, thus producing the glitch on Q_1 and the resulting glitch on the $\overline{CLR}$ line that resets the counter.

Other truncated sequences can be implemented in a similar way, as Example 9–2 shows.



(a)



(b)

FIGURE 9-9 An asynchronously clocked decade counter with asynchronous recycling.**EXAMPLE 9-2**

Show how an asynchronous counter with J-K flip-flops can be implemented having a modulus of twelve with a straight binary sequence from 0000 through 1011.

Solution

Since three flip-flops can produce a maximum of eight states, four flip-flops are required to produce any modulus greater than eight but less than or equal to sixteen.

When the counter gets to its last state, 1011, it must recycle back to 0000 rather than going to its normal next state of 1100, as illustrated in the following sequence chart:

Q_3	Q_2	Q_1	Q_0	
0	0	0	0	← Recycles
·	·	·	·	
·	·	·	·	
·	·	·	·	
1	0	1	1	← Normal next state
1	1	0	0	

Observe that Q_0 and Q_1 both go to 0 anyway, but Q_2 and Q_3 must be forced to 0 on the twelfth clock pulse. Figure 9-10(a) shows the modulus-12 counter. The NAND gate partially decodes count twelve (1100) and resets flip-flop 2 and flip-flop 3.

Thus, on the twelfth clock pulse, the counter is forced to recycle from count eleven to count zero, as shown in the timing diagram of Figure 9–10(b). (It is in count twelve for only a few nanoseconds before it is reset by the glitch on $\overline{CLR}$.)

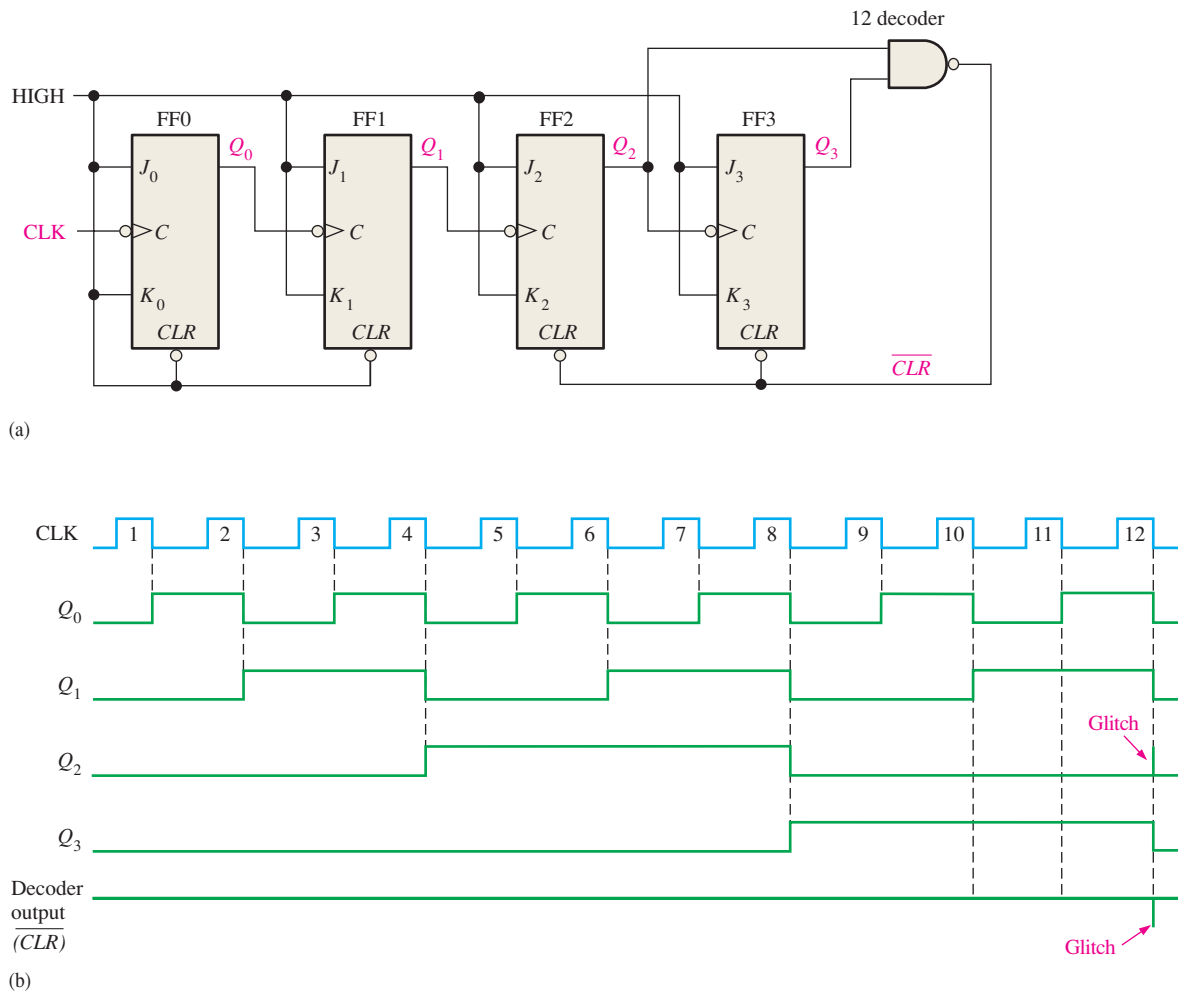
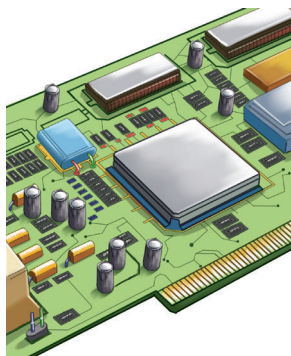


FIGURE 9-10 Asynchronously clocked modulus-12 counter with asynchronous recycling.

Related Problem

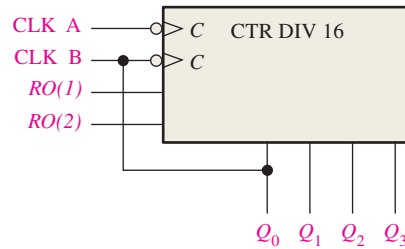
How can the counter in Figure 9–10(a) be modified to make it a modulus-13 counter?

IMPLEMENTATION: 4-BIT ASYNCHRONOUS BINARY COUNTER

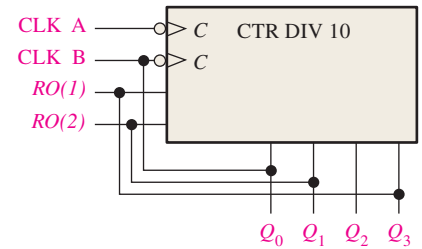


Fixed-Function Device The 74HC93 is an example of a specific integrated circuit asynchronous counter. This device actually consists of a single flip-flop (CLK A) and a 3-bit asynchronous counter (CLK B). This arrangement is for flexibility. It can be used as a divide-by-2 device if only the single flip-flop is used, or it can be used as a modulus-8 counter if only the 3-bit counter portion is used. This device also provides gated reset inputs, $RO(1)$ and $RO(2)$. When both of these inputs are HIGH, the counter is reset to the 0000 state $\overline{CLR}$.

Additionally, the 74HC93 can be used as a 4-bit modulus-16 counter (counts 0 through 15) by connecting the Q_0 output to the CLK B input as shown by the logic symbol in Figure 9–11(a). It can also be configured as a decade counter (counts 0 through 9) with asynchronous recycling by using the gated reset inputs for partial decoding of count ten, as shown by the logic symbol in Figure 9–11(b).



(a) 74HC93 connected as a modulus-16 counter



(b) 74HC93 connected as a decade counter

FIGURE 9-11 Two configurations of the 74HC93 asynchronous counter. (The qualifying label, CTR DIV n , indicates a counter with n states.)



Programmable Logic Device (PLD) The VHDL code for a generic 4-bit asynchronous binary counter using J-K flip flops with preset (PRN) and clear (CLR_N) inputs is as follows:

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity AsyncFourBitBinCntr is
```

```
    port (Clock, Clr: in std_logic; Q0, Q1, Q2, Q3: inout std_logic);
```

Inputs and outputs declared

```
end entity AsyncFourBitBinCntr;
```

```
architecture LogicOperation of AsyncFourBitBinCntr is
```

```
    component jkff is
```

```
        port (J, K, Clk, PRN, CLRN: in std_logic; Q: out std_logic);
```

J-K flip-flop component declaration

```
    end component jkff;
```

```
begin
```

```
    FF0: jkff port map(J=>'1', K=>'1', Clk=>Clock, CLRN=>Clr, PRN=>'1', Q=>Q0);
```

```
    FF1: jkff port map(J=>'1', K=>'1', Clk=>not Q0, CLRN=>Clr, PRN=>'1', Q=>Q1);
```

```
    FF2: jkff port map(J=>'1', K=>'1', Clk=>not Q1, CLRN=>Clr, PRN=>'1', Q=>Q2);
```

```
    FF3: jkff port map(J=>'1', K=>'1', Clk=>not Q2, CLRN=>Clr, PRN=>'1', Q=>Q3);
```

Instantiations define how each flip-flop is connected.

```
end architecture LogicOperation;
```

SECTION 9-2 CHECKUP

1. What does the term *asynchronous* mean in relation to counters?
2. How many states does a modulus-14 counter have? What is the minimum number of flip-flops required?

9-3 Synchronous Counters

The term **synchronous** refers to events that have a fixed time relationship with each other. A **synchronous counter** is one in which all the flip-flops in the counter are clocked at the same time by a common clock pulse. J-K flip-flops are used to illustrate most synchronous counters. D flip-flops can also be used but generally require more logic because of having no direct toggle or no-change states.

After completing this section, you should be able to

- ♦ Describe the operation of a 2-bit synchronous binary counter
- ♦ Describe the operation of a 3-bit synchronous binary counter
- ♦ Describe the operation of a 4-bit synchronous binary counter
- ♦ Describe the operation of a synchronous decade counter
- ♦ Develop counter timing diagrams

A 2-Bit Synchronous Binary Counter

Figure 9–12 shows a 2-bit synchronous binary counter. Notice that an arrangement different from that for the asynchronous counter must be used for the J_1 and K_1 inputs of FF1 in order to achieve a binary sequence. A D flip-flop implementation is shown in part (b).

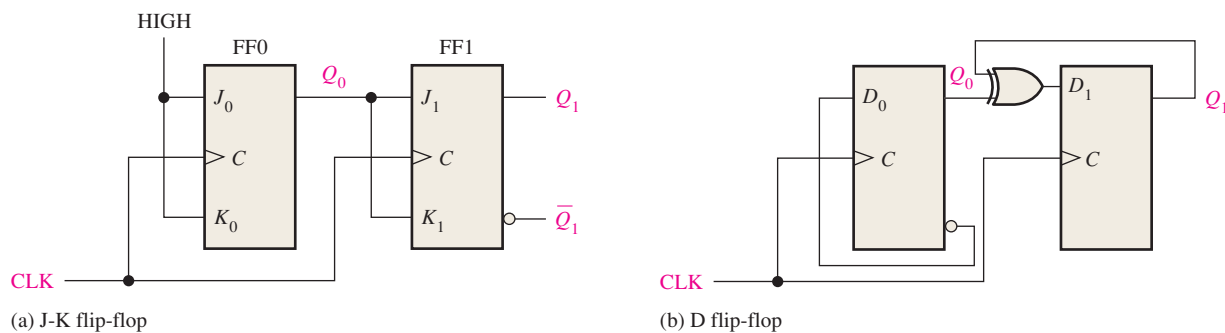


FIGURE 9-12 2-bit synchronous binary counters.

The clock input goes to each flip-flop in a synchronous counter.

The operation of a J-K flip-flop synchronous counter is as follows: First, assume that the counter is initially in the binary 0 state; that is, both flip-flops are RESET. When the positive edge of the first clock pulse is applied, FF0 will toggle and Q_0 will therefore go HIGH. What happens to FF1 at the positive-going edge of CLK1? To find out, let's look at the input conditions of FF1. Inputs J_1 and K_1 are both LOW because Q_0 , to which they are connected, has not yet gone HIGH. Remember, there is a propagation delay from the triggering edge of the clock pulse until the Q output actually makes a transition. So, $J = 0$ and $K = 0$ when the leading edge of the first clock pulse is applied. This is a no-change condition, and therefore FF1 does not change state. A timing detail of this portion of the counter operation is shown in Figure 9–13(a).

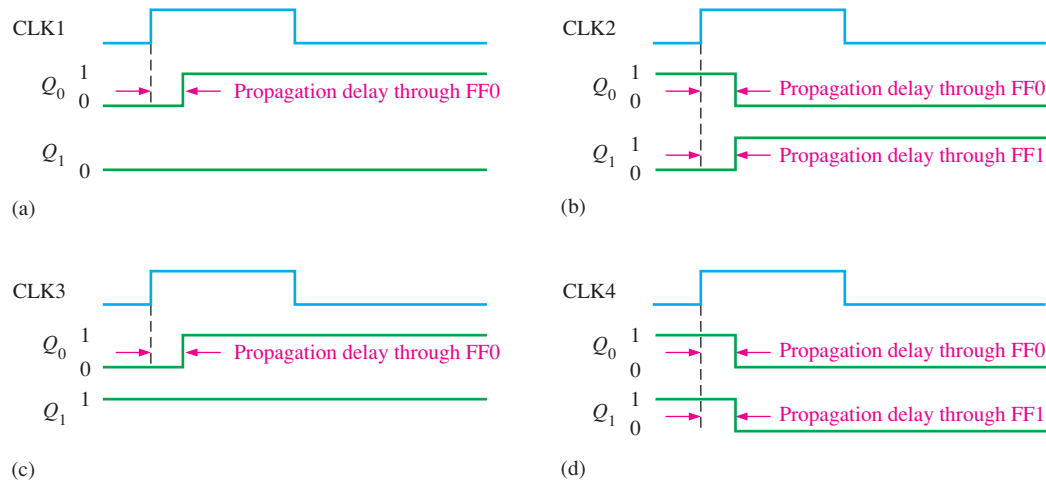


FIGURE 9-13 Timing details for the 2-bit synchronous counter operation (the propagation delays of both flip-flops are assumed to be equal).

After CLK1, $Q_0 = 1$ and $Q_1 = 0$ (which is the binary 1 state). When the leading edge of CLK2 occurs, FF0 will toggle and Q_0 will go LOW. Since FF1 has a HIGH ($Q_0 = 1$) on its J_1 and K_1 inputs at the triggering edge of this clock pulse, the flip-flop toggles and Q_1 goes HIGH. Thus, after CLK2, $Q_0 = 0$ and $Q_1 = 1$ (which is a binary 2 state). The timing detail for this condition is shown in Figure 9–13(b).

When the leading edge of CLK3 occurs, FF0 again toggles to the SET state ($Q_0 = 1$), and FF1 remains SET ($Q_1 = 1$) because its J_1 and K_1 inputs are both LOW ($Q_0 = 0$). After this triggering edge, $Q_0 = 1$ and $Q_1 = 1$ (which is a binary 3 state). The timing detail is shown in Figure 9–13(c).

Finally, at the leading edge of CLK4, Q_0 and Q_1 go LOW because they both have a toggle condition on their J and K inputs. The timing detail is shown in Figure 9–13(d). The counter has now recycled to its original state, binary 0. Examination of the D flip-flop counter in Figure 9–12(b) will show the timing diagram is the same as for the J-K flip-flop counter.

The complete timing diagram for the counters in Figure 9–12 is shown in Figure 9–14. Notice that all the waveform transitions appear coincident; that is, the propagation delays are not indicated. Although the delays are an important factor in the synchronous counter operation, in an overall timing diagram they are normally omitted for simplicity. Major waveform relationships resulting from the normal operation of a circuit can be conveyed completely without showing small delay and timing differences. However, in high-speed digital circuits, these small delays are an important consideration in design and troubleshooting.

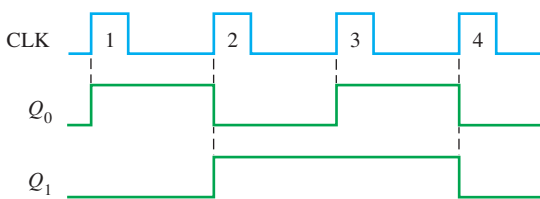


FIGURE 9–14 Timing diagram for the counters of Figure 9–12.

A 3-Bit Synchronous Binary Counter

A 3-bit synchronous binary counter is shown in Figure 9–15, and its timing diagram is shown in Figure 9–16. You can understand this counter operation by examining its sequence of states as shown in Table 9–3.

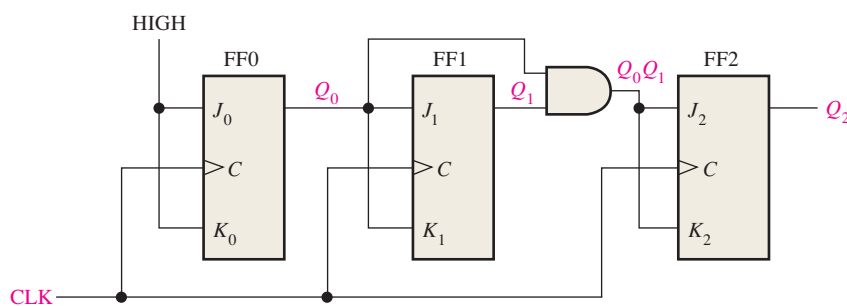


FIGURE 9–15 A 3-bit synchronous binary counter. Open file F09-15 to verify the operation.

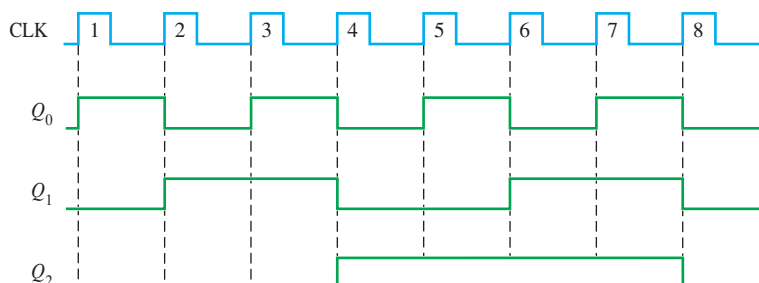


FIGURE 9–16 Timing diagram for the counter of Figure 9–15.

TABLE 9-3

State sequence for a 3-bit binary counter.

Clock Pulse	Q_2	Q_1	Q_0
Initially	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8 (recycles)	0	0	0

InfoNote

The TSC or *time stamp counter* in some microprocessors is used for performance monitoring, which enables a number of parameters important to the overall performance of a system to be determined exactly. By reading the TSC before and after the execution of a procedure, the precise time required for the procedure can be determined based on the processor cycle time. In this way, the TSC forms the basis for all time evaluations in connection with optimizing system operation. For example, it can be accurately determined which of two or more programming sequences is more efficient. This is a very useful tool for compiler developers and system programmers in producing the most effective code.

First, let's look at Q_0 . Notice that Q_0 changes on each clock pulse as the counter progresses from its original state to its final state and then back to its original state. To produce this operation, FF0 must be held in the toggle mode by constant HIGHs on its J_0 and K_0 inputs. Notice that Q_1 goes to the opposite state following each time Q_0 is a 1. This change occurs at CLK2, CLK4, CLK6, and CLK8. The CLK8 pulse causes the counter to recycle. To produce this operation, Q_0 is connected to the J_1 and K_1 inputs of FF1. When Q_0 is a 1 and a clock pulse occurs, FF1 is in the toggle mode and therefore changes state. The other times, when Q_0 is a 0, FF1 is in the no-change mode and remains in its present state.

Next, let's see how FF2 is made to change at the proper times according to the binary sequence. Notice that both times Q_2 changes state, it is preceded by the unique condition in which both Q_0 and Q_1 are HIGH. This condition is detected by the AND gate and applied to the J_2 and K_2 inputs of FF2. Whenever both Q_0 and Q_1 are HIGH, the output of the AND gate makes the J_2 and K_2 inputs of FF2 HIGH, and FF2 toggles on the following clock pulse. At all other times, the J_2 and K_2 inputs of FF2 are held LOW by the AND gate output, and FF2 does not change state.

The analysis of the counter in Figure 9-15 is summarized in Table 9-4.

TABLE 9-4

Summary of the analysis of the counter in Figure 9-15.

Clock Pulse	Outputs			J-K Inputs						At the Next Clock Pulse		
	Q_2	Q_1	Q_0	J_2	K_2	J_1	K_1	J_0	K_0	FF2	FF1	FF0
Initially	0	0	0	0	0	0	0	1	1	NC*	NC	Toggle
1	0	0	1	0	0	1	1	1	1	NC	Toggle	Toggle
2	0	1	0	0	0	0	0	1	1	NC	NC	Toggle
3	0	1	1	1	1	1	1	1	1	Toggle	Toggle	Toggle
4	1	0	0	0	0	0	0	1	1	NC	NC	Toggle
5	1	0	1	0	0	1	1	1	1	NC	Toggle	Toggle
6	1	1	0	0	0	0	0	1	1	NC	NC	Toggle
7	1	1	1	1	1	1	1	1	1	Toggle	Toggle	Toggle
Counter recycles back to 000.												

*NC indicates *No Change*.

A 4-Bit Synchronous Binary Counter

Figure 9-17(a) shows a 4-bit synchronous binary counter, and Figure 9-17(b) shows its timing diagram. This particular counter is implemented with negative edge-triggered flip-flops. The reasoning behind the J and K input control for the first three flip-flops is the same as previously discussed for the 3-bit counter. The fourth stage, FF3, changes only twice in the sequence. Notice that both of these transitions occur following the times that Q_0 , Q_1 , and Q_2 are all HIGH. This condition is decoded by AND gate G_2 so that when a

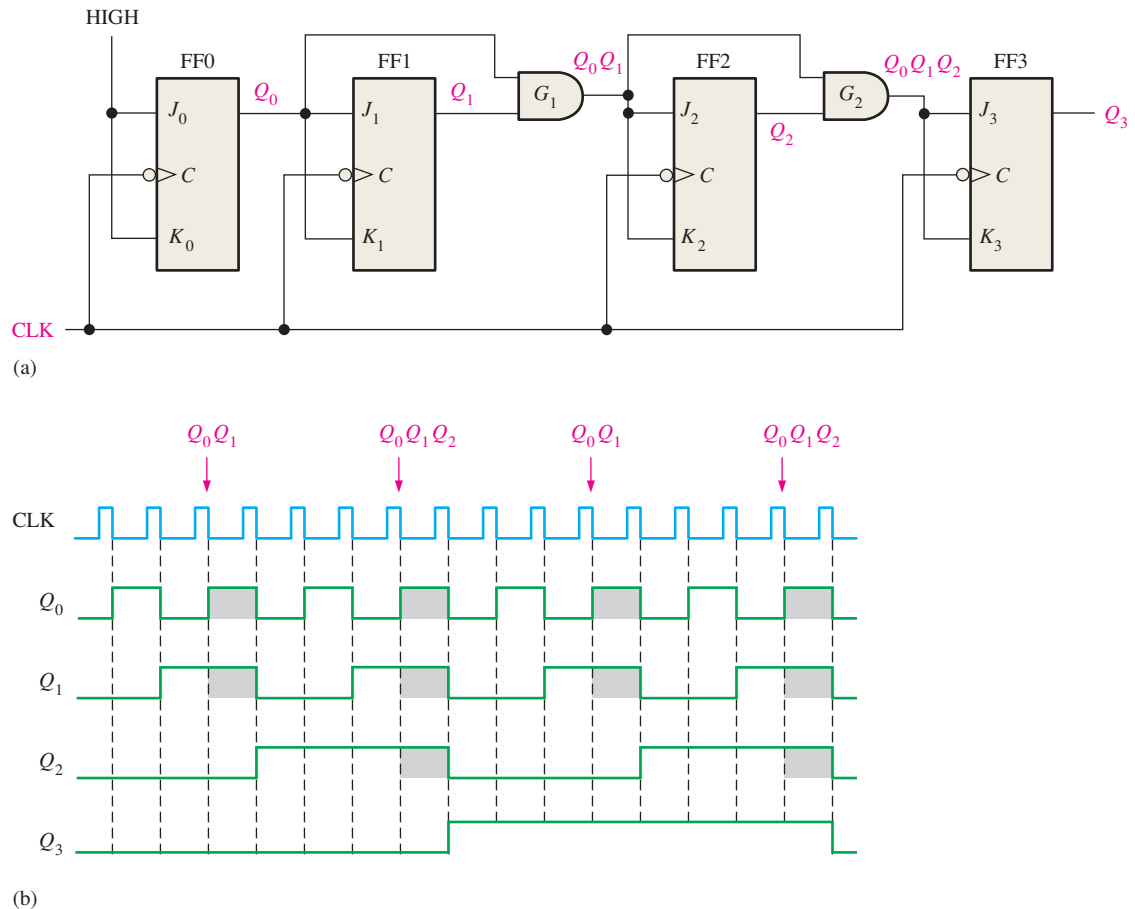


FIGURE 9-17 A 4-bit synchronous binary counter and timing diagram. Times where the AND gate outputs are HIGH are indicated by the shaded areas.

clock pulse occurs, FF3 will change state. For all other times the J_3 and K_3 inputs of FF3 are LOW, and it is in a no-change condition.

A 4-Bit Synchronous Decode Counter

As you know, a BCD decode counter exhibits a truncated binary sequence and goes from 0000 through the 1001 state. Rather than going from the 1001 state to the 1010 state, it recycles to the 0000 state. A synchronous BCD decode counter is shown in Figure 9-18. The timing diagram for the decode counter is shown in Figure 9-19.

A decade counter has ten states.

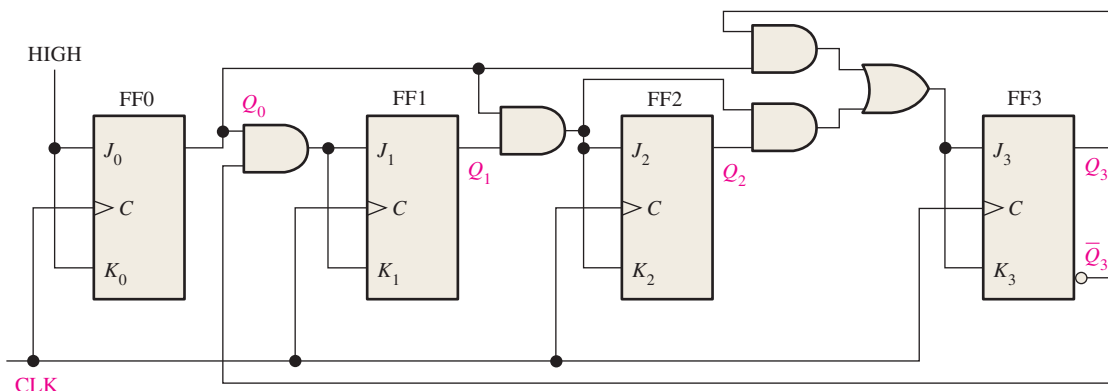


FIGURE 9-18 A synchronous BCD decode counter. Open file F09-18 to verify operation.

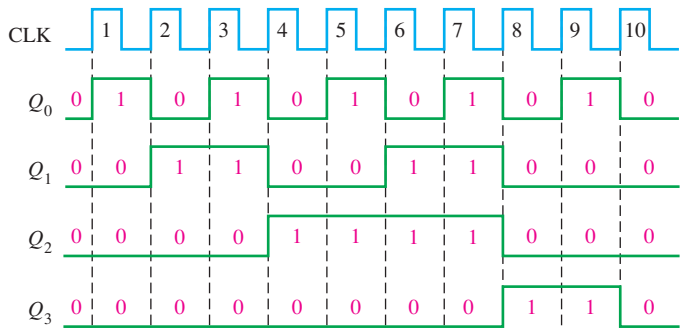


FIGURE 9-19 Timing diagram for the BCD decade counter (Q_0 is the LSB).

The counter operation is shown by the sequence of states in Table 9-5. First, notice that FF0 (Q_0) toggles on each clock pulse, so the logic equation for its J_0 and K_0 inputs is

$$J_0 = K_0 = 1$$

This equation is implemented by connecting J_0 and K_0 to a constant HIGH level.

TABLE 9-5				
States of a BCD decade counter.				
Clock Pulse	Q_3	Q_2	Q_1	Q_0
Initially	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10 (recycles)	0	0	0	0

Next, notice in Table 9-5 that FF1 (Q_1) changes on the next clock pulse each time $Q_0 = 1$ and $Q_3 = 0$, so the logic equation for the J_1 and K_1 inputs is

$$J_1 = K_1 = Q_0\overline{Q_3}$$

This equation is implemented by ANDing Q_0 and $\overline{Q_3}$ and connecting the gate output to the J_1 and K_1 inputs of FF1.

Flip-flop 2 (Q_2) changes on the next clock pulse each time both $Q_0 = 1$ and $Q_1 = 1$. This requires an input logic equation as follows:

$$J_2 = K_2 = Q_0Q_1$$

This equation is implemented by ANDing Q_0 and Q_1 and connecting the gate output to the J_2 and K_2 inputs of FF2.

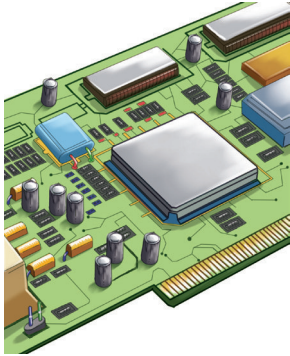
Finally, FF3 (Q_3) changes to the opposite state on the next clock pulse each time $Q_0 = 1$, $Q_1 = 1$, and $Q_2 = 1$ (state 7), or when $Q_0 = 1$ and $Q_3 = 1$ (state 9). The equation for this is as follows:

$$J_3 = K_3 = Q_0Q_1Q_2 + Q_0Q_3$$

This function is implemented with the AND/OR logic connected to the J_3 and K_3 inputs of FF3 as shown in the logic diagram in Figure 9-18. Notice that the differences between this

decade counter and the modulus-16 binary counter in Figure 9–17(a) are the $Q_0\overline{Q}_3$ AND gate, the Q_0Q_3 AND gate, and the OR gate; this arrangement detects the occurrence of the 1001 state and causes the counter to recycle properly on the next clock pulse.

IMPLEMENTATION: 4-BIT SYNCHRONOUS BINARY COUNTER



Fixed-Function Device The 74HC163 is an example of an integrated circuit 4-bit synchronous binary counter. A logic symbol is shown in Figure 9–20 with pin numbers in parentheses. This counter has several features in addition to the basic functions previously discussed for the general synchronous binary counter.

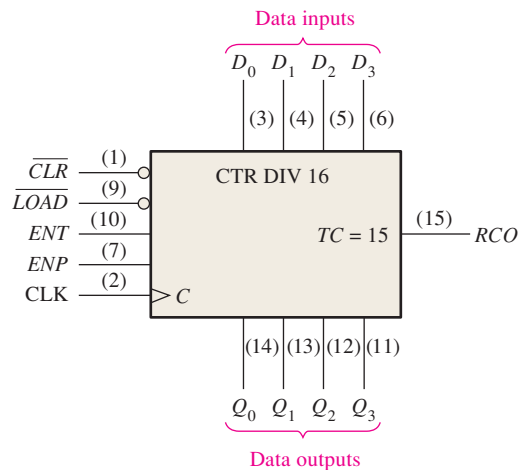


FIGURE 9–20 The 74HC163 4-bit synchronous binary counter. (The qualifying label CTR DIV 16 indicates a counter with sixteen states.)

First, the counter can be synchronously preset to any 4-bit binary number by applying the proper levels to the parallel data inputs. When a LOW is applied to the $\overline{LOAD}$ input, the counter will assume the state of the data inputs on the next clock pulse. Thus, the counter sequence can be started with any 4-bit binary number.

Also, there is an active-LOW clear input ($\overline{CLR}$), which synchronously resets all four flip-flops in the counter. There are two enable inputs, ENP and ENT . These inputs must both be HIGH for the counter to sequence through its binary states. When at least one input is LOW, the counter is disabled. The ripple clock output (RCO) goes HIGH when the counter reaches the last state in its sequence of fifteen, called the **terminal count** ($TC = 15$). This output, in conjunction with the enable inputs, allows these counters to be cascaded for higher count sequences.

Figure 9–21 shows a timing diagram of this counter being preset to twelve (1100) and then counting up to its terminal count, fifteen (1111). Input D_0 is the least significant input bit, and Q_0 is the least significant output bit.

Let's examine this timing diagram in detail. This will aid you in interpreting timing diagrams in this chapter or on manufacturers' data sheets. To begin, the LOW level pulse on the $\overline{CLR}$ input causes all the outputs (Q_0 , Q_1 , Q_2 , and Q_3) to go LOW.

Next, the LOW level pulse on the $\overline{LOAD}$ input synchronously enters the data on the data inputs (D_0 , D_1 , D_2 , and D_3) into the counter. These data appear on the Q outputs at the time of the first positive-going clock edge after $\overline{LOAD}$ goes LOW. This is the preset operation. In this particular example, Q_0 is LOW, Q_1 is LOW, Q_2 is HIGH, and Q_3 is HIGH. This, of course, is a binary 12 (Q_0 is the LSB).

The counter now advances through states 13, 14, and 15 on the next three positive-going clock edges. It then recycles to 0, 1, 2 on the following clock pulses. Notice that

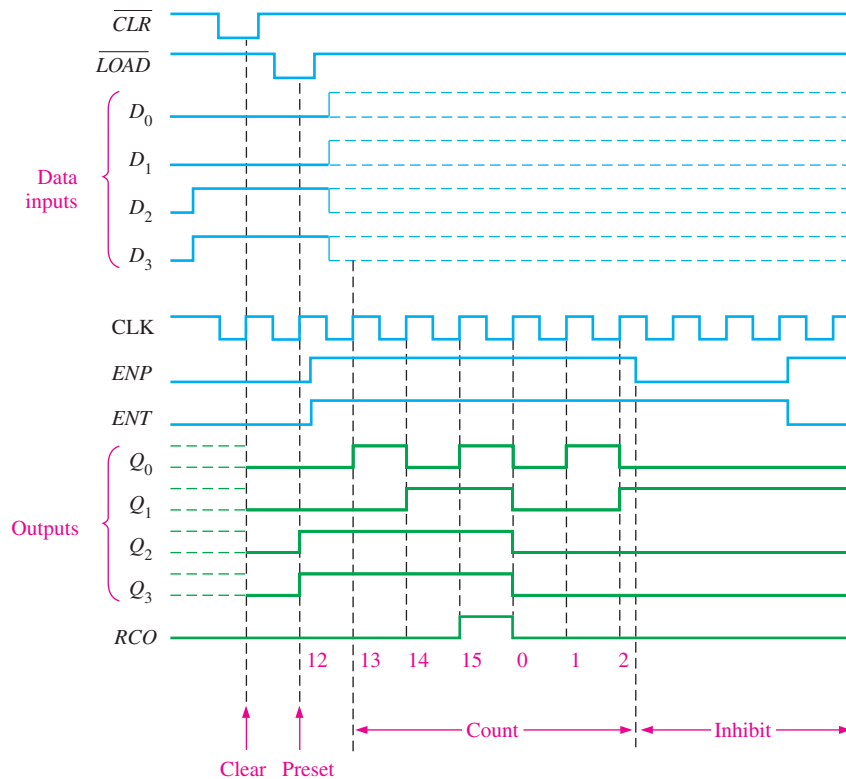


FIGURE 9-21 Timing example for a 74HC163.

both *ENP* and *ENT* inputs are HIGH during the state sequence. When *ENP* goes LOW, the counter is inhibited and remains in the binary 2 state.

Programmable Logic Device (PLD) The VHDL code for a 4-bit synchronous decade counter using J-K flip flops is as follows:



```

library ieee;
use ieee.std_logic_1164.all;

entity FourBitSynchDecadeCounter is
    port (Clk: in std_logic; Q0, Q1, Q2, Q3: inout std_logic);
end entity FourBitSynchDecadeCounter;

architecture LogicOperation of FourBitSynchDecadeCounter is

    component jkff is
        port (J, K, Clk: in std_logic; Q: out std_logic);
    end component jkff;

    signal J1, J2, J3: std_logic;
begin
    J1 <= Q0 and not Q3;
    J2 <= Q1 and Q0;
    J3 <= (Q2 and J2) or (Q0 and Q3);

    FF0: jkff port map (J => '1', K => '1', Clk => Clk, Q => Q0);
    FF1: jkff port map (J => J1, K => J1, Clk => Clk, Q => Q1);
    FF2: jkff port map (J => J2, K => J2, Clk => Clk, Q => Q2);
    FF3: jkff port map (J => J3, K => J3, Clk => Clk, Q => Q3);
end architecture LogicOperation;

```

Input and outputs declared

Component declaration for the J-K flip-flop

Boolean expressions for J input of each flip-flop ($J = K$)

Instantiations define connections for each flip-flop.

SECTION 9-3 CHECKUP

1. How does a synchronous counter differ from an asynchronous counter?
2. Explain the function of the preset feature of counters such as the 74HC163.
3. Describe the purpose of the *ENP* and *ENT* inputs and the *RCO* output for the 74HC163 counter.

9-4 Up/Down Synchronous Counters

An **up/down counter** is one that is capable of progressing in either direction through a certain sequence. An up/down counter, sometimes called a bidirectional counter, can have any specified sequence of states. A 3-bit binary counter that advances upward through its sequence (0, 1, 2, 3, 4, 5, 6, 7) and then can be reversed so that it goes through the sequence in the opposite direction (7, 6, 5, 4, 3, 2, 1, 0) is an illustration of up/down sequential operation.

After completing this section, you should be able to

- ♦ Explain the basic operation of an up/down counter
- ♦ Discuss the 74HC190 up/down decade counter

In general, most up/down counters can be reversed at any point in their sequence. For instance, the 3-bit binary counter can be made to go through the following sequence:

$$\overbrace{0, 1, 2, 3, 4, 5}^{\text{UP}}, \underbrace{4, 3, 2}_{\text{DOWN}}, \overbrace{3, 4, 5, 6, 7}^{\text{UP}}, \underbrace{6, 5}_{\text{DOWN}}, \text{etc.}$$

Table 9-6 shows the complete up/down sequence for a 3-bit binary counter. The arrows indicate the state-to-state movement of the counter for both its UP and its DOWN modes of operation. An examination of Q_0 for both the up and down sequences shows that FF0 toggles on each clock pulse. Thus, the J_0 and K_0 inputs of FF0 are

$$J_0 = K_0 = 1$$

TABLE 9-6

Up/Down sequence for a 3-bit binary counter.

Clock Pulse	Up	Q_2	Q_1	Q_0	Down
0	↶	0	0	0	↷
1	↶	0	0	1	↷
2	↶	0	1	0	↷
3	↶	0	1	1	↷
4	↶	1	0	0	↷
5	↶	1	0	1	↷
6	↶	1	1	0	↷
7	↶	1	1	1	↷

For the up sequence, Q_1 changes state on the next clock pulse when $Q_0 = 1$. For the down sequence, Q_1 changes on the next clock pulse when $Q_0 = 0$. Thus, the J_1 and K_1 inputs of FF1 must equal 1 under the conditions expressed by the following equation:

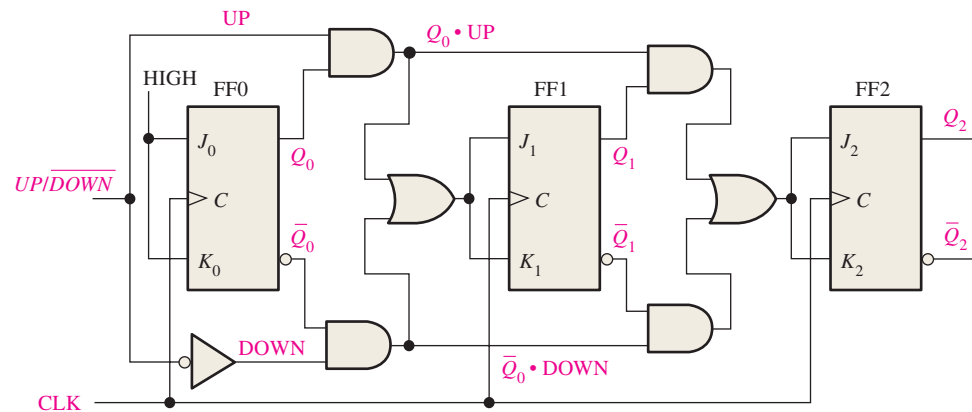
$$J_1 = K_1 = (Q_0 \cdot \text{UP}) + (\overline{Q_0} \cdot \text{DOWN})$$

For the up sequence, Q_2 changes state on the next clock pulse when $Q_0 = Q_1 = 1$. For the down sequence, Q_2 changes on the next clock pulse when $Q_0 = Q_1 = 0$. Thus, the J_2 and K_2 inputs of FF2 must equal 1 under the conditions expressed by the following equation:

$$J_2 = K_2 = (Q_0 \cdot Q_1 \cdot \text{UP}) + (\bar{Q}_0 \cdot \bar{Q}_1 \cdot \text{DOWN})$$

Each of the conditions for the J and K inputs of each flip-flop produces a toggle at the appropriate point in the counter sequence.

Figure 9–22 shows a basic implementation of a 3-bit up/down binary counter using the logic equations just developed for the J and K inputs of each flip-flop. Notice that the $UP/DOWN$ control input is HIGH for UP and LOW for DOWN.

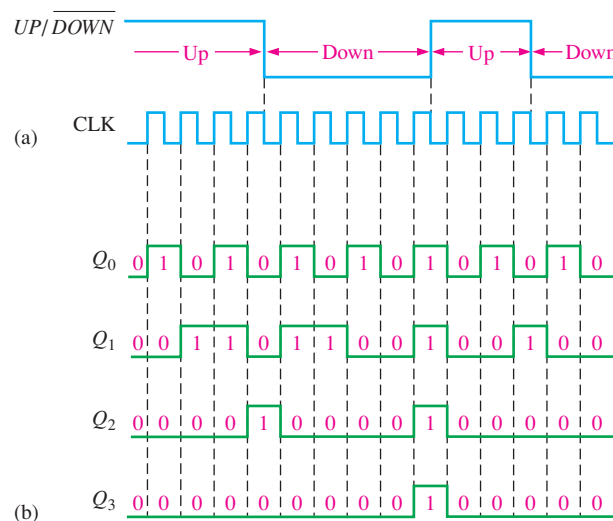


MultiSim

FIGURE 9-22 A basic 3-bit up/down synchronous counter. Open file F09-22 to verify operation.

EXAMPLE 9-3

Show the timing diagram and determine the sequence of a 4-bit synchronous binary up/down counter if the clock and $UP/\overline{DOWN}$ control inputs have waveforms as shown in Figure 9-23(a). The counter starts in the all-0s state and is positive edge-triggered.

**FIGURE 9-23**

Solution

The timing diagram showing the Q outputs is shown in Figure 9–23(b). From these waveforms, the counter sequence is as shown in Table 9–7.

TABLE 9–7			
Q_3	Q_2	Q_1	Q_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	0	1	1
0	0	1	0
0	0	0	1
0	0	0	0
1	1	1	1
0	0	0	0
0	0	0	1
0	0	1	0
0	0	0	1
0	0	0	0

UP

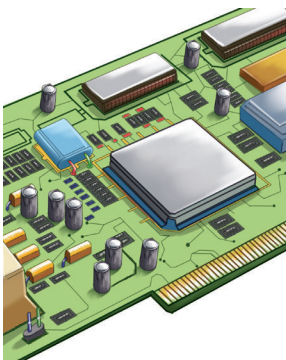
DOWN

UP

DOWN

Related Problem

Show the timing diagram if the $UP/DOWN$ control waveform in Figure 9–23(a) is inverted.

IMPLEMENTATION: UP/DOWN DECADE COUNTER

Fixed-Function Device Figure 9–24 shows a logic diagram for the 74HC190, an example of an integrated circuit up/down synchronous decade counter. The direction of the count is determined by the level of the up/down input ($D/\overline{U}$). When this input is HIGH, the counter counts down; when it is LOW, the counter counts up. Also, this device can be preset to any desired BCD digit as determined by the states of the data inputs when the $\overline{LOAD}$ input is LOW.

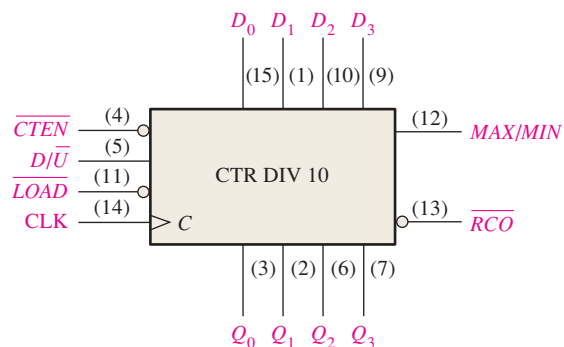


FIGURE 9–24 The 74HC190 up/down synchronous decade counter.

The *MAX/MIN* output produces a HIGH pulse when the terminal count nine (1001) is reached in the UP mode or when the terminal count zero (0000) is reached in the DOWN mode. The *MAX/MIN* output, the ripple clock output (*RCO*), and the count enable input (*CTEN*) are used when cascading counters. (Cascaded counters are discussed in Section 9–6.)

Figure 9–25 is a timing diagram that shows the 74HC190 counter preset to seven (0111) and then going through a count-up sequence followed by a count-down sequence. The *MAX/MIN* output is HIGH when the counter is in either the all-0s state (*MIN*) or the 1001 state (*MAX*).

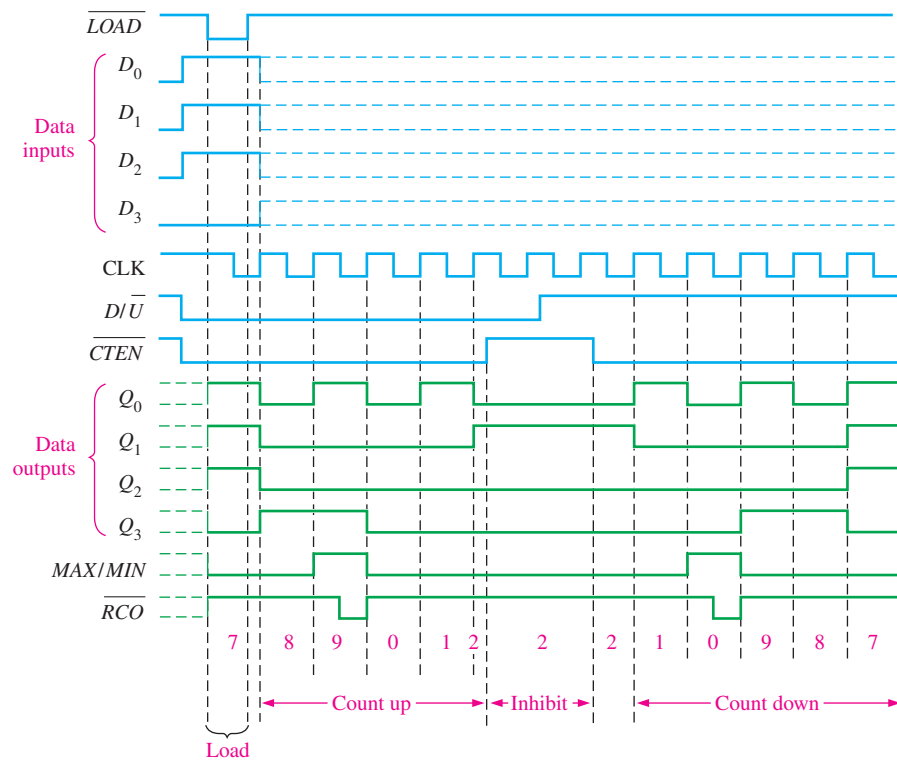


FIGURE 9–25 Timing example for a 74HC190.



Programmable Logic Device (PLD) A VHDL code for an up/down decade counter using J-K flip-flops is as follows:

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity UpDnDecadeCntr is
  port (UPDN, Clk: in std_logic; Q0, Q1, Q2, Q3: buffer std_logic);
end entity UpDnDecadeCntr;
```

UPDN: Counter direction
Clk: System clock
Q0-Q3: Counter output

```
architecture LogicOperation of UpDnDecadeCntr is
```

```
  component jkff is
    port (J, K, Clk: in std_logic; Q: buffer std_logic);
  end component jkff;
```

} J-K flip flop component

```

function UpDown(A, B, C, D: in std_logic)
    return std_logic is
begin
    return((A and B) or (C and D));
end function UpDown;

```

Function UpDown is a helper function performing the common logic between stages performed by the two AND gates applied to the OR gate supplying the J K inputs of the next stage. See Figure 9–22.

```

signal J1Up, J1Dn, J1, J2, J3: std_logic;

```

J1Up: Initial Up logic for FF1.

J1Dn: Initial Down logic for FF1.

J1-J3: Variable for combined UpDown applied to FF1-FF3.

```

begin

```

```

    J1Up <= UPDN and Q0; J1Dn <= not UPDN and not Q0;

```

```

    UpDn1: J1 <= UpDown(UPDN, Q0, not UPDN, not Q0);

```

```

    UpDn2: J2 <= UpDown(J1Up, Q1, J1Dn, not Q1);

```

```

    UpDn3: J3 <= UpDown(J1Up and Q1, Q2, J1Dn and not Q1, not Q2);

```

Identifiers J1, J2, and J3 complete the up/down logic applied to the J and K inputs of flip-flop stages FF0-FF1. Using a function to perform operations common to multiple tasks simplifies the overall code design and implementation.

```

    FF0: jkff port map (J => '1', K => '1', Clk => Clk, Q => Q0);

```

```

    FF1: jkff port map (J => J1, K => J1, Clk => Clk, Q => Q1);

```

```

    FF2: jkff port map (J => J2, K => J2, Clk => Clk, Q => Q2);

```

```

    FF3: jkff port map (J => J3, K => J3, Clk => Clk, Q => Q3);

```

Flip-flop stages FF0-FF3 complete the Up/Down counter.

```

end architecture LogicOperation;

```

SECTION 9–4 CHECKUP

1. A 4-bit up/down binary counter is in the DOWN mode and in the 1010 state. On the next clock pulse, to what state does the counter go?
2. What is the terminal count of a 4-bit binary counter in the UP mode? In the DOWN mode? What is the next state after the terminal count in the DOWN mode?

9–5 Design of Synchronous Counters

In this section, you will learn the six steps to design a counter (state machine). As you learned in Section 9–1, sequential circuits can be classified into two types: (1) those in which the output or outputs depend only on the present internal state (Moore state machines) and (2) those in which the output or outputs depend on both the present state and the input or inputs (Mealy state machines). This section is recommended for those who want an introduction to counter design or to state machine design in general. It is not a prerequisite for any other material.

After completing this section, you should be able to

- ◆ Develop a state diagram for a given sequence
- ◆ Develop a next-state table for a specified counter sequence
- ◆ Create a flip-flop transition table
- ◆ Use the Karnaugh map method to derive the logic requirements for a synchronous counter
- ◆ Implement a counter to produce a specified sequence of states

Step 1: State Diagram

The first step in the design of a state machine (counter) is to create a state diagram. A **state diagram** shows the progression of states through which the counter advances when it is

clocked. As an example, Figure 9–26 is a state diagram for a basic 3-bit Gray code counter. This particular circuit has no inputs other than the clock and no outputs other than the outputs taken off each flip-flop in the counter. You may wish to review the coverage of the Gray code in Chapter 2 at this time.

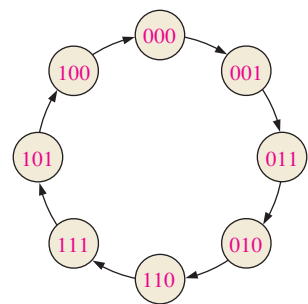


FIGURE 9–26 State diagram for a 3-bit Gray code counter.

Step 2: Next-State Table

Once the sequential circuit is defined by a state diagram, the second step is to derive a next-state table, which lists each state of the counter (present state) along with the corresponding next state. *The next state is the state that the counter goes to from its present state upon application of a clock pulse.* The next-state table is derived from the state diagram and is shown in Table 9–8 for the 3-bit Gray code counter. Q_0 is the least significant bit.

TABLE 9–8					
Next-state table for 3-bit Gray code counter.					
Present State			Next State		
Q_2	Q_1	Q_0	Q_2	Q_1	Q_0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	1	0	1	0
0	1	0	1	1	0
1	1	0	1	1	1
1	1	1	1	0	1
1	0	1	1	0	0
1	0	0	0	0	0

TABLE 9–9			
Transition table for a J-K flip-flop.			
Output Transitions			Flip-Flop Inputs
Q_N		Q_{N+1}	J K
0	→	0	0 X
0	→	1	1 X
1	→	0	X 1
1	→	1	X 0

Q_N : present state
 Q_{N+1} : next state
X: “don’t care”

Step 3: Flip-Flop Transition Table

Table 9–9 is a transition table for the J-K flip-flop. All possible output transitions are listed by showing the Q output of the flip-flop going from present states to next states. Q_N is the present state of the flip-flop (before a clock pulse) and Q_{N+1} is the next state (after a clock pulse). For each output transition, the J and K inputs that will cause the transition to occur are listed. An X indicates a “don’t care” (the input can be either a 1 or a 0).

To design the counter, the transition table is applied to each of the flip-flops in the counter, based on the next-state table (Table 9–8). For example, for the present state 000,

Q_0 goes from a present state of 0 to a next state of 1. To make this happen, J_0 must be a 1 and you don't care what K_0 is ($J_0 = 1, K_0 = X$), as you can see in the transition table (Table 9–9). Next, Q_1 is 0 in the present state and remains a 0 in the next state. For this transition, $J_1 = 0$ and $K_1 = X$. Finally, Q_2 is 0 in the present state and remains a 0 in the next state. Therefore, $J_2 = 0$ and $K_2 = X$. This analysis is repeated for each present state in Table 9–8.

Step 4: Karnaugh Maps

Karnaugh maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter. There is a Karnaugh map for the J input and a Karnaugh map for the K input of each flip-flop. In this design procedure, each cell in a Karnaugh map represents one of the present states in the counter sequence listed in Table 9–8.

From the J and K states in the transition table (Table 9–9) a 1, 0, or X is entered into each present-state cell on the maps depending on the transition of the Q output for a particular flip-flop. To illustrate this procedure, two sample entries are shown for the J_0 and the K_0 inputs to the least significant flip-flop (Q_0) in Figure 9–27.

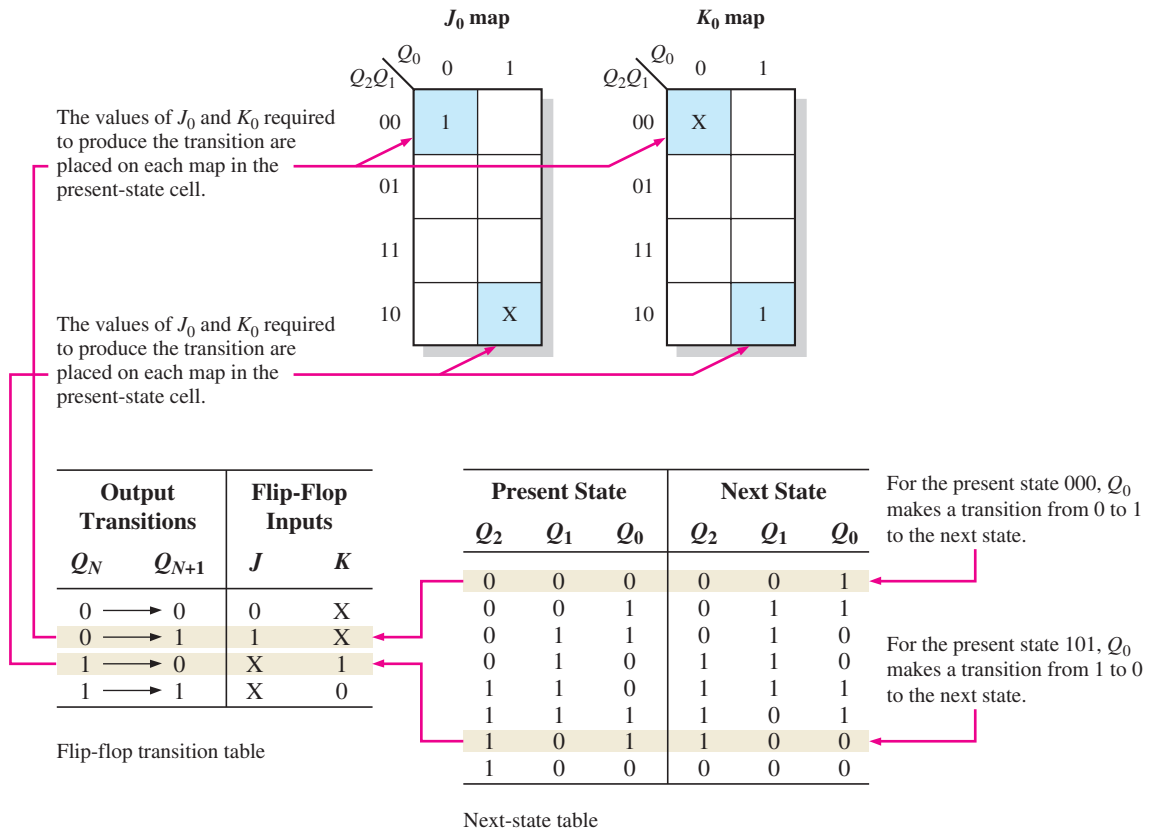


FIGURE 9–27 Examples of the mapping procedure for the counter sequence represented in Table 9–8 and Table 9–9.

The completed Karnaugh maps for all three flip-flops in the counter are shown in Figure 9–28. The cells are grouped as indicated and the corresponding Boolean expressions for each group are derived.

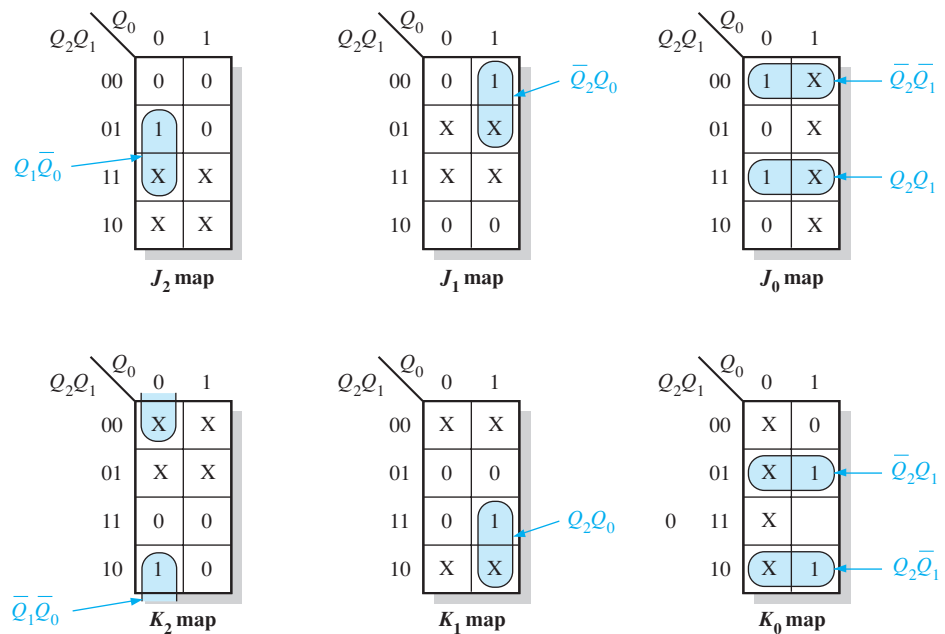


FIGURE 9-28 Karnaugh maps for present-state J and K inputs.

Step 5: Logic Expressions for Flip-Flop Inputs

From the Karnaugh maps of Figure 9-28 you obtain the following expressions for the J and K inputs of each flip-flop:

$$\begin{aligned}
 J_0 &= Q_2Q_1 + \overline{Q_2}\overline{Q_1} = \overline{Q_2 \oplus Q_1} \\
 K_0 &= Q_2\overline{Q_1} + \overline{Q_2}Q_1 = Q_2 \oplus Q_1 \\
 J_1 &= \overline{Q_2}Q_0 \\
 K_1 &= Q_2Q_0 \\
 J_2 &= Q_1\overline{Q_0} \\
 K_2 &= \overline{Q_1}\overline{Q_0}
 \end{aligned}$$

Step 6: Counter Implementation

The final step is to implement the combinational logic from the expressions for the J and K inputs and connect the flip-flops to form the complete 3-bit Gray code counter as shown in Figure 9-29.

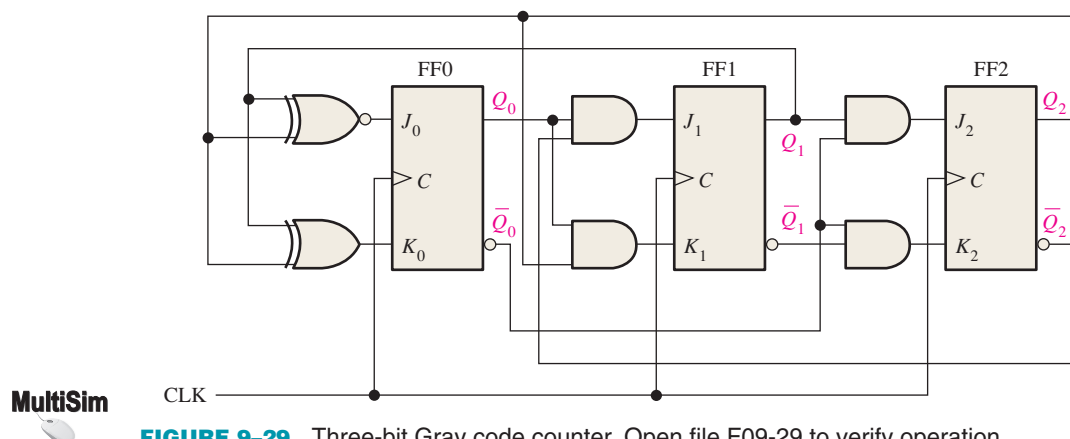


FIGURE 9-29 Three-bit Gray code counter. Open file F09-29 to verify operation.

A summary of steps used in the design of the 3-bit Gray code counter follows. In general, these steps can be applied to any state machine.

1. Specify the counter sequence and draw a state diagram.
2. Derive a next-state table from the state diagram.
3. Develop a transition table showing the flip-flop inputs required for each transition. The transition table is always the same for a given type of flip-flop.
4. Transfer the J and K states from the transition table to Karnaugh maps. There is a Karnaugh map for each input of each flip-flop.
5. Group the Karnaugh map cells to generate and derive the logic expression for each flip-flop input.
6. Implement the expressions with combinational logic, and combine with the flip-flops to create the counter.

This procedure is now applied to the design of other synchronous counters in Examples 9–4 and 9–5.

EXAMPLE 9–4

Design a counter with the irregular binary count sequence shown in the state diagram of Figure 9–30. Use D flip-flops.

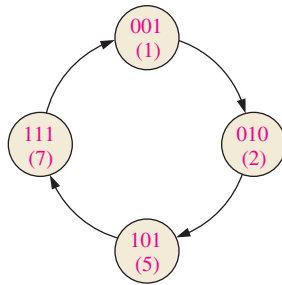


FIGURE 9–30

Solution

Step 1: The state diagram is as shown. Although there are only four states, a 3-bit counter is required to implement this sequence because the maximum binary count is seven. Since the required sequence does not include all the possible binary states, the invalid states (0, 3, 4, and 6) can be treated as “don’t cares” in the design. However, if the counter should erroneously get into an invalid state, you must make sure that it goes back to a valid state.

Step 2: The next-state table is developed from the state diagram and is given in Table 9–10.

TABLE 9–10

Next-state table.

Present State			Next State		
Q_2	Q_1	Q_0	Q_2	Q_1	Q_0
0	0	1	0	1	0
0	1	0	1	0	1
1	0	1	1	1	1
1	1	1	0	0	1

Step 3: The transition table for the D flip-flop is shown in Table 9–11.

TABLE 9–11				
Transition table for a D flip-flop.				
Output Transitions			Flip-Flop Input	
Q_N		Q_{N+1}	D	
0	→	0	0	
0	→	1	1	
1	→	0	0	
1	→	1	1	

Step 4: The D inputs are plotted on the present-state Karnaugh maps in Figure 9–31. Also “don’t cares” can be placed in the cells corresponding to the invalid states of 000, 011, 100, and 110, as indicated by the red Xs.

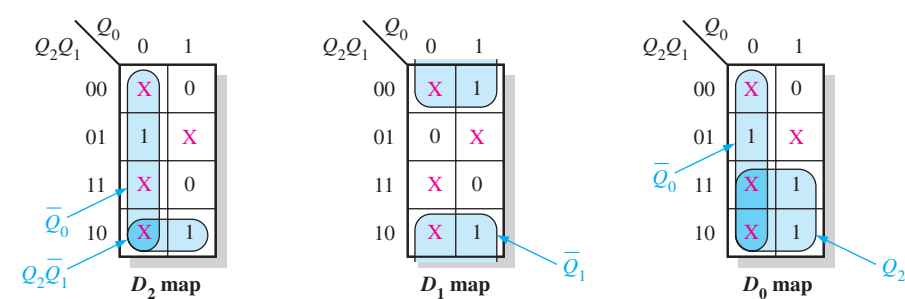


FIGURE 9–31

Step 5: Group the 1s, taking advantage of as many of the “don’t care” states as possible for maximum simplification, as shown in Figure 9–31. The expression for each D input taken from the maps is as follows:

$$\begin{aligned} D_0 &= \overline{Q_0} + Q_2 \\ D_1 &= \overline{Q_1} \\ D_2 &= \overline{Q_0} + Q_2\overline{Q_1} \end{aligned}$$

Step 6: The implementation of the counter is shown in Figure 9–32.

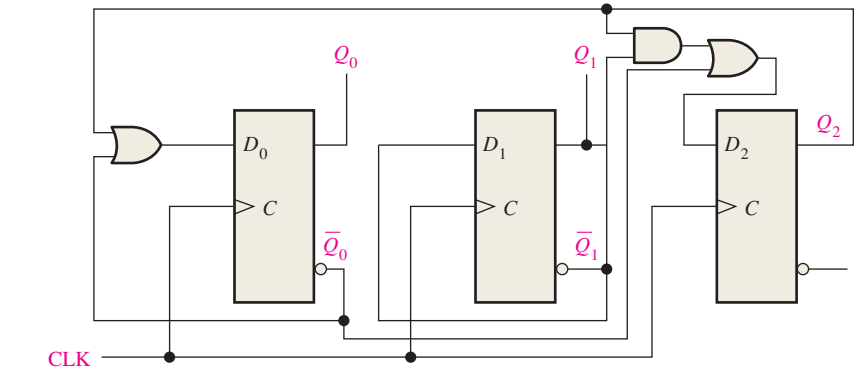


FIGURE 9–32

An analysis shows that if the counter, by accident, gets into one of the invalid states (0, 3, 4, 6), it will always return to a valid state according to the following sequences: $0 \rightarrow 3 \rightarrow 4 \rightarrow 7$, and $6 \rightarrow 1$.

Related Problem

Verify the analysis that proves the counter will always return (eventually) to a valid state from an invalid state.

EXAMPLE 9-5

Develop a synchronous 3-bit up/down counter with a Gray code sequence using J-K flip-flops. The counter should count up when an UP/DOWN control input is 1 and count down when the control input is 0.

Solution

Step 1: The state diagram is shown in Figure 9-33. The 1 or 0 beside each arrow indicates the state of the UP/DOWN control input, Y .

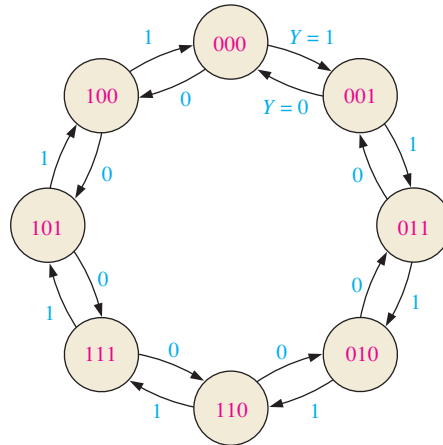


FIGURE 9-33 State diagram for a 3-bit up/down Gray code counter.

Step 2: The next-state table is derived from the state diagram and is shown in Table 9-12. Notice that for each present state there are two possible next states, depending on the UP/DOWN control variable, Y .

TABLE 9-12

Next-state table for 3-bit up/down Gray code counter.

Present State			Next State					
			$Y = 0$ (DOWN)			$Y = 1$ (UP)		
			Q_2	Q_1	Q_0	Q_2	Q_1	Q_0
0	0	0	1	0	0	0	0	1
0	0	1	0	0	0	0	1	1
0	1	1	0	0	1	0	1	0
0	1	0	0	1	1	1	1	0
1	1	0	0	1	0	1	1	1
1	1	1	1	1	0	1	0	1
1	0	1	1	1	1	1	0	0
1	0	0	1	0	1	0	0	0

$Y = \text{UP/DOWN control input.}$

Step 3: The transition table for the J-K flip-flops is repeated in Table 9–13.

TABLE 9–13

Transition table for a J-K flip-flop.

Output Transitions			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Step 4: The Karnaugh maps for the J and K inputs of the flip-flops are shown in Figure 9–34. The UP/DOWN control input, Y , is considered one of the state variables along with Q_0 , Q_1 , and Q_2 . Using the next-state table, the information in the “Flip-Flop Inputs” column of Table 9–13 is transferred onto the maps as indicated for each present state of the counter.

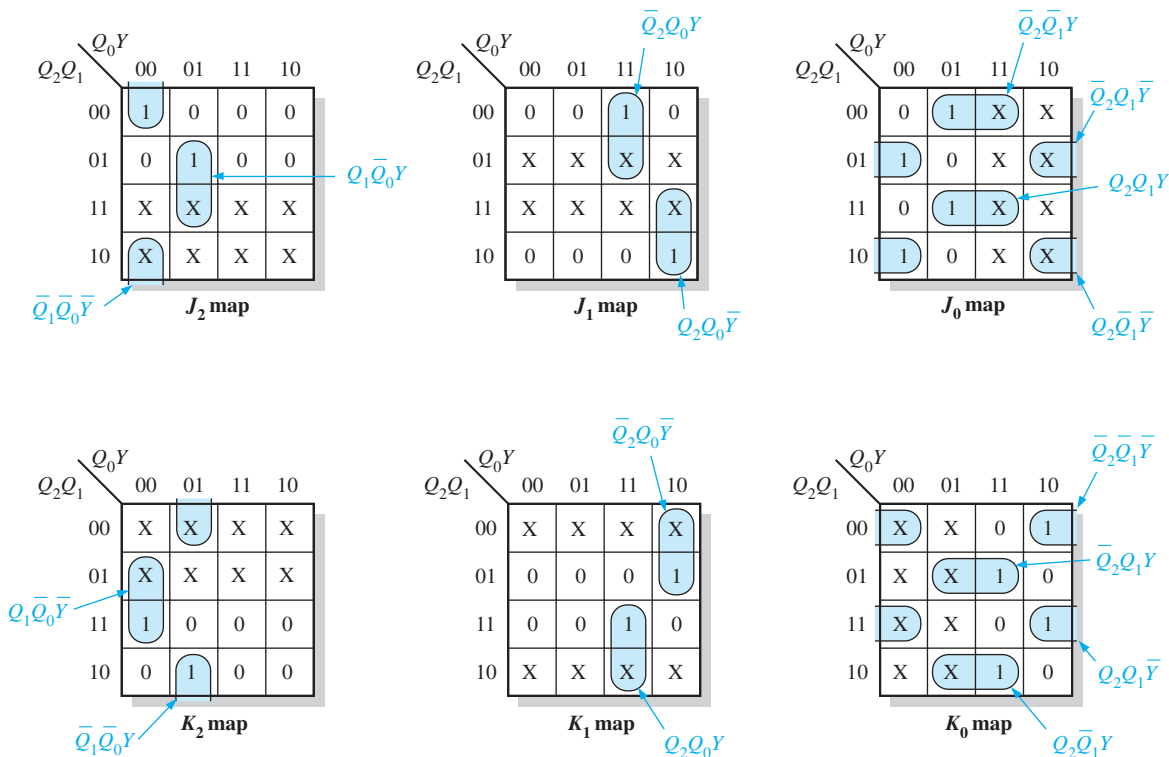


FIGURE 9–34 J and K maps for Table 9–12. The UP/DOWN control input, Y , is treated as a fourth variable.

Step 5: The 1s are combined in the largest possible groupings, with “don’t cares” (Xs) used where possible. The groups are factored, and the expressions for the J and K inputs are as follows:

$$\begin{aligned}
 J_0 &= Q_2Q_1Y + Q_2\bar{Q}_1\bar{Y} + \bar{Q}_2\bar{Q}_1Y + \bar{Q}_2Q_1\bar{Y} & K_0 &= \bar{Q}_2\bar{Q}_1\bar{Y} + \bar{Q}_2Q_1Y + Q_2\bar{Q}_1Y + Q_2Q_1\bar{Y} \\
 J_1 &= \bar{Q}_2Q_0Y + Q_2Q_0\bar{Y} & K_1 &= \bar{Q}_2Q_0\bar{Y} + Q_2Q_0Y \\
 J_2 &= Q_1\bar{Q}_0Y + \bar{Q}_1\bar{Q}_0\bar{Y} & K_2 &= Q_1\bar{Q}_0Y + \bar{Q}_1\bar{Q}_0\bar{Y}
 \end{aligned}$$

Step 6: The J and K equations are implemented with combinational logic. This step is the Related Problem.

Related Problem

Specify the number of flip-flops, gates, and inverters that are required to implement the logic described in Step 5.

SECTION 9-5 CHECKUP

1. A flip-flop is presently in the RESET state and must go to the SET state on the next clock pulse. What must J and K be?
2. A flip-flop is presently in the SET state and must remain SET on the next clock pulse. What must J and K be?
3. A binary counter is in the $Q_3\overline{Q}_2Q_1\overline{Q}_0 = 1010$ state.
 - (a) What is its next state?
 - (b) What condition must exist on each flip-flop input to ensure that it goes to the proper next state on the clock pulse?

9-6 Cascaded Counters

Counters can be connected in cascade to achieve higher-modulus operation. In essence, **cascading** means that the last-stage output of one counter drives the input of the next counter.

After completing this section, you should be able to

- ♦ Determine the overall modulus of cascaded counters
- ♦ Analyze the timing diagram of a cascaded counter configuration
- ♦ Use cascaded counters as a frequency divider
- ♦ Use cascaded counters to achieve specified truncated sequences

Asynchronous Cascading

An example of two asynchronous counters connected in cascade is shown in Figure 9-35 for a 2-bit and a 3-bit ripple counter. The timing diagram is shown in Figure 9-36. Notice

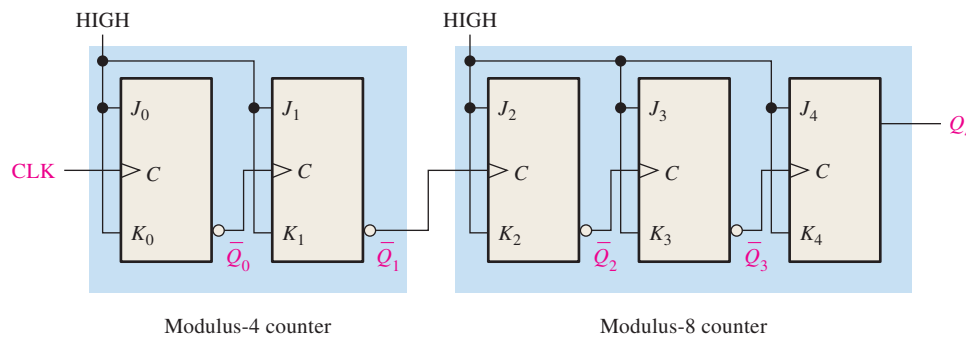


FIGURE 9-35 Two cascaded asynchronous counters (all J and K inputs are HIGH).

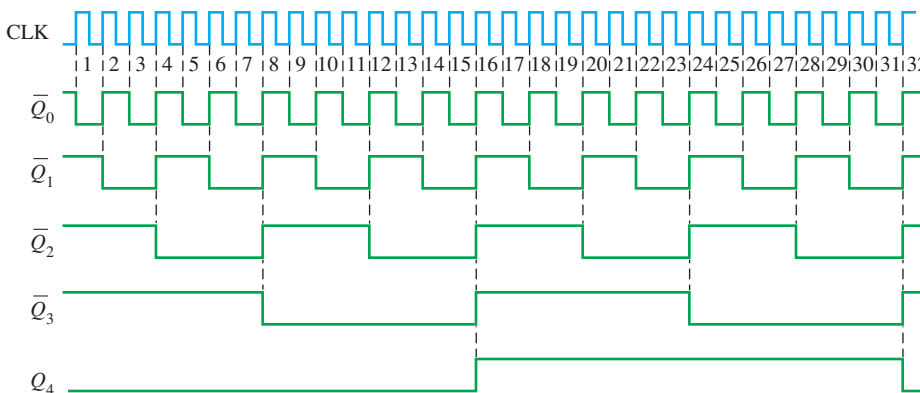


FIGURE 9-36 Timing diagram for the cascaded counter configuration of Figure 9-35.

The overall modulus of cascaded counters is equal to the product of the individual moduli.

InfoNote

The time stamp counter (TSC), mentioned in the last InfoNote, is a 64-bit counter. It is interesting to observe that if this counter (or any full-modulus 64-bit counter) is clocked at a frequency of 1 GHz, it will take 583 years for it to go through all of its states and reach its terminal count. In contrast, a 32-bit full-modulus counter will exhaust all of its states in approximately 4.3 seconds when clocked at 1 GHz. The difference is astounding.

that the final output of the modulus-8 counter, Q_4 , occurs once for every 32 input clock pulses. The overall modulus of the two cascaded counters is $4 \times 8 = 32$; that is, they act as a divide-by-32 counter.

Synchronous Cascading

When operating synchronous counters in a cascaded configuration, it is necessary to use the count enable and the terminal count functions to achieve higher-modulus operation. On some devices the count enable is labeled simply *CTEN* (or some other designation such as *G*), and terminal count (*TC*) is analogous to ripple clock output (*RCO*) on some IC counters.

Figure 9–37 shows two decade counters connected in cascade. The terminal count (*TC*) output of counter 1 is connected to the count enable (*CTEN*) input of counter 2. Counter 2 is inhibited by the LOW on its *CTEN* input until counter 1 reaches its last, or terminal, state and its terminal count output goes HIGH. This HIGH now enables counter 2, so that when the first clock pulse after counter 1 reaches its terminal count (*CLK10*), counter 2 goes from its initial state to its second state. Upon completion of the entire second cycle of counter 1 (when counter 1 reaches terminal count the second time), counter 2 is again enabled and advances to its next state. This sequence continues. Since these are decade counters, counter 1 must go through ten complete cycles before counter 2 completes its first cycle. In other words, for every ten cycles of counter 1, counter 2 goes through one cycle. Thus, counter 2 will complete one cycle after one hundred clock pulses. The overall modulus of these two cascaded counters is $10 \times 10 = 100$.

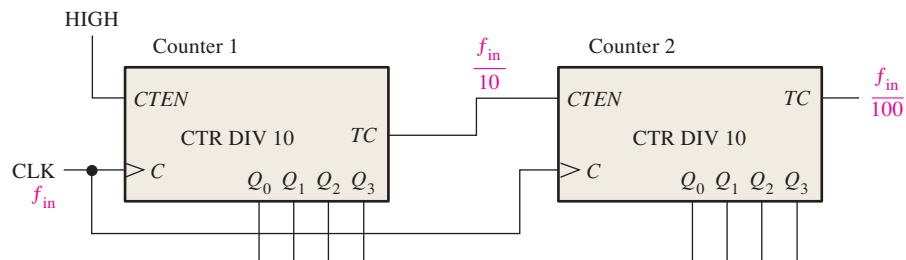


FIGURE 9-37 A modulus-100 counter using two cascaded decade counters.

When viewed as a frequency divider, the circuit of Figure 9–37 divides the input clock frequency by 100. Cascaded counters are often used to divide a high-frequency clock signal to obtain highly accurate pulse frequencies. Cascaded counter configurations used for such purposes are sometimes called *countdown chains*. For example, suppose that you have a basic clock frequency of 1 MHz and you wish to obtain 100 kHz, 10 kHz, and 1 kHz; a series of cascaded decade counters can be used. If the 1 MHz signal is divided by 10, the output is 100 kHz. Then if the 100 kHz signal is divided by 10, the output is 10 kHz. Another division by 10 produces the 1 kHz frequency. The general implementation of this countdown chain is shown in Figure 9–38.

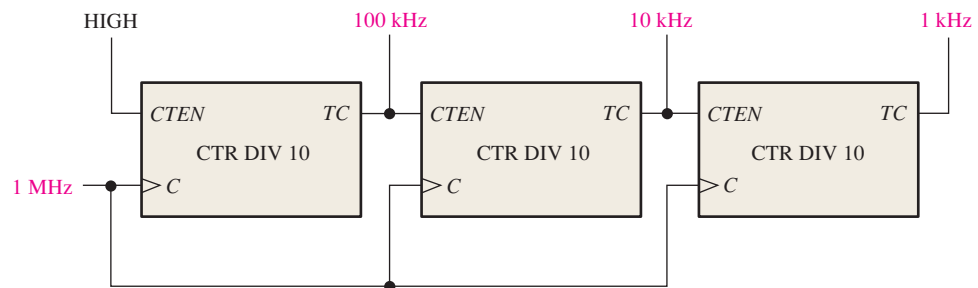
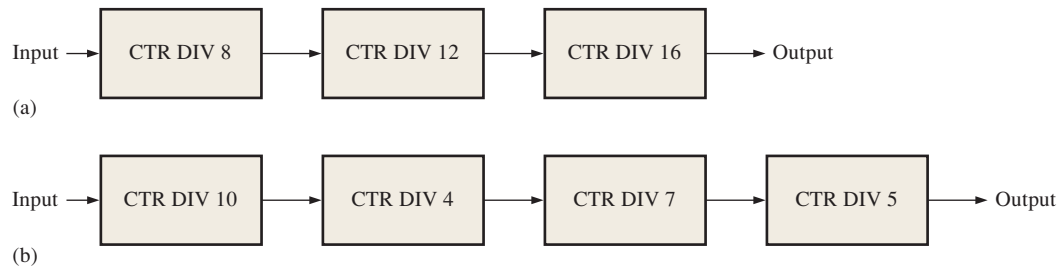


FIGURE 9-38 Three cascaded decade counters forming a divide-by-1000 frequency divider with intermediate divide-by-10 and divide-by-100 outputs.

EXAMPLE 9-6

Determine the overall modulus of the two cascaded counter configurations in Figure 9-39.

**FIGURE 9-39****Solution**

In Figure 9-39(a), the overall modulus for the 3-counter configuration is

$$8 \times 12 \times 16 = \mathbf{1536}$$

In Figure 9-39(b), the overall modulus for the 4-counter configuration is

$$10 \times 4 \times 7 \times 5 = \mathbf{1400}$$

Related Problem

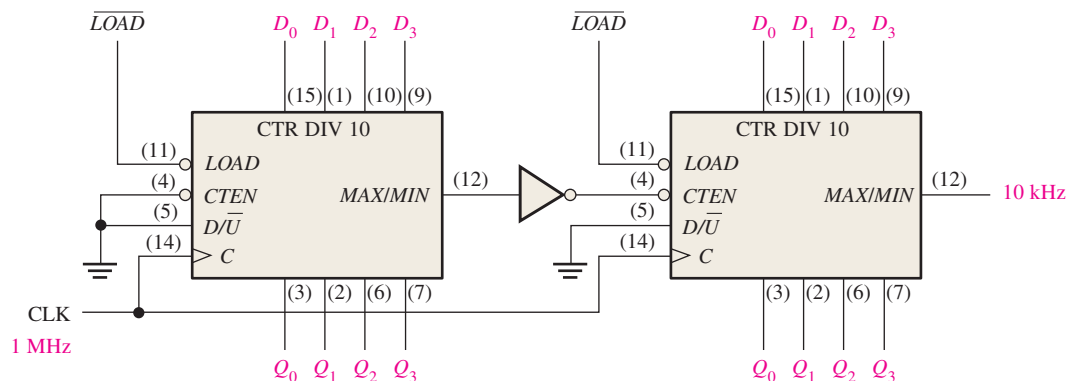
How many cascaded decade counters are required to divide a clock frequency by 100,000?

EXAMPLE 9-7

Use 74HC190 up/down decade counters connected in the UP mode to obtain a 10 kHz waveform from a 1 MHz clock. Show the logic diagram.

Solution

To obtain 10 kHz from a 1 MHz clock requires a division factor of 100. Two 74HC190 counters must be cascaded as shown in Figure 9-40. The left counter produces a terminal count (*MAX/MIN*) pulse for every 10 clock pulses. The right counter produces a terminal count (*MAX/MIN*) pulse for every 100 clock pulses.

**FIGURE 9-40** A divide-by-100 counter using two 74HC190 up/down decade counters connected for the up sequence.**Related Problem**

Determine the frequency of the waveform at the Q_0 output of the second counter (the one on the right) in Figure 9-40.

Cascaded Counters with Truncated Sequences

The preceding discussion has shown how to achieve an overall modulus (divide-by-factor) that is the product of the individual moduli of all the cascaded counters. This can be considered *full-modulus cascading*.

Often an application requires an overall modulus that is less than that achieved by full-modulus cascading. That is, a truncated sequence must be implemented with cascaded counters. To illustrate this method, we will use the cascaded counter configuration in Figure 9–41. This particular circuit uses four 74HC161 4-bit synchronous binary counters. If these four counters (sixteen bits total) were cascaded in a full-modulus arrangement, the modulus would be

$$2^{16} = 65,536$$

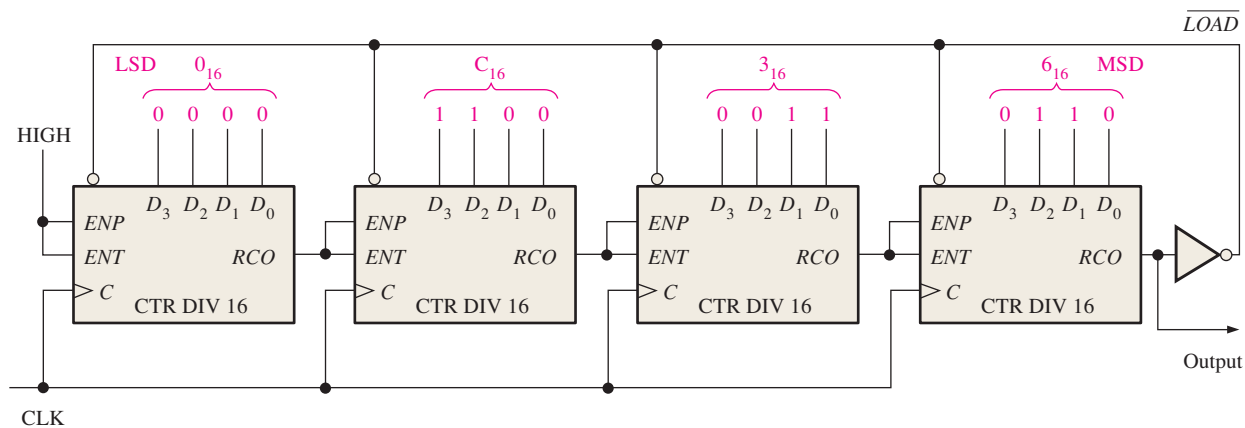


FIGURE 9–41 A divide-by-40,000 counter using 74HC161 4-bit binary counters. Note that each of the parallel data inputs is shown in binary order (the right-most bit D_0 is the LSB in each counter).

Let's assume that a certain application requires a divide-by-40,000 counter (modulus 40,000). The difference between 65,536 and 40,000 is 25,536, which is the number of states that must be *deleted* from the full-modulus sequence. The technique used in the circuit of Figure 9–41 is to preset the cascaded counter to 25,536 (63C0 in hexadecimal) each time it recycles, so that it will count from 25,536 up to 65,535 on each full cycle. Therefore, each full cycle of the counter consists of 40,000 states.

Notice in Figure 9–41 that the $\overline{RCO}$ output of the right-most counter is inverted and applied to the $\overline{LOAD}$ input of each 4-bit counter. Each time the count reaches its terminal value of 65,535, which is 11111111111111_2 , $\overline{RCO}$ goes HIGH and causes the number on the parallel data inputs ($63C0_{16}$) to be synchronously loaded into the counter with the clock pulse. Thus, there is one $\overline{RCO}$ pulse from the right-most 4-bit counter for every 40,000 clock pulses.

With this technique any modulus can be achieved by synchronous loading of the counter to the appropriate initial state on each cycle.

SECTION 9–6 CHECKUP

- How many decade counters are necessary to implement a divide-by-1000 (modulus-1000) counter? A divide-by-10,000?
- Show with general block diagrams how to achieve each of the following, using a flip-flop, a decade counter, and a 4-bit binary counter, or any combination of these:

(a) Divide-by-20 counter	(b) Divide-by-32 counter
(c) Divide-by-160 counter	(d) Divide-by-320 counter

9-7 Counter Decoding

In many applications, it is necessary that some or all of the counter states be decoded. The decoding of a counter involves using decoders or logic gates to determine when the counter is in a certain binary state in its sequence. For instance, the terminal count function previously discussed is a single decoded state (the last state) in the counter sequence.

After completing this section, you should be able to

- ◆ Implement the decoding logic for any given state in a counter sequence
- ◆ Explain why glitches occur in counter decoding logic
- ◆ Use the method of strobing to eliminate decoding glitches

Suppose that you wish to decode binary state 6 (110) of a 3-bit binary counter. When $Q_2 = 1$, $Q_1 = 1$, and $Q_0 = 0$, a HIGH appears on the output of the decoding gate, indicating that the counter is at state 6. This can be done as shown in Figure 9-42. This is called *active-HIGH decoding*. Replacing the AND gate with a NAND gate provides active-LOW decoding.

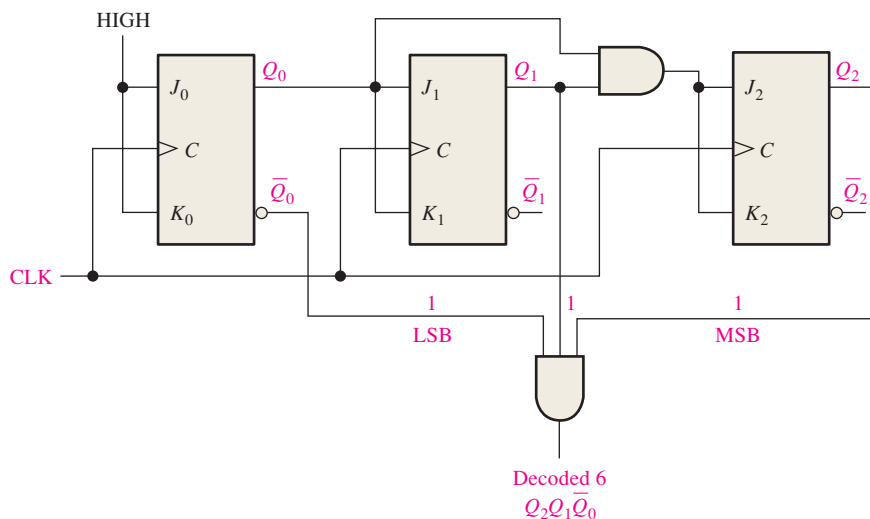


FIGURE 9-42 Decoding of state 6 (110). Open file F09-42 to verify operation.



EXAMPLE 9-8

Implement the decoding of binary state 2 and binary state 7 of a 3-bit synchronous counter. Show the entire counter timing diagram and the output waveforms of the decoding gates. Binary 2 = $\overline{Q}_2 Q_1 \overline{Q}_0$ and binary 7 = $Q_2 Q_1 Q_0$.

Solution

See Figure 9-43. The 3-bit counter was originally discussed in Section 9-3 (Figure 9-15).

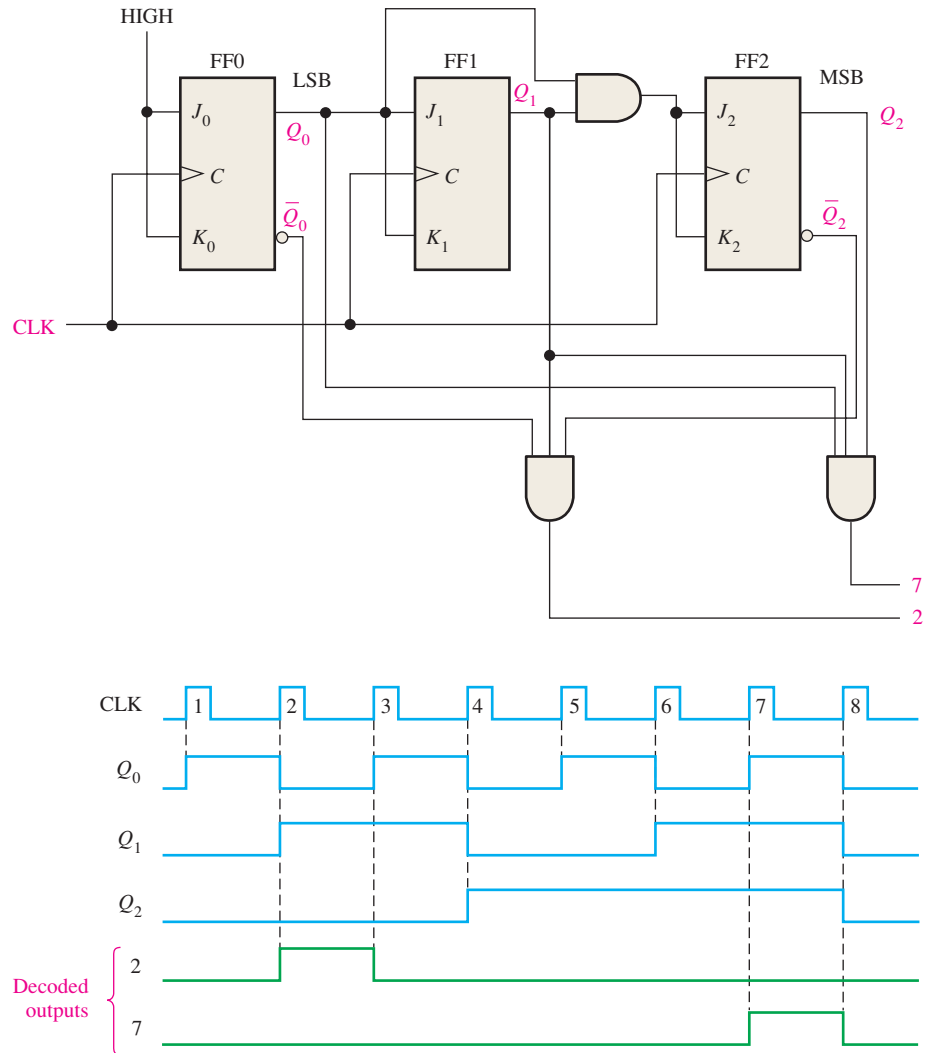


FIGURE 9-43 A 3-bit counter with active-HIGH decoding of count 2 and count 7. Open file F09-43 to verify operation.

Related Problem

Show the logic for decoding state 5 in the 3-bit counter.

Decoding Glitches

A glitch is an unwanted spike of voltage.

The problem of glitches produced by the decoding process was discussed in Chapter 6. As you have learned, the propagation delays due to the ripple effect in asynchronous counters create transitional states in which the counter outputs are changing at slightly different times. These transitional states produce undesired voltage spikes of short duration (glitches) on the outputs of a decoder connected to the counter. The glitch problem can also occur to some degree with synchronous counters because the propagation delays from the clock to the Q outputs of each flip-flop in a counter can vary slightly.

Figure 9-44 shows a basic asynchronous BCD decade counter connected to a BCD-to-decimal decoder. To see what happens in this case, let's look at a timing diagram in which the propagation delays are taken into account, as shown in Figure 9-45. Notice that these delays cause false states of short duration. The value of the false binary state at each critical transition is indicated on the diagram. The resulting glitches can be seen on the decoder outputs.

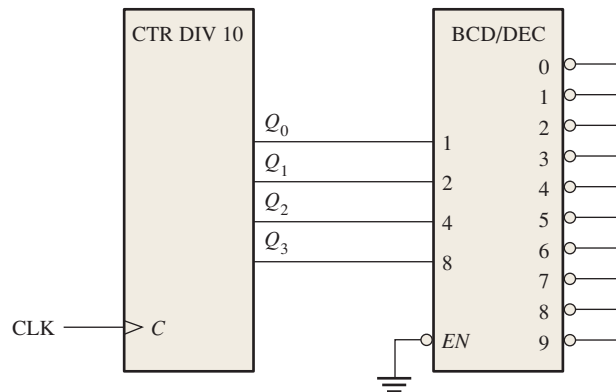


FIGURE 9-44 A basic decade (BCD) counter and decoder.

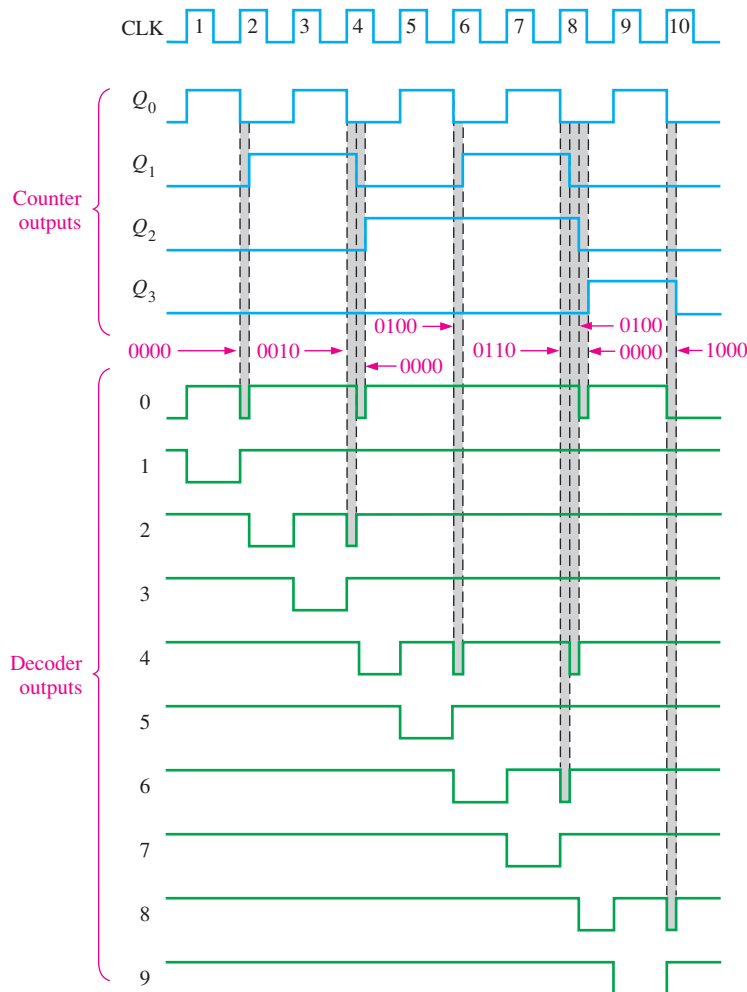


FIGURE 9-45 Outputs with glitches from the decoder in Figure 9-44. Glitch widths are exaggerated for illustration and are usually only a few nanoseconds wide.

One way to eliminate the glitches is to enable the decoded outputs at a time after the glitches have had time to disappear. This method is known as *strobing* and can be accomplished in the case of an active-HIGH clock by using the LOW level of the clock to enable the decoder, as shown in Figure 9-46. The resulting improved timing diagram is shown in Figure 9-47.

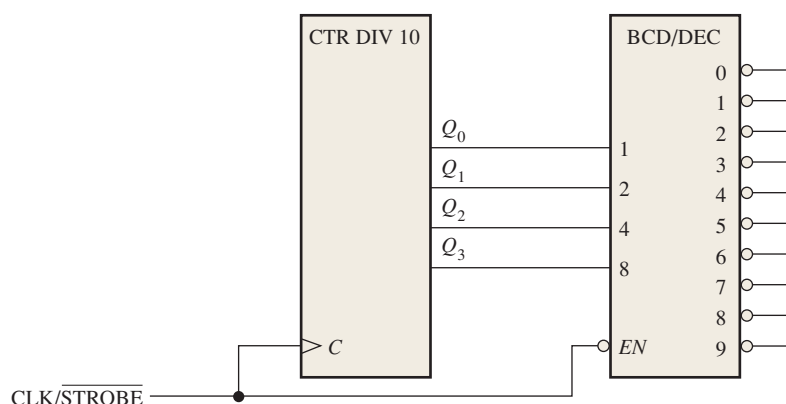


FIGURE 9-46 The basic decade counter and decoder with strobing to eliminate glitches.

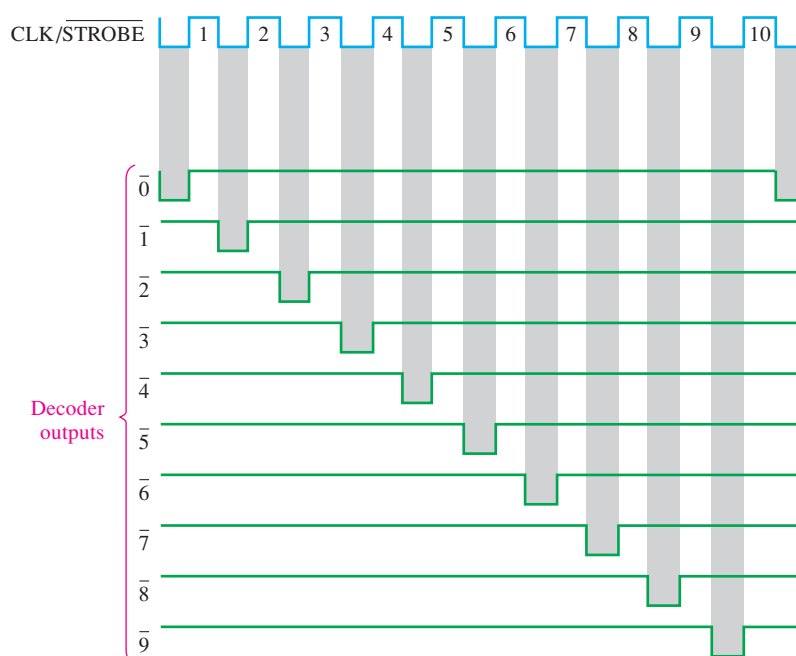


FIGURE 9-47 Strobed decoder outputs for the circuit of Figure 9-46.

SECTION 9-7 CHECKUP

1. What transitional states are possible when a 4-bit asynchronous binary counter changes from
 - (a) count 2 to count 3
 - (b) count 3 to count 4
 - (c) count 10_{10} to count 11_{10}
 - (d) count 15 to count 0

9-8 Counter Applications

The digital counter is a useful and versatile device that is found in many applications. In this section, some representative counter applications are presented.

After completing this section, you should be able to

- ♦ Describe how counters are used in a basic digital clock system
- ♦ Explain how a divide-by-60 counter is implemented and how it is used in a digital clock

- ◆ Explain how the hours counter is implemented
- ◆ Discuss the application of a counter in an automobile parking control system
- ◆ Describe how a counter is used in the process of parallel-to-serial data conversion

A Digital Clock

A common example of a counter application is in timekeeping systems. Figure 9–48 is a simplified logic diagram of a digital clock that displays seconds, minutes, and hours. First, a 60 Hz sinusoidal ac voltage is converted to a 60 Hz pulse waveform and divided down to a 1 Hz pulse waveform by a divide-by-60 counter formed by a divide-by-10 counter followed by a divide-by-6 counter. Both the *seconds* and *minutes* counts are also produced by divide-by-60 counters, the details of which are shown in Figure 9–49. These counters count from 0 to 59 and then recycle to 0; synchronous decade counters are used in this particular implementation. Notice that the divide-by-6 portion is formed with a decade counter with a truncated sequence achieved by using the decoder count 6 to asynchronously clear the counter. The terminal count, 59, is also decoded to enable the next counter in the chain.

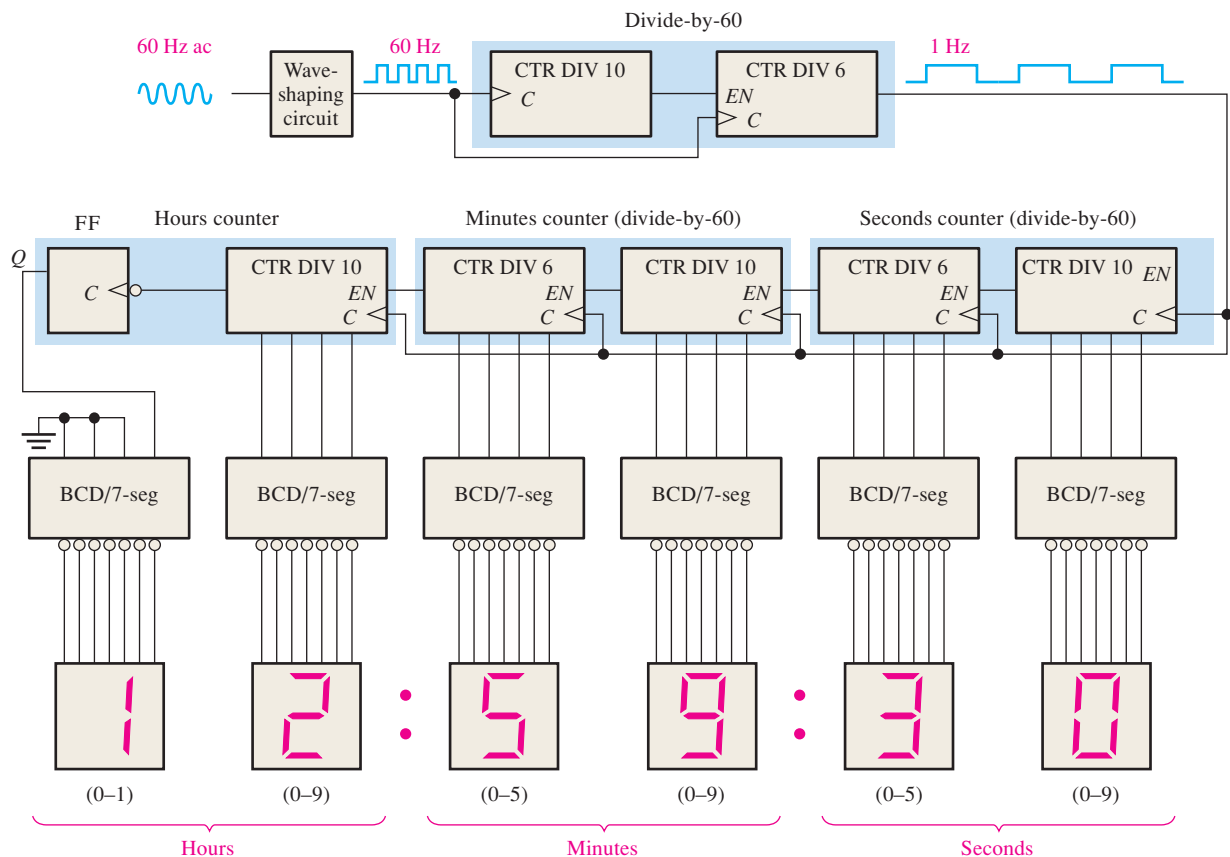


FIGURE 9–48 Simplified logic diagram for a 12-hour digital clock. Logic details using specific devices are shown in Figures 9–49 and 9–50.

The *hours* counter is implemented with a decade counter and a flip-flop as shown in Figure 9–50. Consider that initially both the decade counter and the flip-flop are RESET, and the decode-12 gate and decode-9 gate outputs are HIGH. The decade counter advances through all of its states from zero to nine, and on the clock pulse that recycles it from nine back to zero, the flip-flop goes to the SET state ($J = 1$, $K = 0$). This illuminates a 1 on the tens-of-hours display. The total count is now ten (the decade counter is in the zero state and the flip-flop is SET).

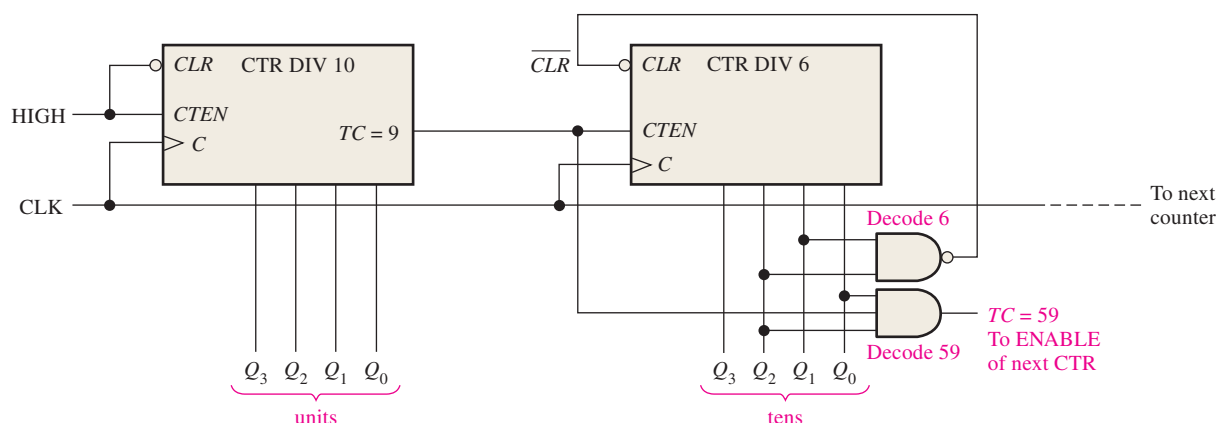


FIGURE 9-49 Logic diagram of typical divide-by-60 counter using synchronous decade counters. Note that the outputs are in binary order (the right-most bit is the LSB).

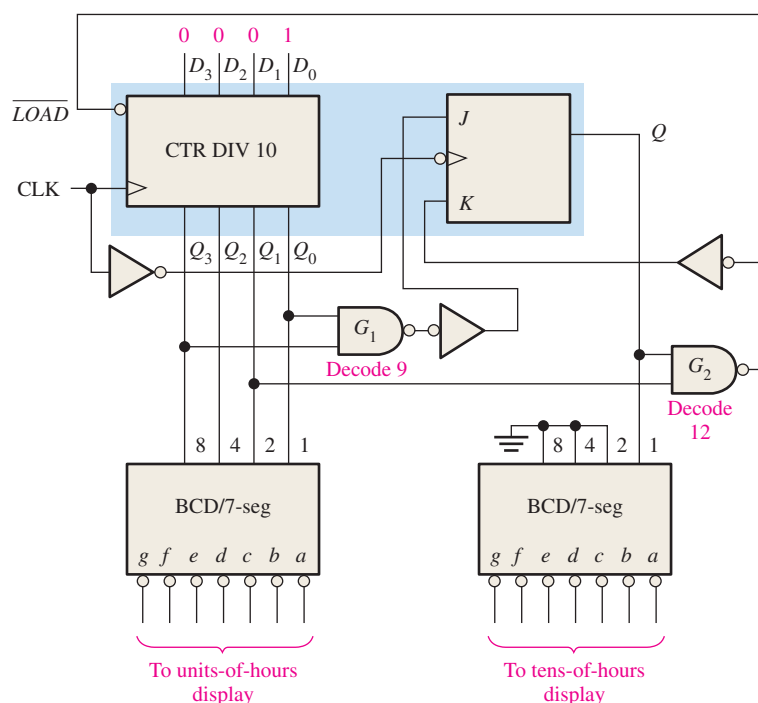


FIGURE 9-50 Logic diagram for hours counter and decoders. Note that on the counter inputs and outputs, the right-most bit is the LSB.

Next, the total count advances to eleven and then to twelve. In state 12 the Q_2 output of the decade counter is HIGH, the flip-flop is still SET, and thus the decode-12 gate output is LOW. This activates the $\overline{LOAD}$ input of the decade counter. On the next clock pulse, the decade counter is preset to 0001 from the data inputs, and the flip-flop is RESET ($J = 0$, $K = 1$). As you can see, this logic always causes the counter to recycle from twelve back to one rather than back to zero.

Automobile Parking Control

This counter example illustrates the use of an up/down counter to solve an everyday problem. The problem is to devise a means of monitoring available spaces in a one-hundred-space parking garage and provide for an indication of a full condition by illuminating a display sign and lowering a gate bar at the entrance.

A system that solves this problem consists of optoelectronic sensors at the entrance and exit of the garage, an up/down counter and associated circuitry, and an interface circuit that uses the counter output to turn the FULL sign on or off as required and lower or raise the gate bar at the entrance. A general block diagram of this system is shown in Figure 9–51.

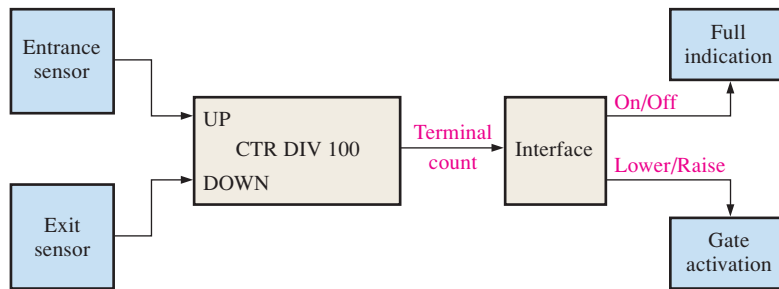


FIGURE 9–51 Functional block diagram for parking garage control.

A logic diagram of the up/down counter is shown in Figure 9–52. It consists of two cascaded up/down decade counters. The operation is described in the following paragraphs.

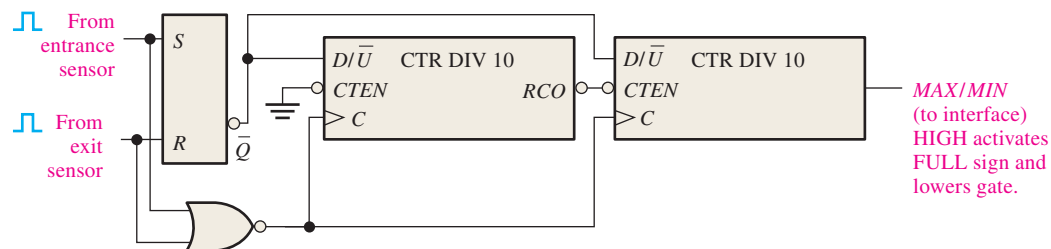


FIGURE 9–52 Logic diagram for modulus-100 up/down counter for automobile parking control.

The counter is initially preset to 0 using the parallel data inputs, which are not shown. Each automobile entering the garage breaks a light beam, activating a sensor that produces an electrical pulse. This positive pulse sets the S-R latch on its leading edge. The LOW on the $\bar{Q}$ output of the latch puts the counter in the UP mode. Also, the sensor pulse goes through the NOR gate and clocks the counter on the LOW-to-HIGH transition of its trailing edge. Each time an automobile enters the garage, the counter is advanced by one (**incremented**). When the one-hundredth automobile enters, the counter goes to its last state (100_{10}). The MAX/MIN output goes HIGH and activates the interface circuit (no detail), which lights the FULL sign and lowers the gate bar to prevent further entry.

Incrementing a counter increases its count by one.

When an automobile exits, an optoelectronic sensor produces a positive pulse, which resets the S-R latch and puts the counter in the DOWN mode. The trailing edge of the clock decreases the count by one (**decremented**). If the garage is full and an automobile leaves, the MAX/MIN output of the counter goes LOW, turning off the FULL sign and raising the gate.

Decrementing a counter decreases its count by one.

Parallel-to-Serial Data Conversion (Multiplexing)

A simplified example of data transmission using multiplexing and demultiplexing techniques was introduced in Chapter 6. Essentially, the parallel data bits on the multiplexer inputs are converted to serial data bits on the single transmission line. A group of bits appearing simultaneously on parallel lines is called *parallel data*. A group of bits appearing on a single line in a time sequence is called *serial data*.

Parallel-to-serial conversion is normally accomplished by the use of a counter to provide a binary sequence for the data-select inputs of a data selector/multiplexer, as illustrated in Figure 9–53. The Q outputs of the modulus-8 counter are connected to the data-select inputs of an 8-bit multiplexer.

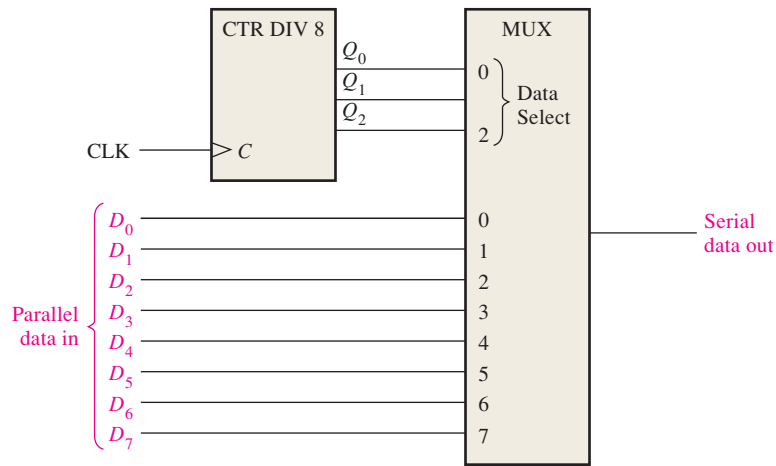


FIGURE 9-53 Parallel-to-serial data conversion logic.

Figure 9-54 is a timing diagram illustrating the operation of this circuit. The first byte (eight-bit group) of parallel data is applied to the multiplexer inputs. As the counter goes through a binary sequence from zero to seven, each bit, beginning with D_0 , is sequentially

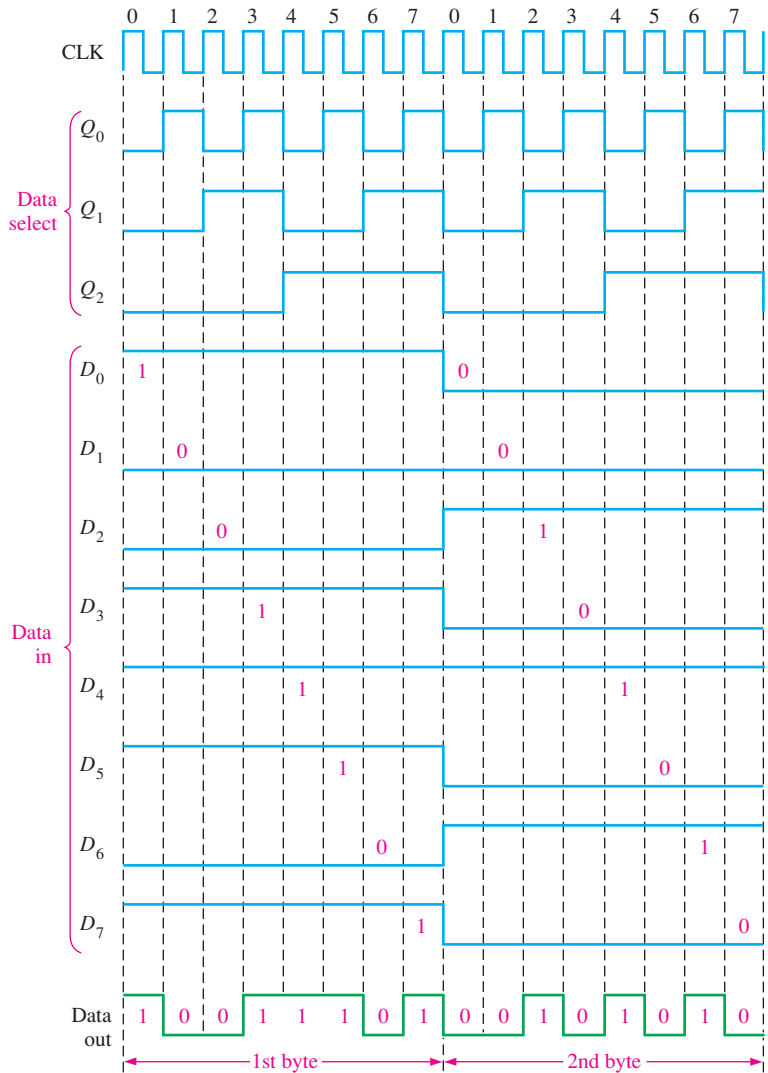


FIGURE 9-54 Example of parallel-to-serial conversion timing for the circuit in Figure 9-53.

InfoNote

Computers contain an internal counter that can be programmed for various frequencies and tone durations, thus producing “music.” To select a particular tone, the programmed instruction selects a divisor that is sent to the counter. The divisor sets the counter up to divide the basic peripheral clock frequency to produce an audio tone. The duration of a tone can also be set by a programmed instruction; thus, a basic counter is used to produce melodies by controlling the frequency and duration of tones.

selected and passed through the multiplexer to the output line. After eight clock pulses the data byte has been converted to a serial format and sent out on the transmission line. When the counter recycles back to 0, the next byte is applied to the data inputs and is sequentially converted to serial form as the counter cycles through its eight states. This process continues repeatedly as each parallel byte is converted to a serial byte.

SECTION 9-8 CHECKUP

1. Explain the purpose of each NAND gate in Figure 9-50.
2. Identify the two recycle conditions for the hours counter in Figure 9-48, and explain the reason for each.

9-9 Logic Symbols with Dependency Notation

Up to this point, the logic symbols with dependency notation specified in ANSI/IEEE Standard 91-1984 have been introduced on a limited basis. In many cases, the symbols do not deviate greatly from the traditional symbols. A significant departure does occur, however, for some devices, including counters and other more complex devices. Although we will continue to use primarily the more traditional symbols throughout this book, a brief coverage of logic symbols with dependency notation is provided. A specific IC counter is used as an example.

After completing this section, you should be able to

- ◆ Interpret logic symbols that include dependency notation
- ◆ Identify the common block and the individual elements of a counter symbol
- ◆ Interpret the qualifying symbol
- ◆ Discuss control dependency
- ◆ Discuss mode dependency
- ◆ Discuss AND dependency

Dependency notation is fundamental to the ANSI/IEEE standard. Dependency notation is used in conjunction with the logic symbols to specify the relationships of inputs and outputs so that the logical operation of a given device can be determined entirely from its logic symbol without a prior knowledge of the details of its internal structure and without a detailed logic diagram for reference. This coverage of a specific logic symbol with dependency notation is intended to aid in the interpretation of other such symbols that you may encounter in the future.

The 74HC163 4-bit synchronous binary counter is used for illustration. For comparison, Figure 9-55 shows a traditional block symbol and the ANSI/IEEE symbol with dependency notation. Basic descriptions of the symbol and the dependency notation follow.

Common Control Block

The upper block with notched corners in Figure 9-55(b) has inputs and an output that are considered common to all elements in the device and not unique to any one of the elements.

Individual Elements

The lower block in Figure 9-55(b), which is partitioned into four abutted sections, represents the four storage elements (D flip-flops) in the counter, with inputs D_0 , D_1 , D_2 , and D_3 and outputs Q_0 , Q_1 , Q_2 , and Q_3 .

Qualifying Symbol

The label “CTR DIV 16” in Figure 9-55(b) identifies the device as a counter (CTR) with sixteen states (DIV 16).

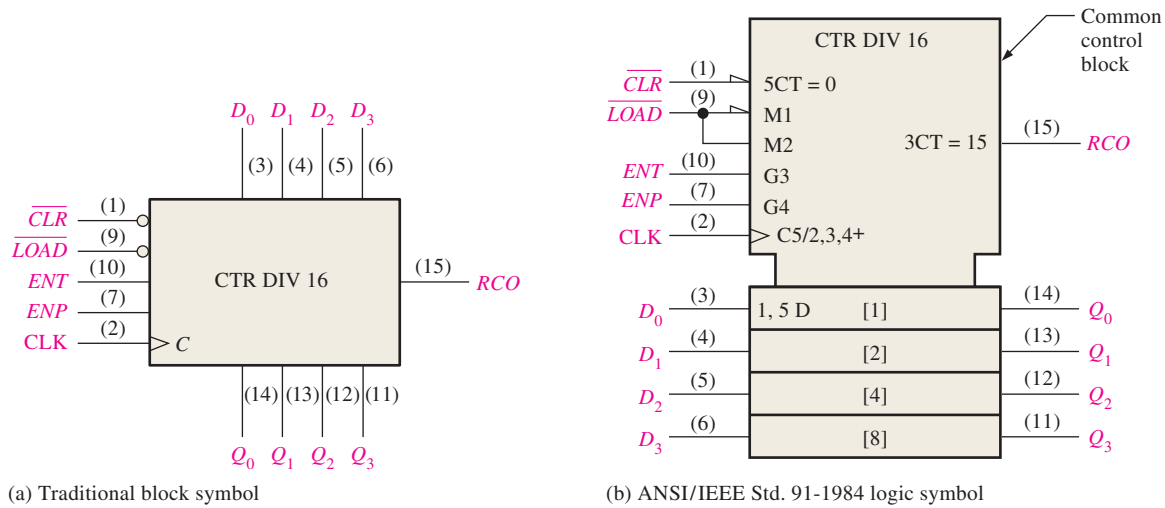


FIGURE 9-55 The 74HC163 4-bit synchronous counter.

Control Dependency (C)

As shown in Figure 9-55(b), the letter *C* denotes control dependency. Control inputs usually enable or disable the data inputs (*D*, *J*, *K*, *S*, and *R*) of a storage element. The *C* input is usually the clock input. In this case the digit 5 following *C* (*C*5/2,3,4+) indicates that the inputs labeled with a 5 prefix are dependent on the clock (synchronous with the clock). For example, 5CT = 0 on the $\overline{CLR}$ input indicates that the clear function is dependent on the clock; that is, it is a synchronous clear. When the $\overline{CLR}$ input is LOW (0), the counter is reset to zero (*CT* = 0) on the triggering edge of the clock pulse. Also, the 5 D label at the input of storage element [1] indicates that the data storage is dependent on (synchronous with) the clock. All labels in the [1] storage element apply to the [2], [4], and [8] elements below it since they are not labeled differently.

Mode Dependency (M)

As shown in Figure 9-55(b), the letter *M* denotes mode dependency. This label is used to indicate how the functions of various inputs or outputs depend on the mode in which the device is operating. In this case the device has two modes of operation. When the $\overline{LOAD}$ input is LOW (0), as indicated by the triangle input, the counter is in a preset mode (M1) in which the input data (*D*₀, *D*₁, *D*₂, and *D*₃) are synchronously loaded into the four flip-flops. The digit 1 following *M* in M1 and the 1 in the label 1, 5 D show a dependency relationship and indicate that input data are stored only when the device is in the preset mode (M1), in which $\overline{LOAD}$ = 0. When the $\overline{LOAD}$ input is HIGH (1), the counter advances through its normal binary sequence, as indicated by M2 and the 2 in C5/2,3,4+.

AND Dependency (G)

As shown in Figure 9-55(b), the letter *G* denotes AND dependency, indicating that an input designated with *G* followed by a digit is ANDed with any other input or output having the same digit as a prefix in its label. In this particular example, the G3 at the *ENT* input and the 3CT = 15 at the *RCO* output are related, as indicated by the 3, and that relationship is an AND dependency, indicated by the *G*. This tells us that *ENT* must be HIGH (no triangle on the input) and the count must be fifteen (*CT* = 15) for the *RCO* output to be HIGH.

Also, the digits 2, 3, and 4 in the label C5/2,3,4+ indicate that the counter advances through its states when $\overline{LOAD}$ = 1, as indicated by the mode dependency label M2, and when *ENT* = 1 and *ENP* = 1, as indicated by the AND dependency labels G3 and G4. The + indicates that the counter advances by one count when these conditions exist.

SECTION 9-9 CHECKUP

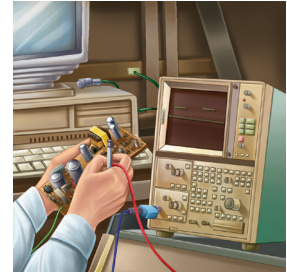
1. In dependency notation, what do the letters *C*, *M*, and *G* stand for?
2. By what letter is data storage denoted?

9-10 Troubleshooting

The troubleshooting of counters can be simple or quite involved, depending on the type of counter and the type of fault. This section will give you some insight into how to approach the troubleshooting of sequential circuits.

After completing this section, you should be able to

- ♦ Detect a faulty counter
- ♦ Isolate faults in maximum-modulus cascaded counters
- ♦ Isolate faults in cascaded counters with truncated sequences
- ♦ Determine faults in counters implemented with individual flip-flops

**Counters**

The symptom for a faulty counter is usually that it does not advance its count. If this is the case, then check power and ground on the chip. Look at these lines with a scope to make sure there is no noise present (a noisy ground may actually be open). Check that there are clock pulses and that they have the correct amplitude and rise time and that there is not extraneous noise on the line. (Sometimes clock pulses can be loaded down by other ICs, making it appear that the counter is faulty when it is not). If power, ground, and the clock pulses are okay, check all inputs (including enable, load, and clear inputs), to see that they are connected correctly and that the logic is correct. An open input can cause a counter to work correctly some of the time—inputs should never be left open, even if they are not used. (An unused input should be connected to an inactive level). If the counter is stuck in a state and the clock is present, determine what input should be present to advance the counter. This may point to a faulty input (including clear or load inputs), which can be caused by logic elsewhere in the circuit. If inputs are all checked okay, an output may be pulled LOW or HIGH by an external short or open (or another faulty IC), keeping the output from advancing.

Cascaded Counters with Maximum Modulus

A failure in one of the counters in a chain of cascaded counters can affect all the counters that follow it. For example, if a count enable input opens, it effectively acts as a HIGH (for TTL logic), and the counter is always enabled. This type of failure in one of the counters will cause that counter to run at the full clock rate and will also cause all the succeeding counters to run at higher than normal rates. This is illustrated in Figure 9-56 for a divide-by-1000 cascaded counter arrangement where an open enable (*CTEN*) input acts as a TTL HIGH and continuously enables the second counter. Other faults that can affect “downstream” counter stages are open or shorted clock inputs or terminal count outputs. In some of these situations, pulse activity can be observed, but it may be at the wrong frequency. Exact frequency or frequency ratio measurements must be made.

Cascaded Counters with Truncated Sequences

The count sequence of a cascaded counter with a truncated sequence, such as that in Figure 9-57, can be affected by other types of faults in addition to those mentioned for maximum-modulus cascaded counters. For example, a failure in one of the parallel data inputs, the *LOAD* input, or the inverter can alter the preset count and thus change the modulus of the counter.

Counters

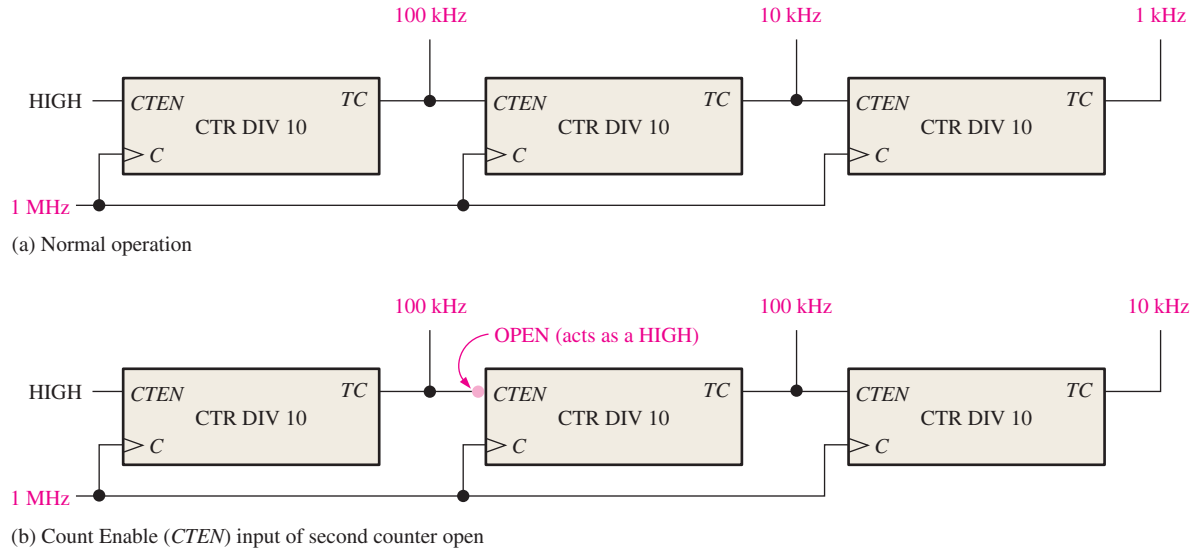


FIGURE 9-56 Example of a failure that affects following counters in a cascaded arrangement.

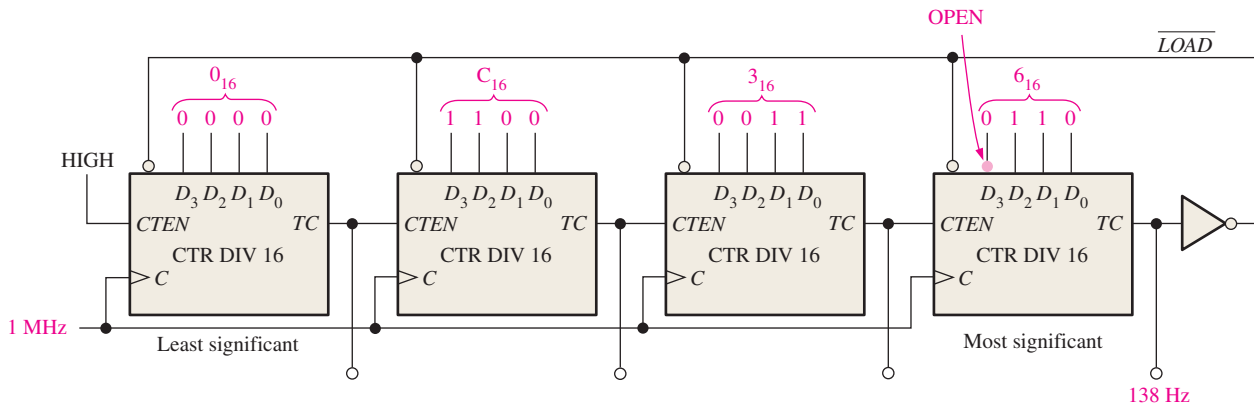


FIGURE 9-57 Example of a failure in a cascaded counter with a truncated sequence.

For example, suppose the D_3 input of the most significant counter in Figure 9-57 is open and acts as a HIGH. Instead of 6_{16} (0110) being preset into the counter, E_{16} (1110) is preset in. So, instead of beginning with $63C0_{16}$ ($25,536_{10}$) each time the counter recycles, the sequence will begin with $E3C0_{16}$ ($58,304_{10}$). This changes the modulus of the counter from 40,000 to $65,536 - 58,304 = 7232$.

To check this counter, apply a known clock frequency, for example 1 MHz, and measure the output frequency at the final terminal count output. If the counter is operating properly, the output frequency is

$$f_{\text{out}} = \frac{f_{\text{in}}}{\text{modulus}} = \frac{1 \text{ MHz}}{40,000} = 25 \text{ Hz}$$

In this case, the specific failure described in the preceding paragraph will cause the output frequency to be

$$f_{\text{out}} = \frac{f_{\text{in}}}{\text{modulus}} = \frac{1 \text{ MHz}}{7232} \cong 138 \text{ Hz}$$

EXAMPLE 9-9

Frequency measurements are made on the truncated counter in Figure 9-58 as indicated. Determine if the counter is working properly, and if not, isolate the fault.

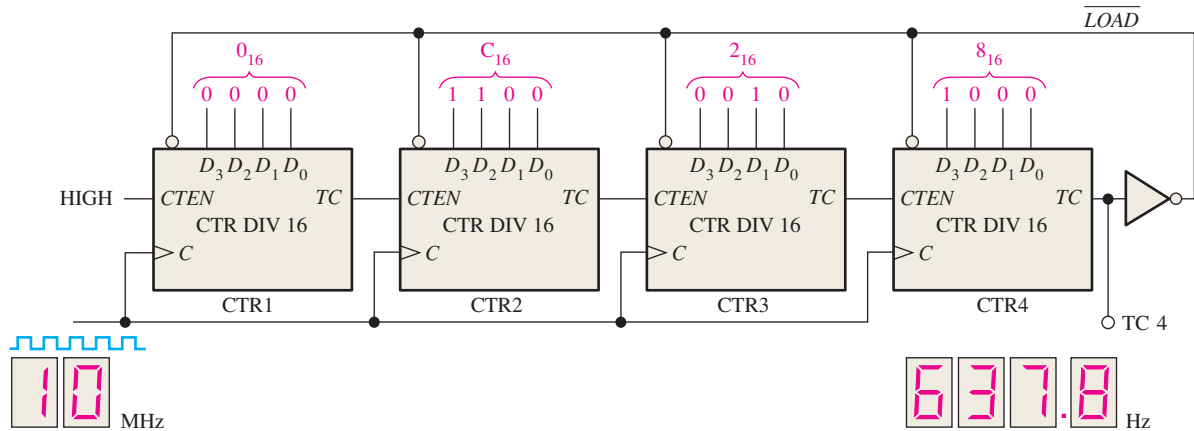


FIGURE 9-58

Solution

Check to see if the frequency measured at TC 4 is correct. If it is, the counter is working properly.

$$\begin{aligned}\text{truncated modulus} &= \text{full modulus} - \text{preset count} \\ &= 16^4 - 82C0_{16} \\ &= 65,536 - 33,472 = 32,064\end{aligned}$$

The correct frequency at TC 4 is

$$f_4 = \frac{10 \text{ MHz}}{32,064} \cong 312 \text{ Hz}$$

There is a problem. The measured frequency of 637.8 Hz does not agree with the correct calculated frequency of 312 Hz.

To find the faulty counter, determine the actual truncated modulus as follows:

$$\text{modulus} = \frac{f_{\text{in}}}{f_{\text{out}}} = \frac{10 \text{ MHz}}{637.8 \text{ Hz}} = 15,679$$

Because the truncated modulus should be 32,064, most likely the counter is being preset to the wrong count when it recycles. The actual preset count is determined as follows:

$$\begin{aligned}\text{truncated modulus} &= \text{full modulus} - \text{preset count} \\ \text{preset count} &= \text{full modulus} - \text{truncated modulus} \\ &= 65,536 - 15,679 \\ &= 49,857 \\ &= C2C0_{16}\end{aligned}$$

This shows that the counter is being preset to C2C0₁₆ instead of 82C0₁₆ each time it recycles.

Counters 1, 2, and 3 are being preset properly but counter 4 is not. Since C₁₆ = 1100₂, the D₂ input to counter 4 is HIGH when it should be LOW. This is most likely caused by an **open input**. Check for an external open caused by a bad solder connection, a broken conductor, or a bent pin on the IC. If none can be found, replace the IC and the counter should work properly.

Related Problem

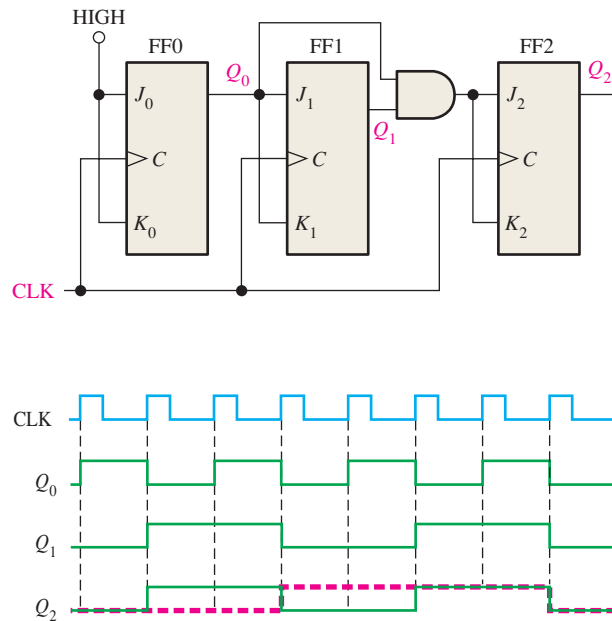
Determine what the output frequency at TC 4 would be if the D₃ input of counter 3 were open.

Counters Implemented with Individual Flip-Flops

Counters implemented with individual flip-flop and gate ICs are sometimes more difficult to troubleshoot because there are many more inputs and outputs with external connections than there are in an IC counter. The sequence of a counter can be altered by a single open or short on an input or output, as Example 9-10 illustrates.

EXAMPLE 9-10

Suppose that you observe the output waveforms (green) that are indicated for the counter in Figure 9-59. Determine if there is a problem with the counter.

**FIGURE 9-59****Solution**

The Q_2 waveform is incorrect. The correct waveform is shown as a red dashed line. You can see that the Q_2 waveform looks exactly like the Q_1 waveform, so whatever is causing FF1 to toggle appears to also be controlling FF2.

Checking the J and K inputs to FF2, you find a waveform that looks like Q_0 . This result indicates that Q_0 is somehow getting through the AND gate. The only way this can happen is if the Q_1 input to the AND gate is always HIGH. However, you have seen that Q_1 has a correct waveform. This observation leads to the conclusion that the lower input to the AND gate must be internally open and acting as a HIGH. Replace the AND gate and retest the circuit.

Related Problem

Describe the Q_2 output of the counter in Figure 9-59 if the Q_1 output of FF1 is open.



To observe the time relationship between two digital signals with a dual-trace analog oscilloscope, the proper way to trigger the scope is with the slower of the two signals. The reason for this is that the slower signal has fewer possible trigger points than the faster signal and there will be no ambiguity for starting the sweep. Vertical mode triggering uses a composite of both channels and should never be used for determining absolute time information. Since clock signals are usually the fastest signal in a digital system, they should not be used for triggering.

SECTION 9-10 CHECKUP

1. What failures can cause the counter in Figure 9-56 to have no pulse activity on any of the TC outputs?
2. What happens if the inverter in Figure 9-58 develops an open output?

Applied Logic

Elevator Controller: Part 1

This Applied Logic describes the operation and implementation of a service elevator controller for a seven-story building. The controller consists of logic that controls the elevator operation, a counter that determines the floor at which the elevator is located at any given time, and a floor number display. For simplicity, there is only one floor call and one floor request for each elevator cycle. A cycle occurs when the elevator is called to a given floor to pick up a passenger and the passenger is delivered to a requested floor. The elevator sequence for one cycle is shown in Figure 9–60.

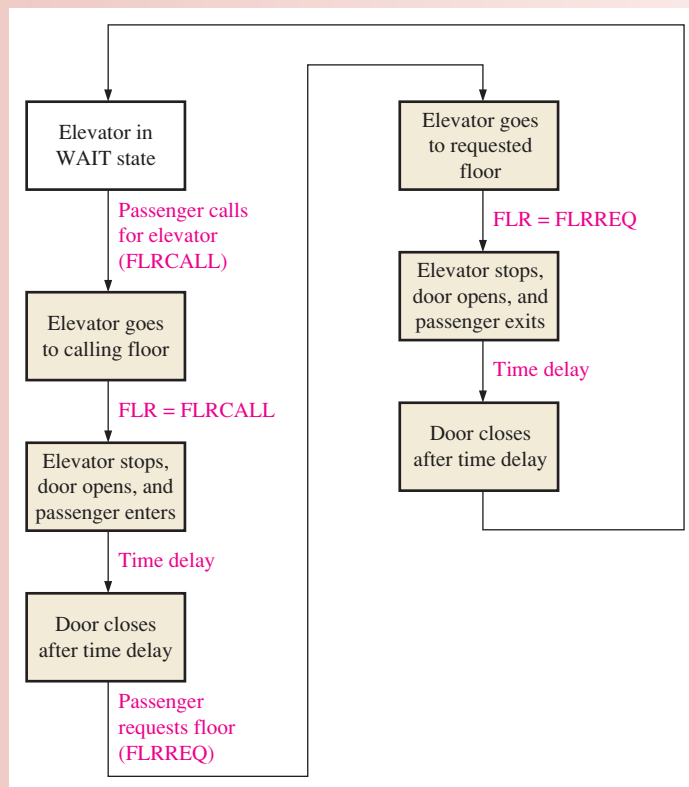
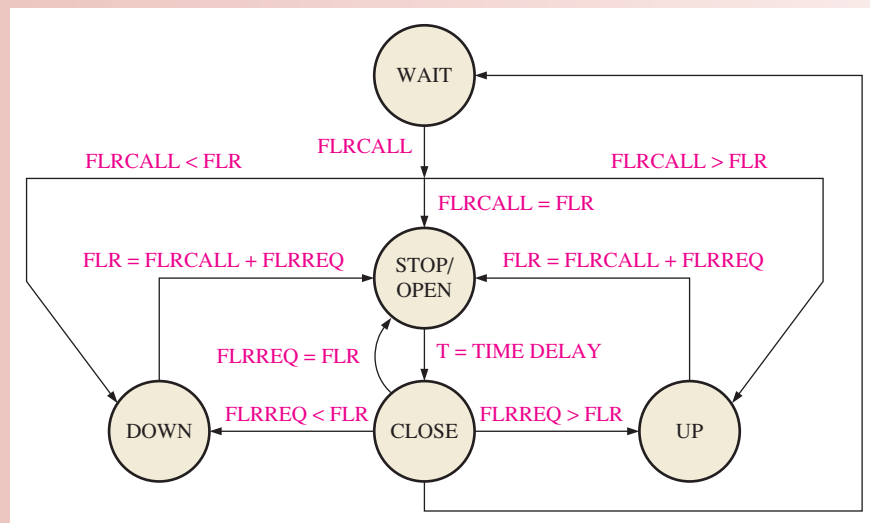


FIGURE 9–60 One cycle of the elevator operation.

The five states in the elevator control sequence are WAIT, DOWN, UP, STOP/OPEN, and CLOSE. In the WAIT state, the elevator is waiting on the last floor serviced for an external call button (FLRCALL) on any floor to be pressed. When there is a call for the elevator from any floor, the appropriate command (UP or DOWN) is issued. When the elevator arrives and stops at the calling floor, the door opens; the person enters and presses a number to request a destination floor. If the number of the requested floor is less than the number of the current floor, the elevator goes into the DOWN mode. If the number of the requested floor is greater than the number of the current floor, the elevator goes into the UP mode. The elevator goes to the STOP/OPEN mode at the requested floor to allow exit. After the door is open for a specified time, it closes and then goes back to the WAIT state until another floor call is received.

FIGURE 9-61 Elevator controller state diagram.

The following states are shown in the state diagram of Figure 9-61:

WAIT The system always begins in the WAIT state on the floor last serviced. When a floor call (FLRCALL) signal is received, the control logic determines if the number of the calling floor is greater than the current floor ($\text{FLRCALL} > \text{FLR}$), less than the current floor ($\text{FLRCALL} < \text{FLR}$), or equal to the current floor ($\text{FLRCALL} = \text{FLR}$) and puts the system in the UP mode, DOWN mode, or OPEN mode, respectively.

DOWN In this state, the elevator moves down toward the calling floor.

UP In this mode, the elevator moves up toward the calling floor.

STOP/OPEN This state occurs when the calling floor has been reached. When the number of the floor where the elevator is equals the number of the calling or requested floor, a signal is issued to stop the elevator and open the door.

CLOSE After a preset time (T) to allow entry or exit, the door closes.

The signals used by the elevator controller are defined as follows:

FLR Number of floor represented by a 3-bit binary code.

Floor sensor pulse A pulse issued at each floor to clock the floor counter to the next state.

FLRCALL Number of floor where a call for elevator service originates, represented by a 3-bit binary code.

Call pulse A pulse issued in conjunction with FLRCALL to clock the 3-bit code into a register.

FLRREQ Number of floor to which the passenger desires to go, represented by a 3-bit binary code.

Request pulse A pulse issued in conjunction with FLRREQ to clock the 3-bit code into a register.

UP A signal issued to the elevator motor control to cause the elevator to move from a lower floor to a higher floor.

DOWN A signal issued to the elevator motor control to cause the elevator to move from a higher floor to a lower floor.

STOP A signal issued to the elevator motor control to cause the elevator to stop.

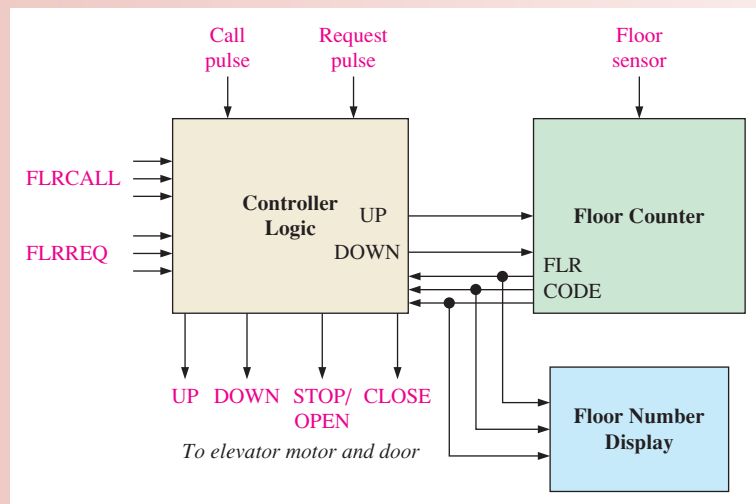
OPEN A signal issued to door motor control to cause the door to open.

CLOSE A signal issued to the door motor control to cause the door to close.

Elevator Controller Block Diagram

Figure 9-62 shows the elevator controller block diagram, which consists of controller logic, a floor counter, and a floor number display. Assume that the elevator is on the first floor in

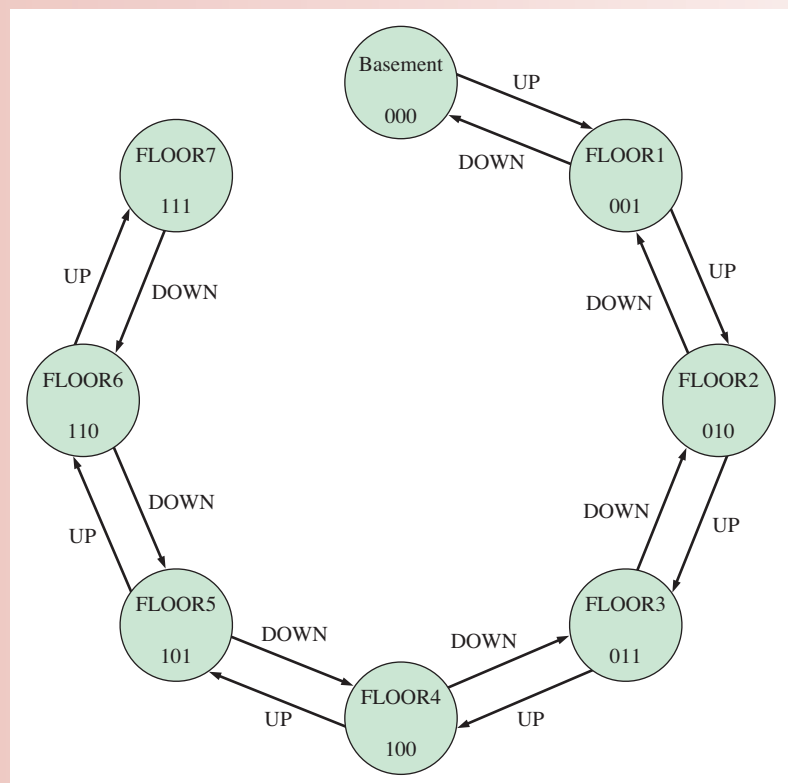
FIGURE 9-62 Elevator controller block diagram.



the WAIT state. The floor counter contains 001, which is the first floor code. Suppose the FLRCALL (101) comes in from the call button on the fifth floor. Since FLRCALL > FLR (101 > 001), the controller issues an UP command to the elevator motor. As the elevator moves up, the floor counter receives a floor sensor pulse as it reaches each floor which advances its state (001, 010, 011, 100, 101). When the fifth floor is reached and FLR = FLRCALL, the controller logic stops the elevator and opens its door. The process is repeated for a FLRREQ input.

The floor counter sequentially tracks the number of the floor and always contains the number of the current floor. It can count up or down and can reverse its state at any point under the direction of the state controller and the floor sensor input. A 3-bit counter is required since there are eight floors ($2^3 = 8$) including the basement, as shown in the floor counter state diagram in Figure 9-63.

FIGURE 9-63 Floor counter state diagram.



Operation of Elevator Controller

The elevator controller logic diagram is shown in Figure 9–64. Elevator action is initiated by either a floor call (FLRCALL) or a floor request (FLRREQ). Keep in mind that FLRCALL is when a person calls the elevator to come to a particular floor. FLRREQ is when a passenger in the elevator requests to go to a specified floor. This simplified operation is based on a CALL/REQ sequence; that is, a call followed by a request followed by a call.

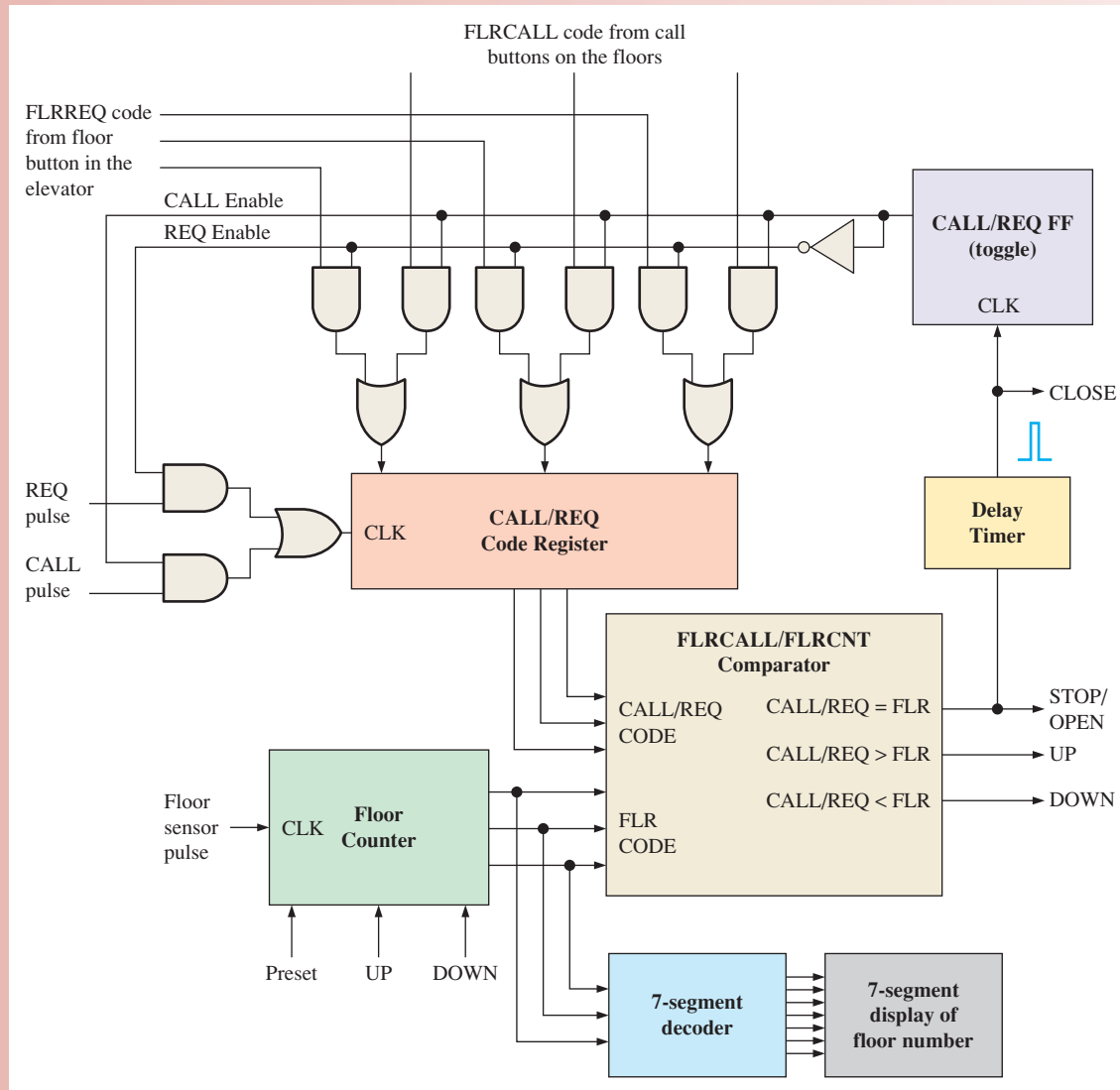


FIGURE 9–64 Elevator controller logic diagram.

As you know, FLRCALL and FLRREQ are 3-bit codes representing specific floors. When a person presses a call button on a given floor, the specific 3-bit code for that floor is placed on the inputs to the CALL/REQ code register and a CALL pulse is generated to enter the code into the register. The same process occurs when a request button is pressed inside the elevator. The code is input to the CALL/REQ code register, and a REQ pulse is generated to store the code in the register.

The elevator does not know the difference between a call and a request. The comparator determines if the destination floor number is greater than, less than, or equal to the current

floor where the elevator is located. As a result of this comparison, either an UP command, a DOWN command, or an OPEN command is issued to the elevator motor control. As the elevator moves toward the desired floor, the floor counter is either incremented at each floor as it goes up or decremented at each floor as it goes down. Once the elevator reaches the desired floor, a STOP/OPEN command is issued to the elevator motor control and to the door control. After a preset time, the delay timer issues a CLOSE signal to the elevator door control. As mentioned, this elevator design is limited to one floor call and one floor request per cycle.

Initialization The initial one-time setup requires that the elevator be placed at the basement level and the floor counter be preset to 000. After this, the counter will automatically move through the sequence of states determined by the elevator position.

Exercise

1. Explain the purpose of the floor counter.
2. Describe what happens during the WAIT mode.
3. How does the system know when the desired floor has been reached?
4. Discuss the limitations of the elevator design in Figure 9–64.

Implementation

The elevator controller can be implemented using fixed-function logic devices, a PLD programmed with a VHDL (or Verilog) code, or a programmed microcontroller or microprocessor. In the Chapter 10 Applied Logic, the VHDL program code for the elevator controller is presented. You will see how to program a PLD step by step.

Putting Your Knowledge to Work

What changes are required in the logic diagram of Figure 9–64 to upgrade the elevator controller for a ten-story building?

SUMMARY

- Asynchronous and synchronous counters differ only in the way in which they are clocked. The first stage of an asynchronous counter is driven by a clock pulse. Each succeeding stage is clocked by the output of the previous stage. In a synchronous counter, all stages are clocked by the same clock pulse. Synchronous counters can run at faster clock rates than asynchronous counters.
- The maximum modulus of a counter is the maximum number of possible states and is a function of the number of stages (flip-flops). Thus,

$$\text{Maximum modulus} = 2^n$$

where n is the number of stages in the counter. The modulus of a counter is the *actual* number of states in its sequence and can be equal to or less than the maximum modulus.

- The overall modulus of cascaded counters is equal to the product of the moduli of the individual counters.

KEY TERMS

Key terms and other bold terms in the chapter are defined in the end-of-book glossary.

Asynchronous Not occurring at the same time.

Cascade To connect “end-to-end” as when several counters are connected from the terminal count output of one counter to the enable input of the next counter.

00 00 00
00 00 10
00 11 11
11 11 00
11 11 11
11 01 01
01 01 01
01 10 00
10 01 00
01 01 11
01 00 11
00 10 11
10 10 01
10 00 01
00 11 10



Decade Characterized by ten states or values.

Modulus The number of unique states through which a counter will sequence.

Recycle To undergo transition (as in a counter) from the final or terminal state back to the initial state.

State diagram A graphic depiction of a sequence of states or values.

State machine A logic system or circuit exhibiting a sequence of states conditioned by internal logic and external inputs; any sequential circuit exhibiting a specified sequence of states. Two types of state machine are Moore and Mealy.

Synchronous Occurring at the same time.

Terminal count The final state in a counter's sequence.

TRUE/FALSE QUIZ

Answers are at the end of the chapter.

1. A state machine is a sequential circuit having a limited number of states occurring in a prescribed order.
2. Synchronous counters cannot be realized using J-K flip-flops.
3. An asynchronous counter is also known as a ripple counter.
4. A decade counter has twelve states.
5. A counter with four stages has a maximum modulus of sixteen.
6. To achieve a maximum modulus of 32, sixteen stages are required.
7. If the present state is 1000, the next state of a 4-bit up/down counter in the DOWN mode is 0111.
8. Two cascaded decade counters divide the clock frequency by 10.
9. A counter with a truncated sequence has less than its maximum number of states.
10. To achieve a modulus of 100, ten decade counters are required.

SELF-TEST

Answers are at the end of the chapter.

1. A Moore state machine consists of combinational logic circuits that determine
 - (a) sequences
 - (b) memory
 - (c) both (a) and (b)
 - (d) neither (a) nor (b)
2. The output of a Mealy machine depends on its
 - (a) inputs
 - (b) next state
 - (c) present state
 - (d) answers (a) and (c)
3. The maximum cumulative delay of an asynchronous counter must be
 - (a) more than the period of the clock waveform
 - (b) less than the period of the clock waveform
 - (c) equal to the period of the clock waveform
 - (d) both (a) and (c)
4. A decade counter with a count of zero (0000) through nine (1001) is known as
 - (a) an ASCII counter
 - (b) a binary counter
 - (c) A BCD counter
 - (d) a decimal counter
5. The modulus of a counter is
 - (a) the number of flip-flops
 - (b) the actual number of states in its sequence
 - (c) the number of times it recycles in a second
 - (d) the maximum possible number of states
6. A 3-bit binary counter has a maximum modulus of
 - (a) 3
 - (b) 6
 - (c) 8
 - (d) 16
7. A 5-bit binary counter has a maximum modulus of
 - (a) 4
 - (b) 8
 - (c) 16
 - (d) 32
8. A modulus-12 counter must have
 - (a) 12 flip-flops
 - (b) 3 flip-flops
 - (c) 4 flip-flops
 - (d) synchronous clocking

9. Which one of the following is an example of a counter with a truncated modulus?
 - (a) Modulus 8
 - (b) Modulus 14
 - (c) Modulus 16
 - (d) Modulus 32
10. A 4-bit ripple counter consists of flip-flops that each have a propagation delay from clock to Q output of 12 ns. For the counter to recycle from 1111 to 0000, it takes a total of
 - (a) 12 ns
 - (b) 24 ns
 - (c) 48 ns
 - (d) 36 ns
11. A BCD counter is an example of
 - (a) a full-modulus counter
 - (b) a decade counter
 - (c) a truncated-modulus counter
 - (d) answers (b) and (c)
12. Which of the following is a valid state in an 8421 BCD counter?
 - (a) 1010
 - (b) 1011
 - (c) 1111
 - (d) 1000
13. Three cascaded modulus-10 counters have an overall modulus of
 - (a) 30
 - (b) 100
 - (c) 1000
 - (d) 10,000
14. A 10 MHz clock frequency is applied to a cascaded counter consisting of a modulus-5 counter, a modulus-8 counter, and two modulus-10 counters. The lowest output frequency possible is
 - (a) 10 kHz
 - (b) 2.5 kHz
 - (c) 5 kHz
 - (d) 25 kHz
15. A 4-bit binary up/down counter is in the binary state of zero. The next state in the DOWN mode is
 - (a) 0001
 - (b) 1111
 - (c) 1000
 - (d) 1110
16. The initial count of a modulus-13 binary counter is
 - (a) 0000
 - (b) 1111
 - (c) 1101
 - (d) 1100

PROBLEMS

Answers to odd-numbered problems are at the end of the book.

Section 9-1 Finite State Machines

1. Represent a decade counter with the terminal state decoded as a state machine. Identify the type and show the block diagram and the state diagram.
2. Identify the type of state machine for the traffic signal controller in Chapter 6. State the reason why it is the type you specified.

Section 9-2 Asynchronous Counters

3. For the ripple counter shown in Figure 9-65, show the complete timing diagram for eight clock pulses, showing the clock, Q_0 , and Q_1 waveforms.

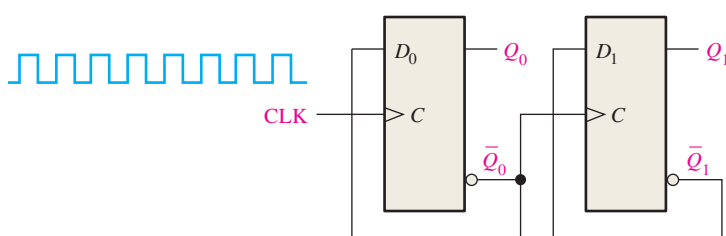


FIGURE 9-65

4. For the ripple counter in Figure 9-66, show the complete timing diagram for sixteen clock pulses. Show the clock, Q_0 , Q_1 , and Q_2 waveforms.

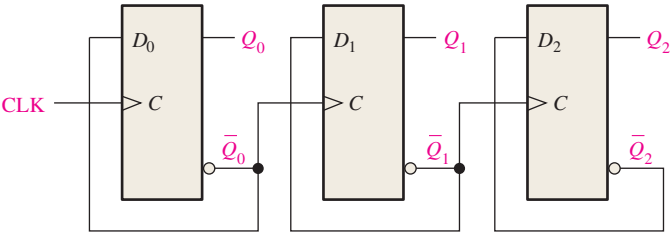


FIGURE 9-66

5. In the counter of Problem 4, assume that each flip-flop has a propagation delay from the triggering edge of the clock to a change in the Q output of 8 ns. Determine the worst-case (longest) delay time from a clock pulse to the arrival of the counter in a given state. Specify the state or states for which this worst-case delay occurs.
6. Show how to connect a 74HC93 4-bit asynchronous counter for each of the following moduli:
(a) 9 (b) 11 (c) 13 (d) 14 (e) 15

Section 9-3 Synchronous Counters

7. If the counter of Problem 5 were synchronous rather than asynchronous, what would be the longest delay time?
8. Show the complete timing diagram for the 5-stage synchronous binary counter in Figure 9-67. Verify that the waveforms of the Q outputs represent the proper binary number after each clock pulse.

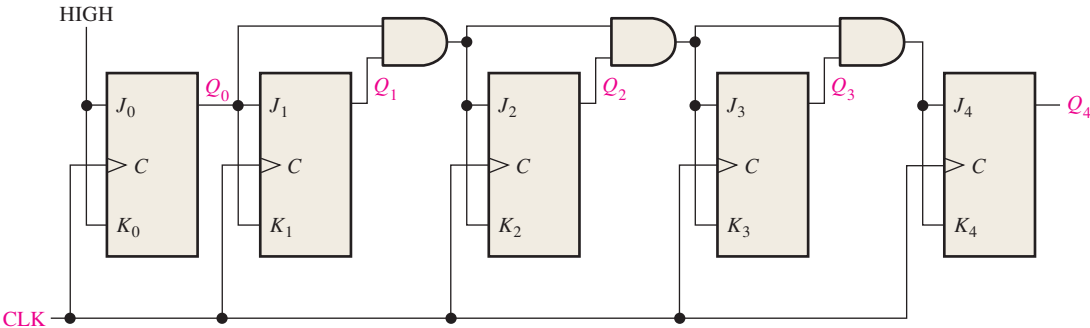


FIGURE 9-67

9. By analyzing the J and K inputs to each flip-flop prior to each clock pulse, prove that the decade counter in Figure 9-68 progresses through a BCD sequence. Explain how these conditions in each case cause the counter to go to the next proper state.

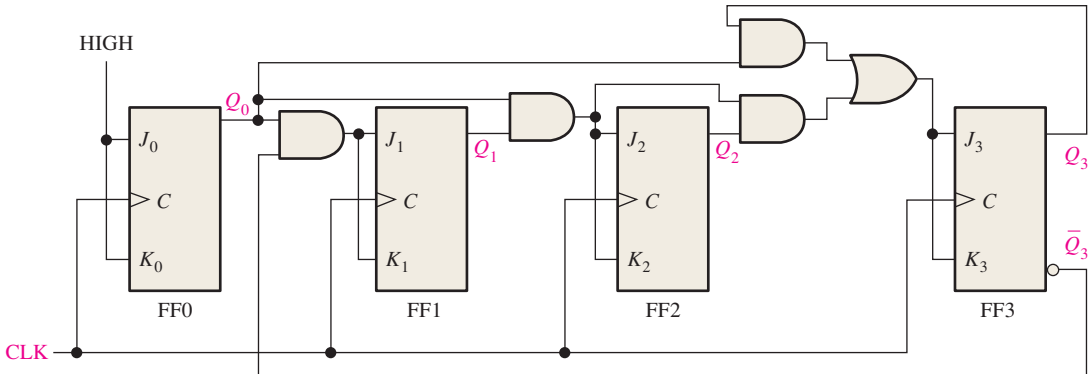


FIGURE 9-68

10. The waveforms in Figure 9–69 are applied to the count enable, clear, and clock inputs as indicated. Show the counter output waveforms in proper relation to these inputs. The clear input is asynchronous.

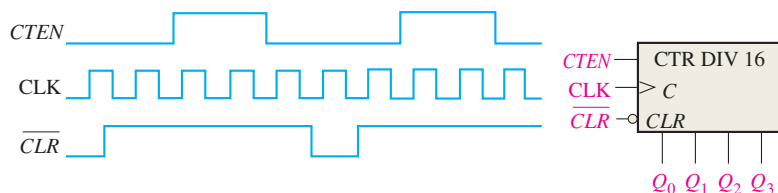


FIGURE 9–69

11. A BCD decade counter is shown in Figure 9–70. The waveforms are applied to the clock and clear inputs as indicated. Determine the waveforms for each of the counter outputs (Q_0 , Q_1 , Q_2 , and Q_3). The clear is synchronous, and the counter is initially in the binary 1000 state.

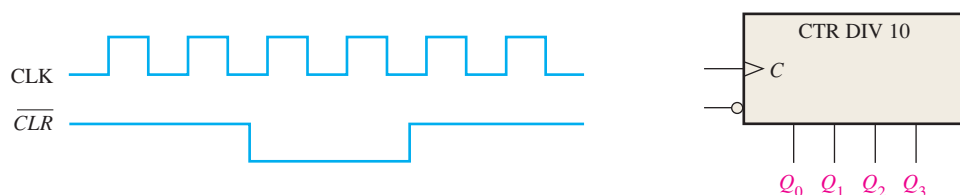


FIGURE 9–70

12. The waveforms in Figure 9–71 are applied to a 74HC163 binary counter. Determine the Q outputs and the RCO . The inputs are $D_0 = 1$, $D_1 = 1$, $D_2 = 0$, and $D_3 = 1$.

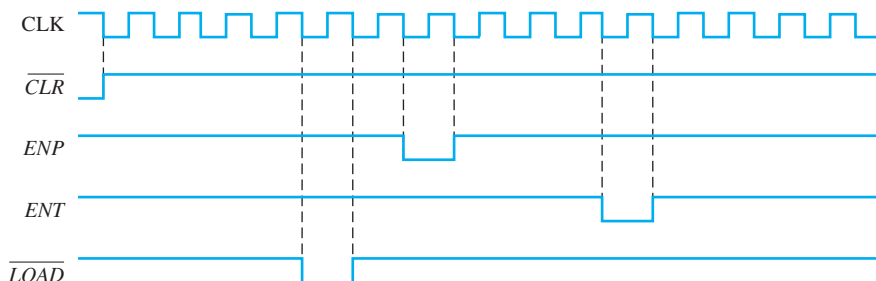


FIGURE 9–71

13. The waveforms in Figure 9–71 are applied to a 74HC161 counter. Determine the Q outputs and the RCO . The inputs are $D_0 = 1$, $D_1 = 0$, $D_2 = 0$, and $D_3 = 1$.

Section 9–4 Up/Down Synchronous Counters

14. Show a complete timing diagram for a 3-bit up/down counter that goes through the following sequence. Indicate when the counter is in the UP mode and when it is in the DOWN mode. Assume positive edge-triggering.

0, 1, 2, 3, 2, 1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1, 0

15. Develop the Q output waveforms for a 74HC190 up/down counter with the input waveforms shown in Figure 9–72. A binary 0 is on the data inputs. Start with a count of 0000.

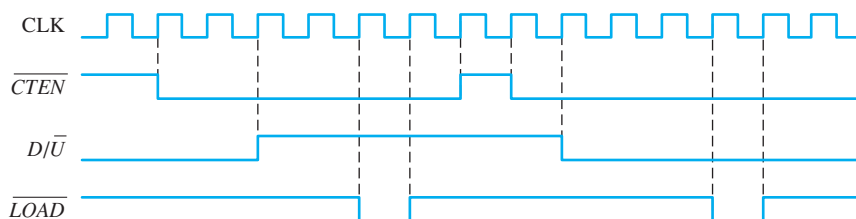


FIGURE 9–72

16. Repeat Problem 15 if the $D/\bar{U}$ input signal is inverted with the other inputs the same.
17. Repeat Problem 15 if the $\overline{CTEN}$ is inverted with the other inputs the same.

Section 9-5 Design of Synchronous Counters

18. Determine the sequence of the counter in Figure 9-73.

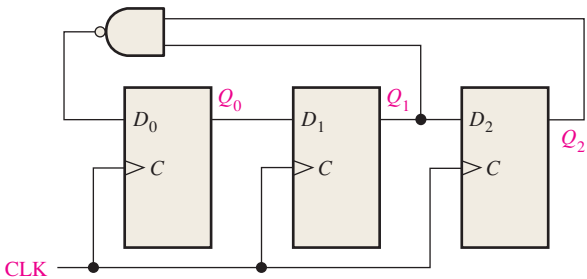


FIGURE 9-73

19. Determine the sequence of the counter in Figure 9-74. Begin with the counter cleared.

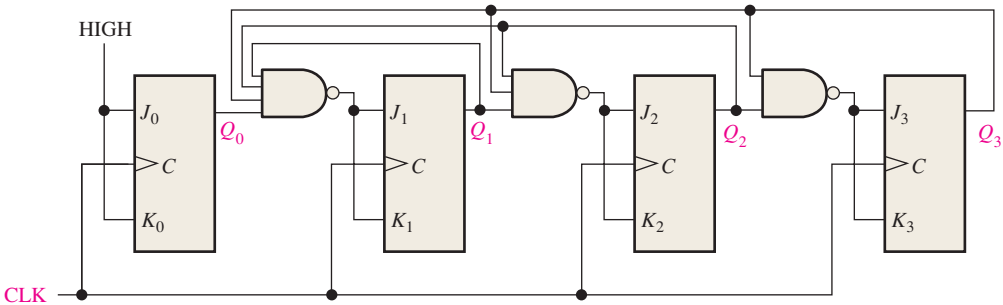


FIGURE 9-74

20. Design a counter to produce the following sequence. Use J-K flip-flops.
00, 10, 01, 11, 00, . . .
21. Design a counter to produce the following binary sequence. Use J-K flip-flops.
1, 4, 3, 5, 7, 6, 2, 1, . . .
22. Design a counter to produce the following binary sequence. Use J-K flip-flops.
0, 9, 1, 8, 2, 7, 3, 6, 4, 5, 0, . . .
23. Design a binary counter with the sequence shown in the state diagram of Figure 9-75.

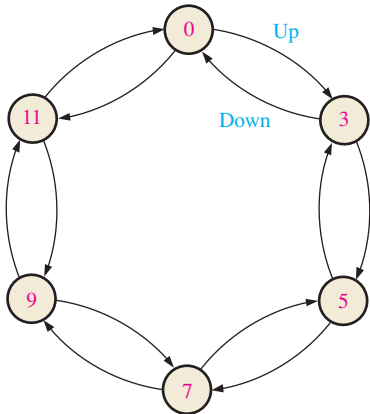


FIGURE 9-75

Section 9-6 Cascaded Counters

24. For each of the cascaded counter configurations in Figure 9-76, determine the frequency of the waveform at each point indicated by a circled number, and determine the overall modulus.

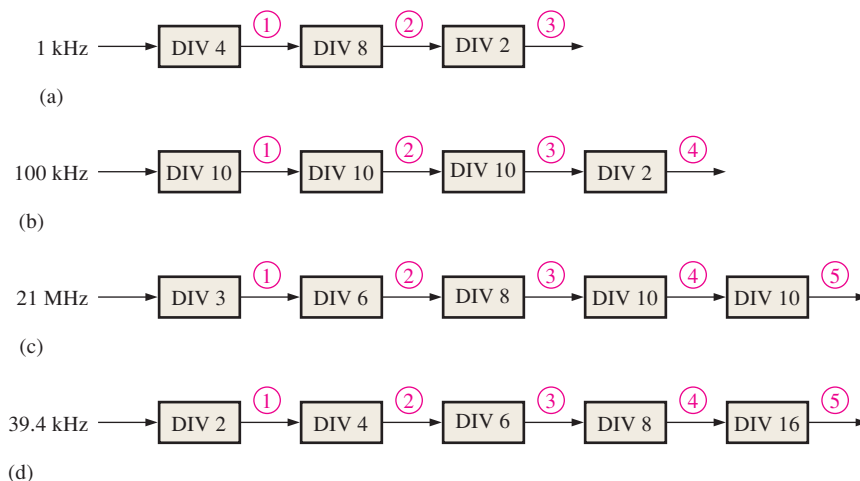


FIGURE 9-76

25. Expand the counter in Figure 9-38 to create a divide-by-10,000 counter and a divide-by-100,000 counter.
26. With general block diagrams, show how to obtain the following frequencies from a 10 MHz clock by using single flip-flops, modulus-5 counters, and decade counters:
- (a) 5 MHz (b) 2.5 MHz (c) 2 MHz (d) 1 MHz (e) 500 kHz
 (f) 250 kHz (g) 62.5 kHz (h) 40 kHz (i) 10 kHz (j) 1 kHz

Section 9-7 Counter Decoding

27. Given a BCD decode counter with only the Q outputs available, show what decoding logic is required to decode each of the following states and how it should be connected to the counter. A HIGH output indication is required for each decoded state. The MSB is to the left.
- (a) 0001 (b) 0011 (c) 0101 (d) 0111 (e) 1000
28. For the 4-bit binary counter connected to the decoder in Figure 9-77, determine each of the decoder output waveforms in relation to the clock pulses.

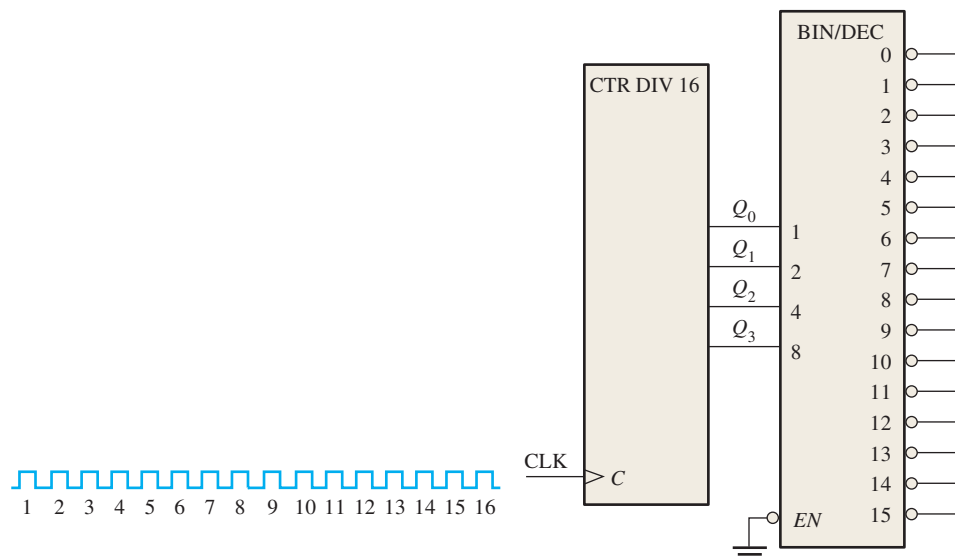


FIGURE 9-77

29. If the counter in Figure 9-77 is asynchronous, determine where the decoding glitches occur on the decoder output waveforms.
30. Modify the circuit in Figure 9-77 to eliminate decoding glitches.
31. Analyze the counter in Figure 9-42 for the occurrence of glitches on the decode gate output. If glitches occur, suggest a way to eliminate them.
32. Analyze the counter in Figure 9-43 for the occurrence of glitches on the outputs of the decoding gates. If glitches occur, make a design change that will eliminate them.

Section 9-8 Counter Applications

33. Assume that the digital clock of Figure 9-48 is initially reset to 12 o'clock. Determine the binary state of each counter after sixty-two 60 Hz pulses have occurred.
34. What is the output frequency of each counter in the digital clock circuit of Figure 9-48?
35. For the automobile parking control system in Figure 9-51, a pattern of entrance and exit sensor pulses during a given 24-hour period are shown in Figure 9-78. If there were 53 cars already in the garage at the beginning of the period, what is the state of the counter at the end of the 24 hours?

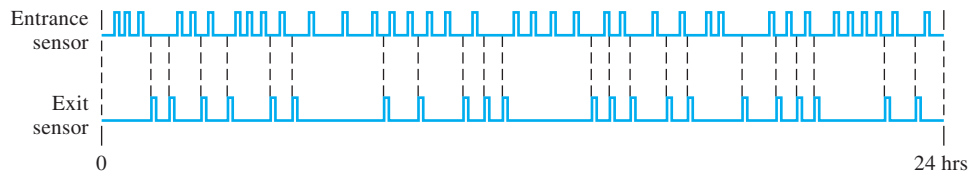


FIGURE 9-78

36. The binary number for decimal 57 appears on the parallel data inputs of the parallel-to-serial converter in Figure 9-53 (D_0 is the LSB). The counter initially contains all zeros and a 10 kHz clock is applied. Develop the timing diagram showing the clock, the counter outputs, and the serial data output.

Section 9-10 Troubleshooting

37. For the counter in Figure 9-4, show the timing diagram for the Q_0 and Q_1 waveforms for each of the following faults (assume Q_0 and Q_1 are initially LOW):
 - (a) clock input to FF0 shorted to ground
 - (b) Q_0 output open
 - (c) clock input to FF1 open
 - (d) D input to FF0 open
 - (e) D input to FF1 shorted to ground
38. Solve Problem 37 for the counter in Figure 9-12(b).
39. Isolate the fault in the counter in Figure 9-6 by analyzing the waveforms in Figure 9-79.
40. From the waveform diagram in Figure 9-80, determine the most likely fault in the counter of Figure 9-15.

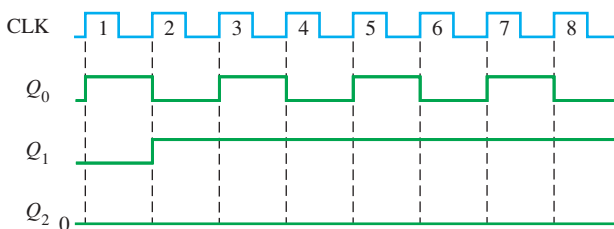


FIGURE 9-79

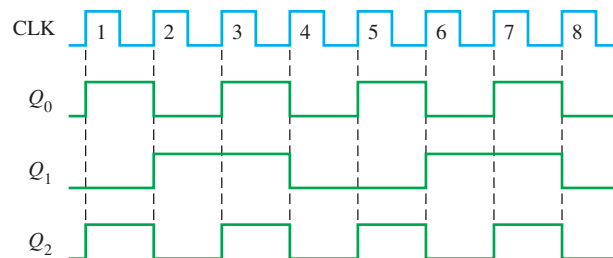


FIGURE 9-80

41. Solve Problem 40 if the Q_2 output has the waveform observed in Figure 9–81. Outputs Q_0 and Q_1 are the same as in Figure 9–80.

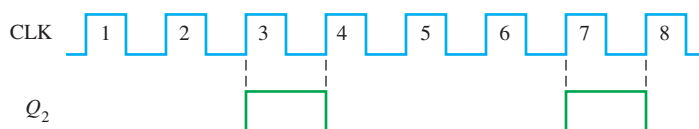


FIGURE 9–81

42. You apply a 5 MHz clock to the cascaded counter in Figure 9–41 and measure a frequency of 76.2939 Hz at the last RCO output. Is this correct, and if not, what is the most likely problem?
43. Develop a table for use in testing the counter in Figure 9–41 that will show the frequency at the final RCO output for all possible open failures of the parallel data inputs (D_0 , D_1 , D_2 , and D_3) taken one at a time. Use 10 MHz as the test frequency for the clock.
44. The tens-of-hours 7-segment display in the digital clock system of Figure 9–48 continuously displays a 1. All the other digits work properly. What could be the problem?
45. What would be the visual indication of an open Q_1 output in the tens portion of the minutes counter in Figure 9–48? Also see Figure 9–49.
46. One day (perhaps a Monday) complaints begin flooding in from patrons of a parking garage that uses the control system depicted in Figures 9–51 and 9–52. The patrons say that they enter the garage because the gate is up and the FULL sign is off but that, once in, they can find no empty space. As the technician in charge of this facility, what do you think the problem is, and how will you troubleshoot and repair the system as quickly as possible?

Applied Logic

47. Propose a general design for generation of the 3-bit FLRCALL code and the Call pulse by the pressing of a single button.
48. Propose a general design for generation of the 3-bit FLRREQ code and the Request pulse by the pressing of one of seven buttons.
49. What changes are required to the logic diagram in Figure 9–64 to modify the elevator controller for a four-story building?

Special Design Problems

50. Design a modulus-1000 counter by using decade counters.
51. Modify the design of the counter in Figure 9–41 to achieve a modulus of 30,000.
52. Repeat Problem 51 for a modulus of 50,000.
53. Modify the digital clock in Figures 9–48, 9–49, and 9–50 so that it can be preset to any desired time.
54. Design an alarm circuit for the digital clock that can detect a predetermined time (hours and minutes only) and produce a signal to activate an audio alarm.
55. Modify the design of the circuit in Figure 9–52 for a 1000-space parking garage and a 3000-space parking garage.
56. Implement the parallel-to-serial data conversion logic in Figure 9–53 with specific fixed-function devices.
57. In Problem 19 it was found that the counter locks up and alternates between two states. It turns out that this operation is the result of a design flaw. Redesign the counter so that when it goes into the second of the lock-up states, it will recycle to the all-0s state on the next clock pulse.

Multisim Troubleshooting Practice

58. Open file P09-58. For the specified fault, predict the effect on the circuit. Then introduce the fault and verify whether your prediction is correct.
59. Open file P09-59. For the specified fault, predict the effect on the circuit. Then introduce the fault and verify whether your prediction is correct.
60. Open file P09-60. For the specified fault, predict the effect on the circuit. Then introduce the fault and verify whether your prediction is correct.



61. Open file P09-61. For the observed behavior indicated, predict the fault in the circuit. Then introduce the suspected fault and verify whether your prediction is correct.
62. Open file P09-62. For the observed behavior indicated, predict the fault in the circuit. Then introduce the suspected fault and verify whether your prediction is correct.

ANSWERS

SECTION CHECKUPS

Section 9-1 Checkup

1. A finite state machine is a sequential circuit having a finite number of states that occur in a specified order.
2. Moore state machine and Mealy state machine
3. The Moore state machine has an output(s) that is dependent on the present internal state only. The Mealy state machine has an output(s) that is dependent on both the present internal state and the value of the inputs.

Section 9-2 Asynchronous Counters

1. Asynchronous means that each flip-flop after the first one is enabled by the output of the preceding flip-flop.
2. A modulus-14 counter has fourteen states requiring four flip-flops.

Section 9-3 Synchronous Counters

1. All flip-flops in a synchronous counter are clocked simultaneously.
2. The counter can be preset (initialized) to any given state.
3. Counter is enabled when *ENP* and *ENT* are both HIGH; *RCO* goes HIGH when final state in sequence is reached.

Section 9-4 Up/Down Synchronous Counters

1. The counter goes to 1001.
2. UP: 1111; DOWN: 0000; the next state is 1111.

Section 9-5 Design of Synchronous Counters

1. $J = 1$, $K = X$ ("don't care")
2. $J = X$ ("don't care"), $K = 0$
3. (a) The next state is 1011.
(b) Q_3 (MSB): no-change or SET; Q_2 : no-change or RESET; Q_1 : no change or SET; Q_0 (LSB): SET or toggle

Section 9-6 Cascaded Counters

1. Three decade counters produce $\div 1000$; 4 decade counters produce $\div 10,000$.
2. (a) $\div 20$: flip-flop and DIV 10
(b) $\div 32$: flip-flop and DIV 16
(c) $\div 160$: DIV 16 and DIV 10
(d) $\div 320$: DIV 16 and DIV 10 and flip-flop

Section 9-7 Counter Decoding

1. (a) No transitional states because there is a single bit change
(b) 0000, 0001, 0010, 0101, 0110, 0111
(c) No transitional states because there is a single bit change
(d) 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, 1011, 1100, 1101, 1110

Section 9-8 Counter Applications

1. Gate G_1 resets flip-flop on first clock pulse after count 9. Gate G_2 decodes count 12 to preset counter to 0001.
2. The hours decade counter advances through each state from zero to nine, and as it recycles from nine back to zero, the flip-flop is toggled to the SET state. This produces a ten (10) on the display. When the hours decade counter is in state 12, the decode NAND gate causes the counter to recycle to state 1 on the next clock pulse. The flip-flop resets. This results in a one (01) on the display.

Section 9-9 Logic Symbols with Dependency Notation

1. C : control, usually clock; M : mode; G : AND
2. D indicates data storage.

Section 9-10 Troubleshooting

1. No pulses on TC outputs: $CTEN$ of first counter shorted to ground or to a LOW; clock input of first counter open; clock line shorted to ground or to a LOW; TC output of first counter shorted to ground or to a LOW.
2. With inverter output open, the counter does not recycle at the preset count but acts as a full-modulus counter.

RELATED PROBLEMS FOR EXAMPLES

9-1 See Figure 9-82.

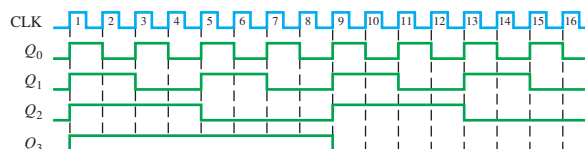


FIGURE 9-82

9-2 Connect Q_0 to the NAND gate as a third input (Q_2 and Q_3 are two of the inputs). Connect the $\overline{CLR}$ line to the $\overline{CLR}$ input of FF0 as well as FF2 and FF3.

9-3 See Figure 9-83.

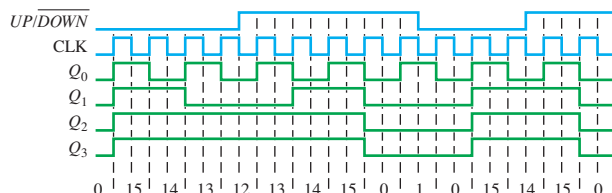


FIGURE 9-83

9-4 See Table 9-14.

TABLE 9-14

Present Invalid State			D Inputs			Next State			
Q_2	Q_1	Q_0	D_2	D_1	D_0	Q_2	Q_1	Q_0	
0	0	0	1	1	1	1	1	1	valid state
0	1	1	0	0	0	0	0	0	
1	0	0	1	1	1	1	1	1	
1	1	0	1	0	1	1	0	1	valid state

000 $\rightarrow$ 111

011 $\rightarrow$ 000 $\rightarrow$ 111

100 $\rightarrow$ 111

110 $\rightarrow$ 101

- 9-5** Three flip-flops, sixteen 3-input AND gates, two 4-input OR gates, four 2-input OR gates, and one inverter
- 9-6** Five decade counters are required. $10^5 = 100,000$
- 9-7** $f_{Q0} = 1 \text{ MHz}/[(10)(2)] = 50 \text{ kHz}$
- 9-8** See Figure 9-84.
- 9-9** $8AC0_{16}$ would be loaded. $16^4 - 8AC0_{16} = 65,536 - 32,520 = 30,016$
 $f_{TC4} = 10 \text{ MHz}/30,016 = 333.2 \text{ Hz}$
- 9-10** See Figure 9-85.

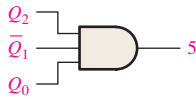


FIGURE 9-84

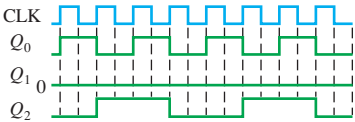


FIGURE 9-85

TRUE/FALSE QUIZ

1. T 2. F 3. T 4. F 5. T 6. F 7. T 8. F 9. T 10. F

SELF-TEST

1. (c) 2. (a) 3. (b) 4. (c) 5. (b) 6. (c) 7. (d) 8. (c)
9. (b) 10. (c) 11. (d) 12. (d) 13. (c) 14. (b) 15. (b) 16. (a)